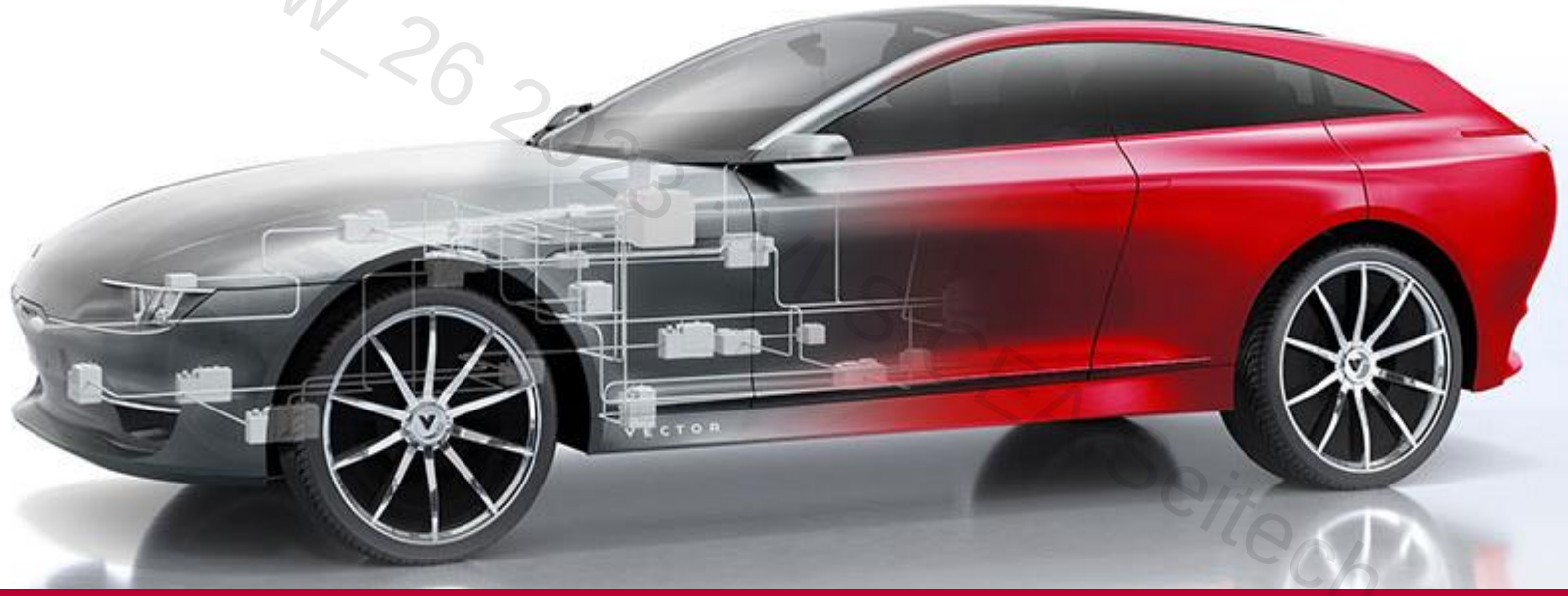
VECTOR



cw_26 2023 Mo 25 Juni Seitech

AUTOSAR

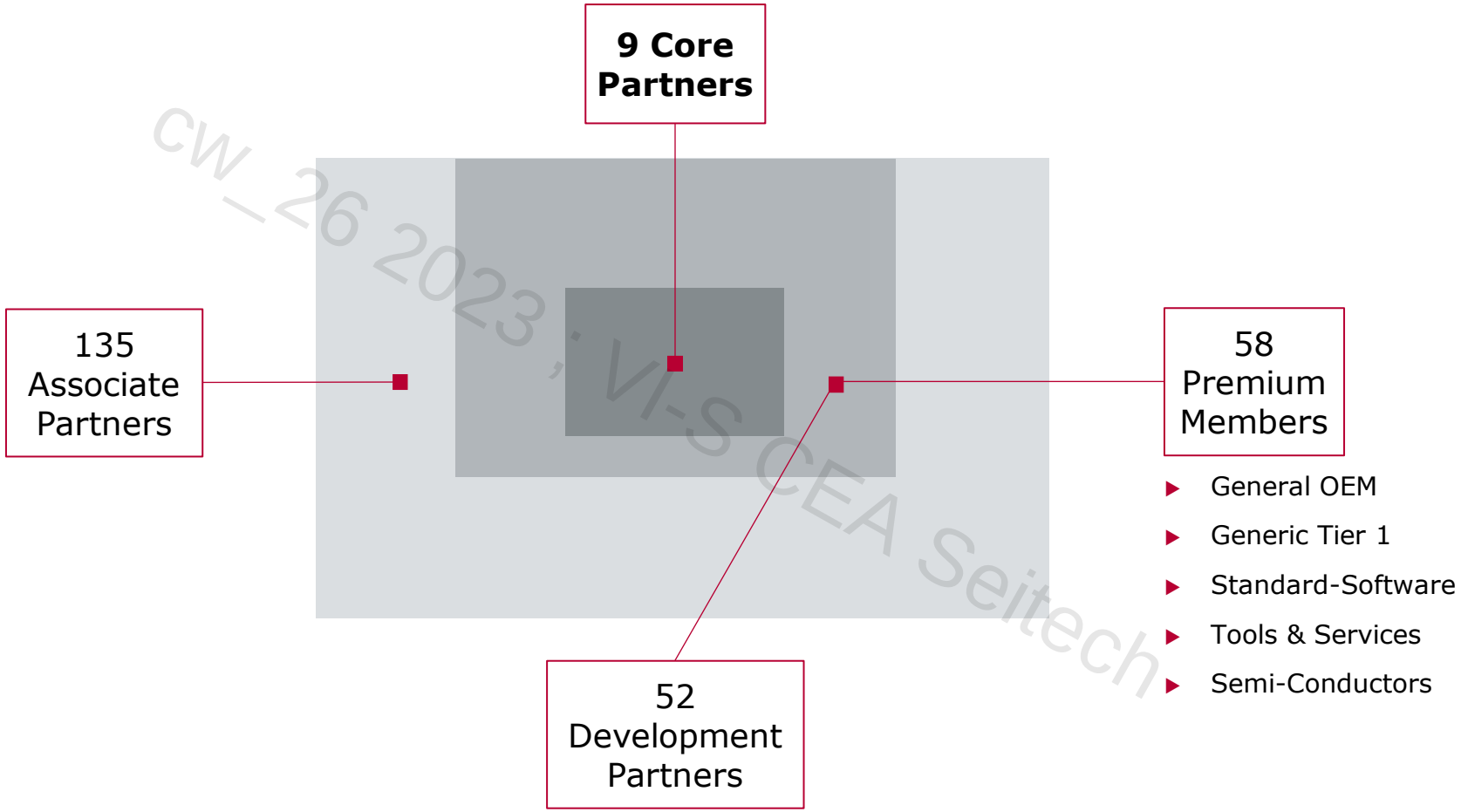
Fundamentals

Agenda

- **Overview and Objectives**
- AUTOSAR Application
- AUTOSAR RTE
- AUTOSAR BSW
- AUTOSAR Methodology
- AUTOSAR in Practice
- Appendix

cw_26 2023 ; VI-S CEA Seitech

AUTOSAR Partnership



- General OEM
- Generic Tier 1
- Standard-Software
- Tools & Services
- Semi-Conductors

cw_26 2023 ; VI-S CEA Seitech

2019.11.28: up-to-date status see www.autosar.org

AUTOSAR Partnership

Core Partners:

BMW GROUP, BOSCH, Continental, DAIMLER, Ford, GM, PSA GROUPE, TOYOTA, VOLKSWAGEN

cw_26 2023 ,VI-SCEA Seitech

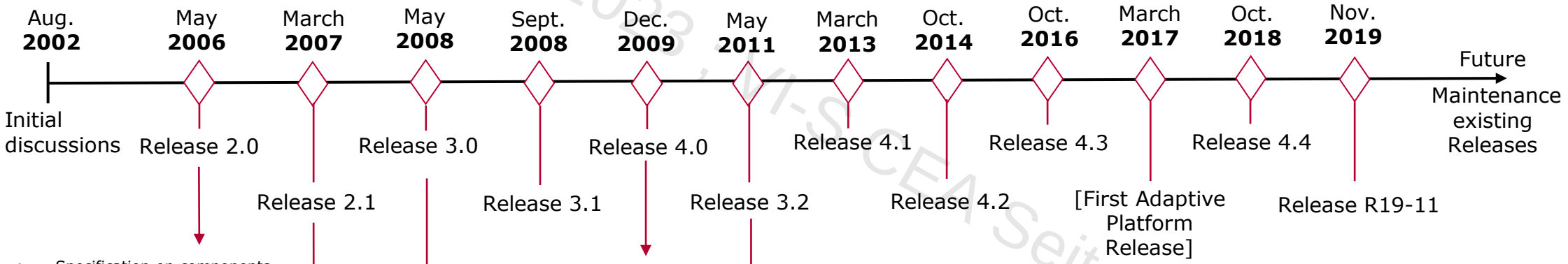
58 Premium Members:

ADIT Corporation	Altran	Aptiv	APTJ	Honda Motor	Huawei Technologies Co. Ltd.	Hyundai	IAV	NXP Semiconductors B.V.	Panasonic Corporation	Renault SAS	RENESAS Electronics Corp.
Arccore	Assystem S.A.	Autoliv	Baidu	Infineon Technologies AG	Intel	ITK Engineering	iSOFT	SCSK Corporation	Sodius SAS	ST Microelectronics	Sumitomo Electric Industries, Ltd.
CEA	Dassault Systemes	Deloitte GmbH	dSPACE	ixia	JTEKT Corporation	KPIT Technologies Ltd.	Larsen and Toubro Limited	TATA Consultancy Services Ltd.	Tata Elxsi Ltd.	Tata Motors	ThyssenKrupp
Elektrobit	eSol	ETAS	Fraunhofer-Gesellschaft	LEAR Corporation	Luxoft Holding Inc.	The MathWorks	Mentor Graphics Corporation	Valeo SAS	Vector	AB Volvo	Volvo Car Corporation
Great Wall Motor	Green Hills Software	HCL Technologies Limited	Hella	Mitsubishi Electric Corporation	NEC Corporation	NISSAN Motor Company	NVIDIA Corporation	Wind River Systems, Inc.	ZF Friedrichshafen AG		

+ 52 Development Partners + 135 Associate Partners

up-to-date status see www.autosar.org

History



- Specification on components (BSW and RTE)

- includes specifications on development methodology and templates
- first descriptions for application interfaces

- Specifications of standardized application interfaces (powertrain, chassis domain)
- harmonized the concept for wakeup of ECUs and startup of networks
- state managers for the bus systems CAN, LIN, and FlexRay added

- Functional Safety
- Architectural Improvements
- RTE enhancement
- COM
- [...]

- Improvement of error handling
- Back-porting of AUTOSAR 4.0 features into AUTOSAR 3.2

cw_26 2023 ; MI-S CEA Seitech

The Situation Today

- Growing E/E **complexity**

- Quantity of functions implemented in **software** rising very steeply (e.g. in the field of driver assistance systems)

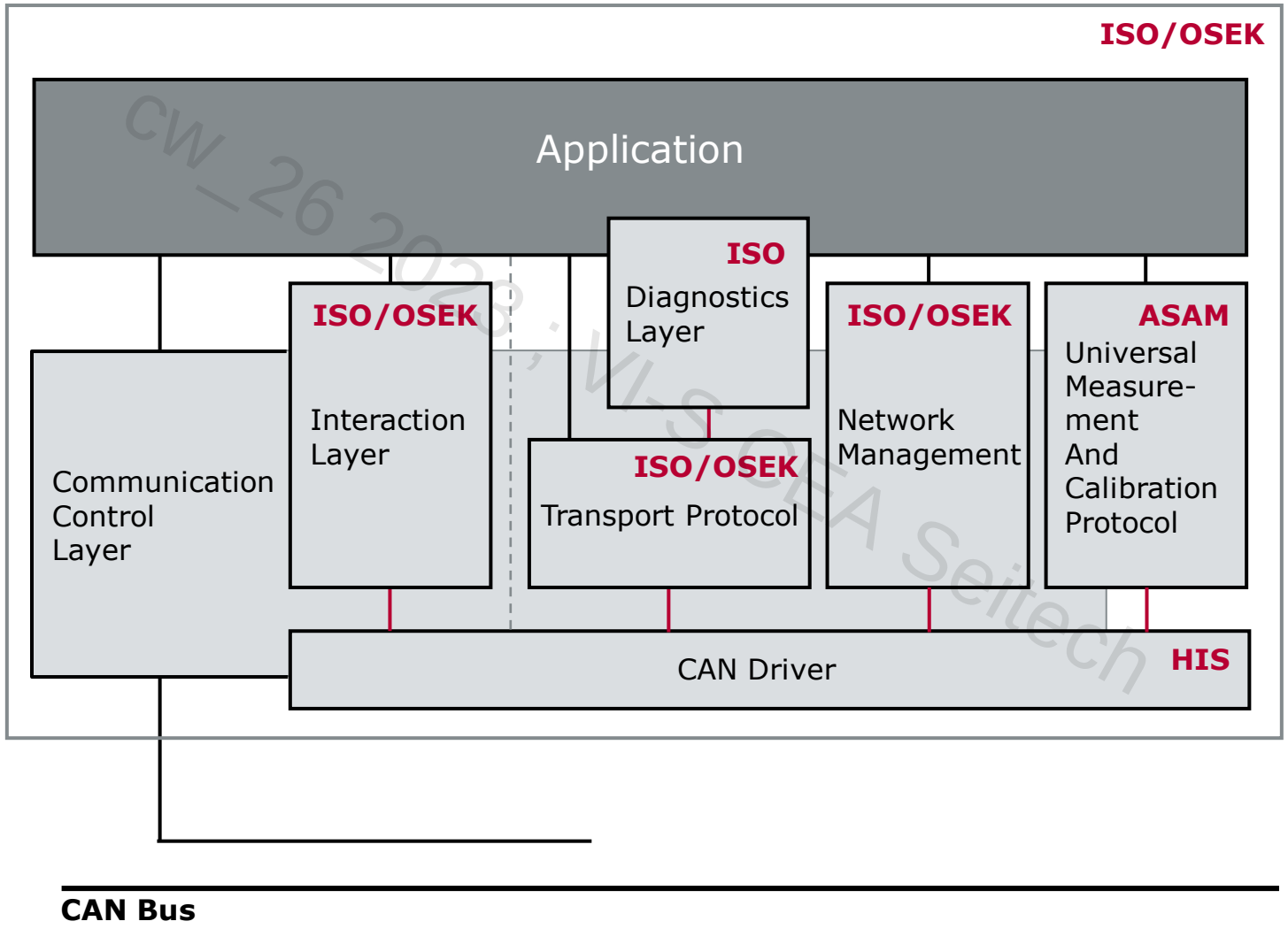
- Many different **hardware platforms** are used
  - Embedded systems traditionally do not support full **hardware abstraction**
  - Limited **modularity** of the software
  - Poor **reusability**: Software must often be rewritten from scratch when hardware (processor type) is changed

- **Life cycle shortfall**: The life of cars is usually longer than that of their ECUs (need for sustainable spare parts, especially software for maintenance reasons)

- **Variability:** Suppliers must support a large variety of OEMs and vehicle platform variants with their software

CW_RS_2025_MS CEA Series

The Situation Before AUTOSAR



cw_26 2020 ; VI-SOFA Seitech

VECTOR

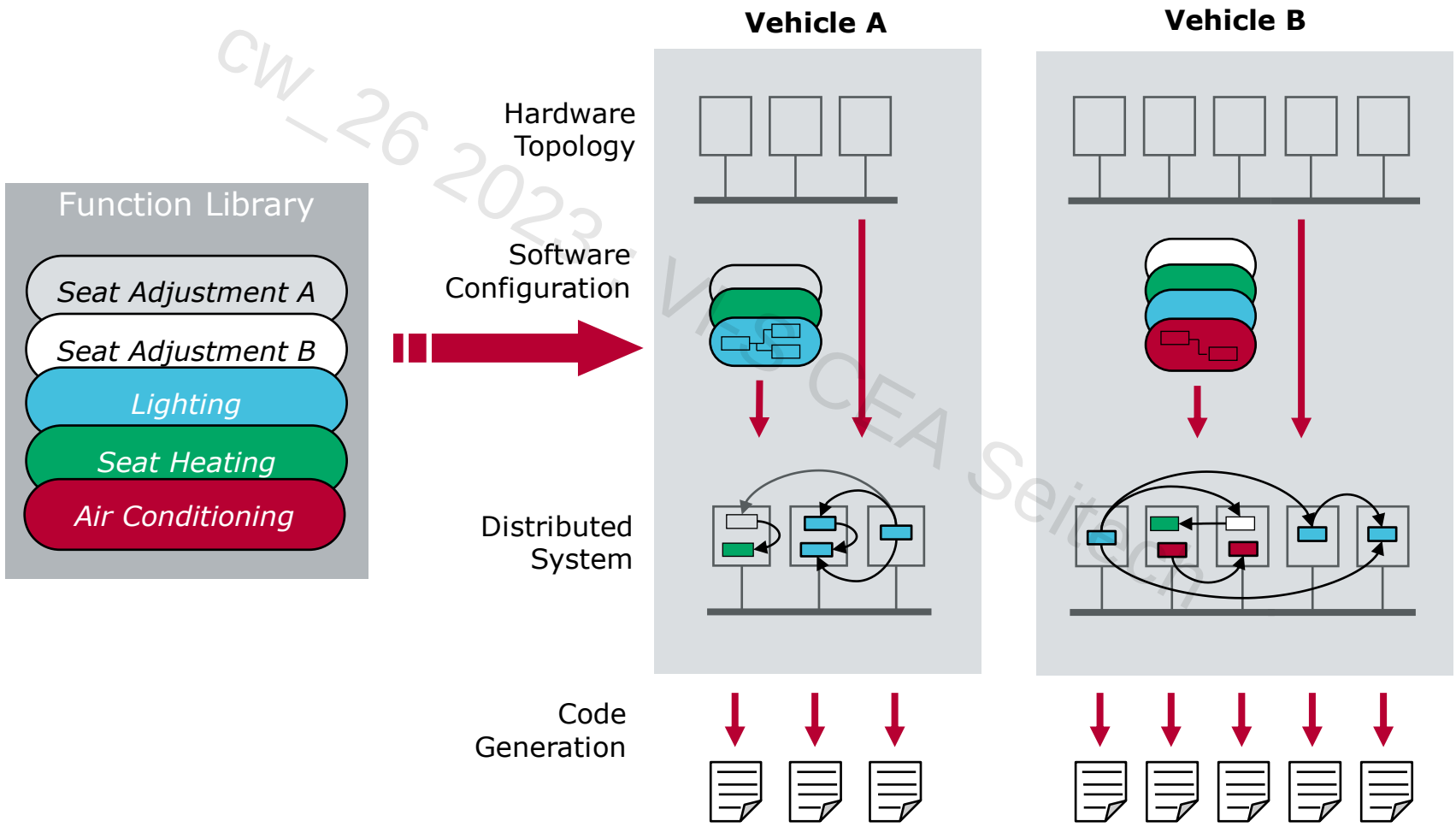
Objectives

Serviceability	**Serviceability** over the entire product life cycle Software updates and upgrades over the entire life of a vehicle.
Abstraction	**Abstraction** of hardware from software, making development more flexible.
Configuration	Shift development activities from implementation to **configuration** Standardized AUTOSAR **code generation/modeling tools**
Software Quality	Improvement in **software quality** by standardized BSW
Competition	**Competition** is focused onto OEM-relevant features
Reusability	**Reusability** of functions across vehicle networks and across OEM boundaries.

cw_26_2023 . VIS CEA Seitech

AUTOSAR Objectives

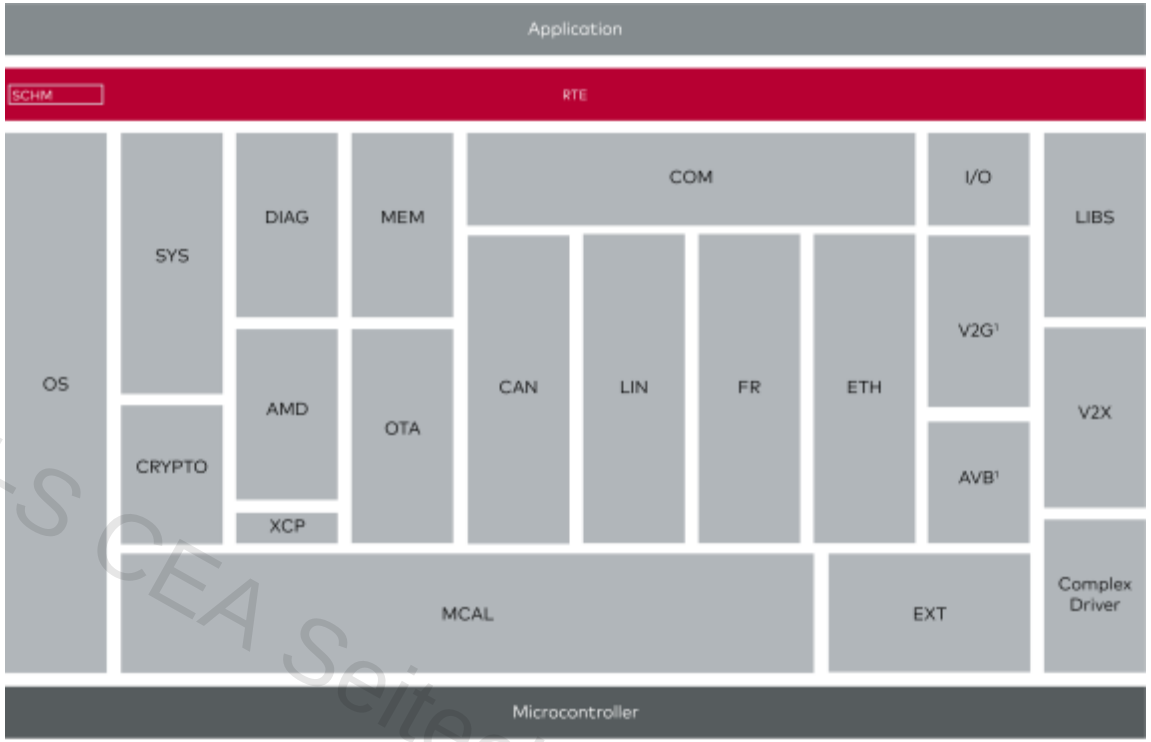
- **Reusability** of functions over different vehicles.



cw_26 2022 VW CEA Seitz

Objectives

AUTOmotive **O**pen **S**ystem **AR**chitecture



- **Standardization**
  - of interfaces
  - of exchange formats
  - of methodology

- Implementation and standardization of fundamental system functions as the cross-OEM "**basic software stack**".
- **Integration** of functional modules from different suppliers.
- **Scalability** to different vehicle and platform variants.

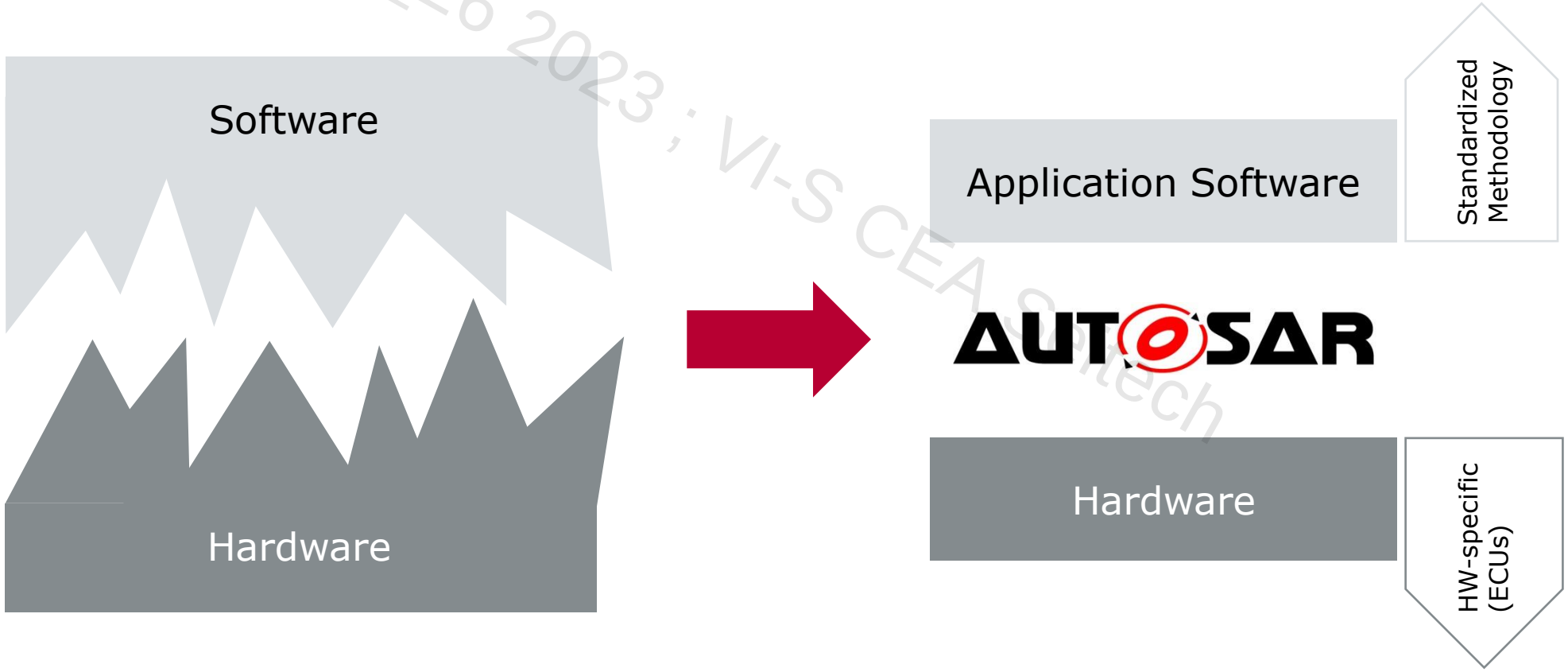
CV_26 2023 ; VI-S CEA Seitech

AUTOSAR Slogan

The AUTOSAR statement:

“Cooperate on standards – compete on implementation”

cw_26 2023 ; VI-S CEA Sotech



Agenda

Overview and Objectives

- **AUTOSAR Application**

AUTOSAR RTE

AUTOSAR BSW

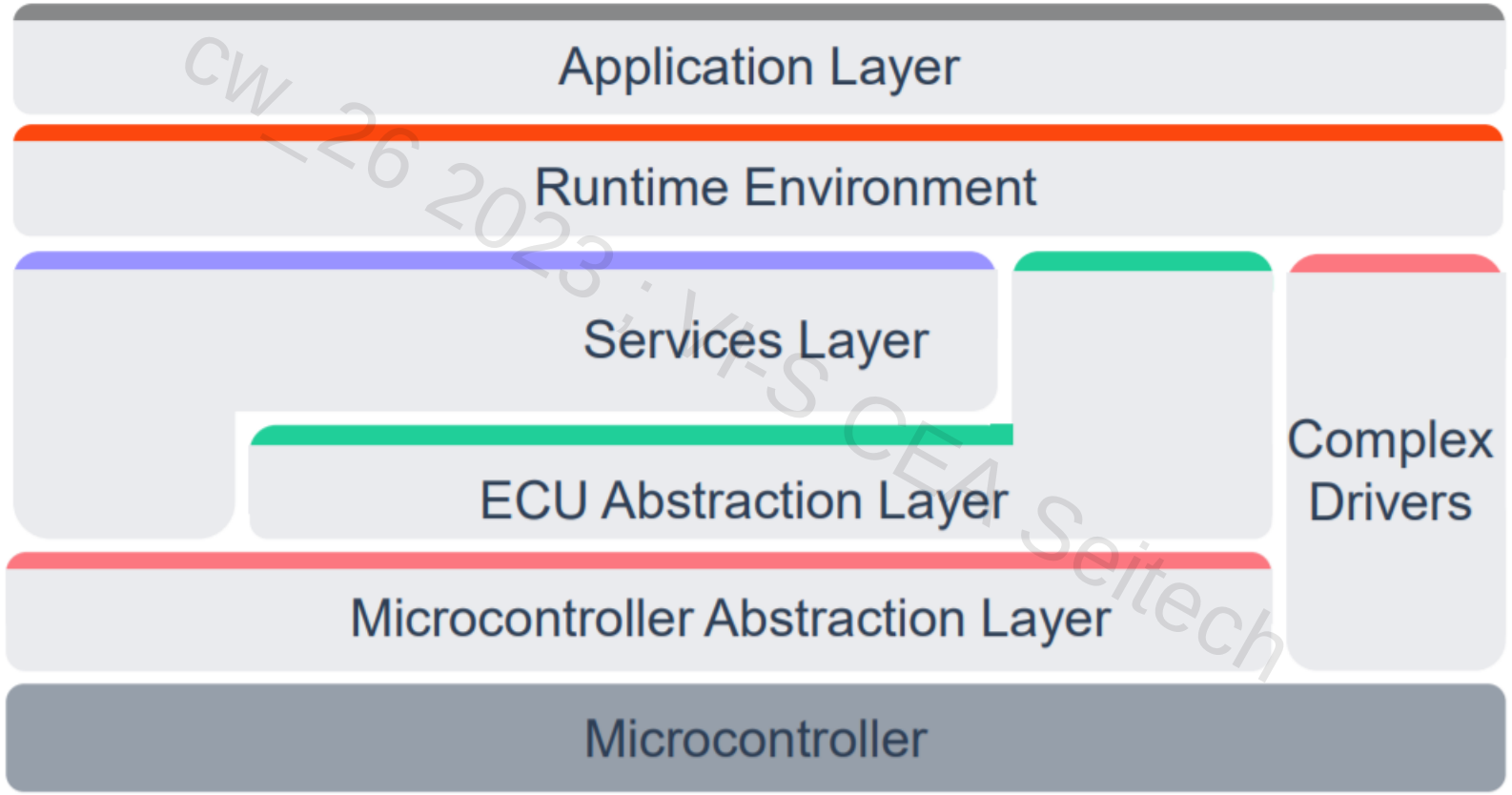
AUTOSAR Methodology

AUTOSAR in Practice

Appendix

cw_26 2023 ; VI-S CEA Seitech

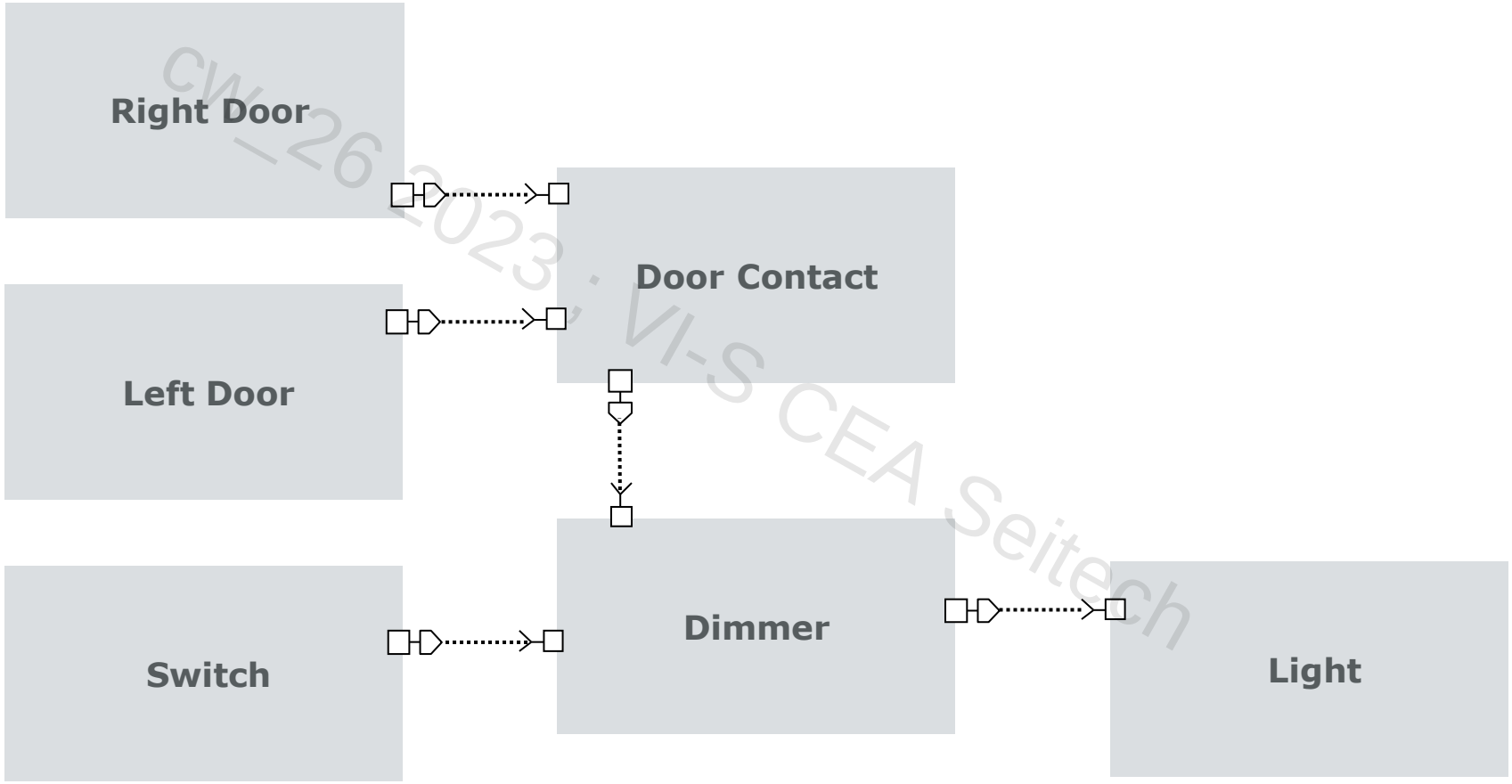
Architecture – Overview of Software Layers



cw_26 2023 : VRS CEA Seitech

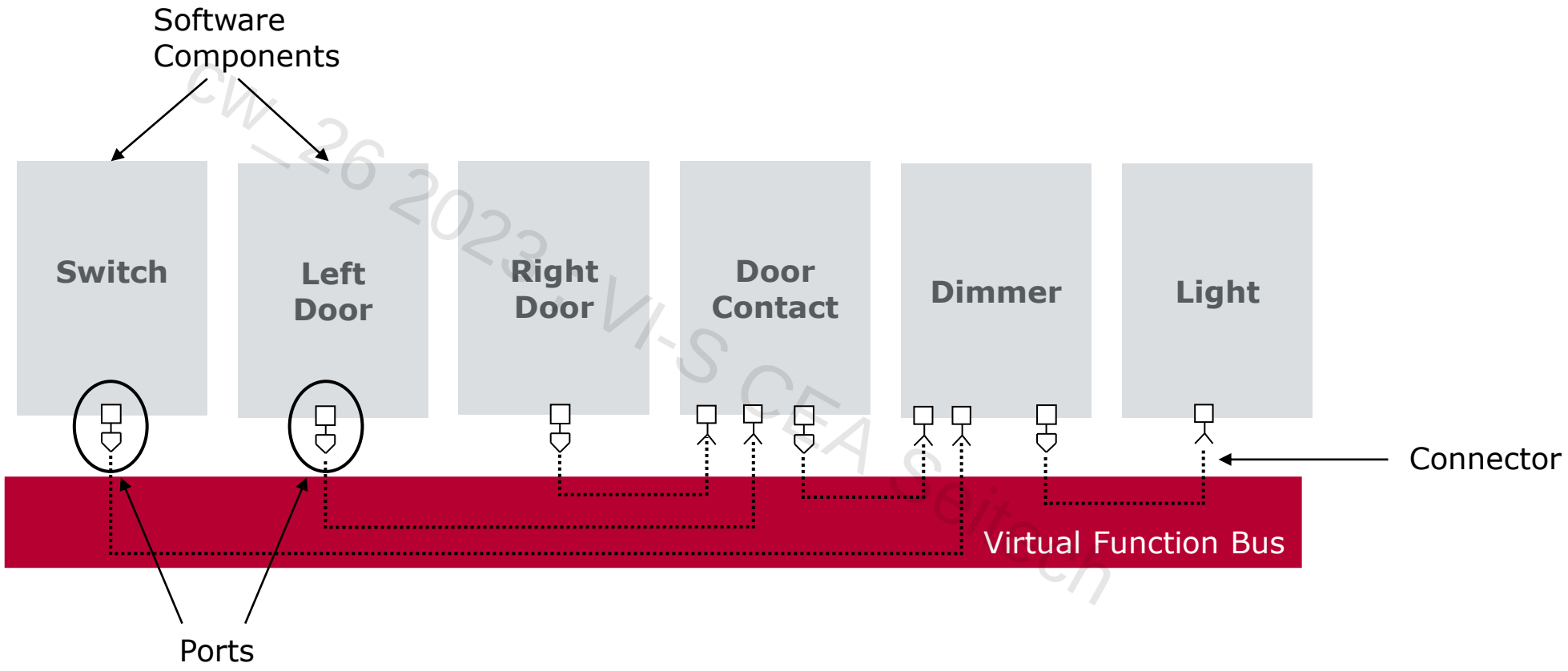
Source: http://www.autosar.org

Components View for Lighting Control



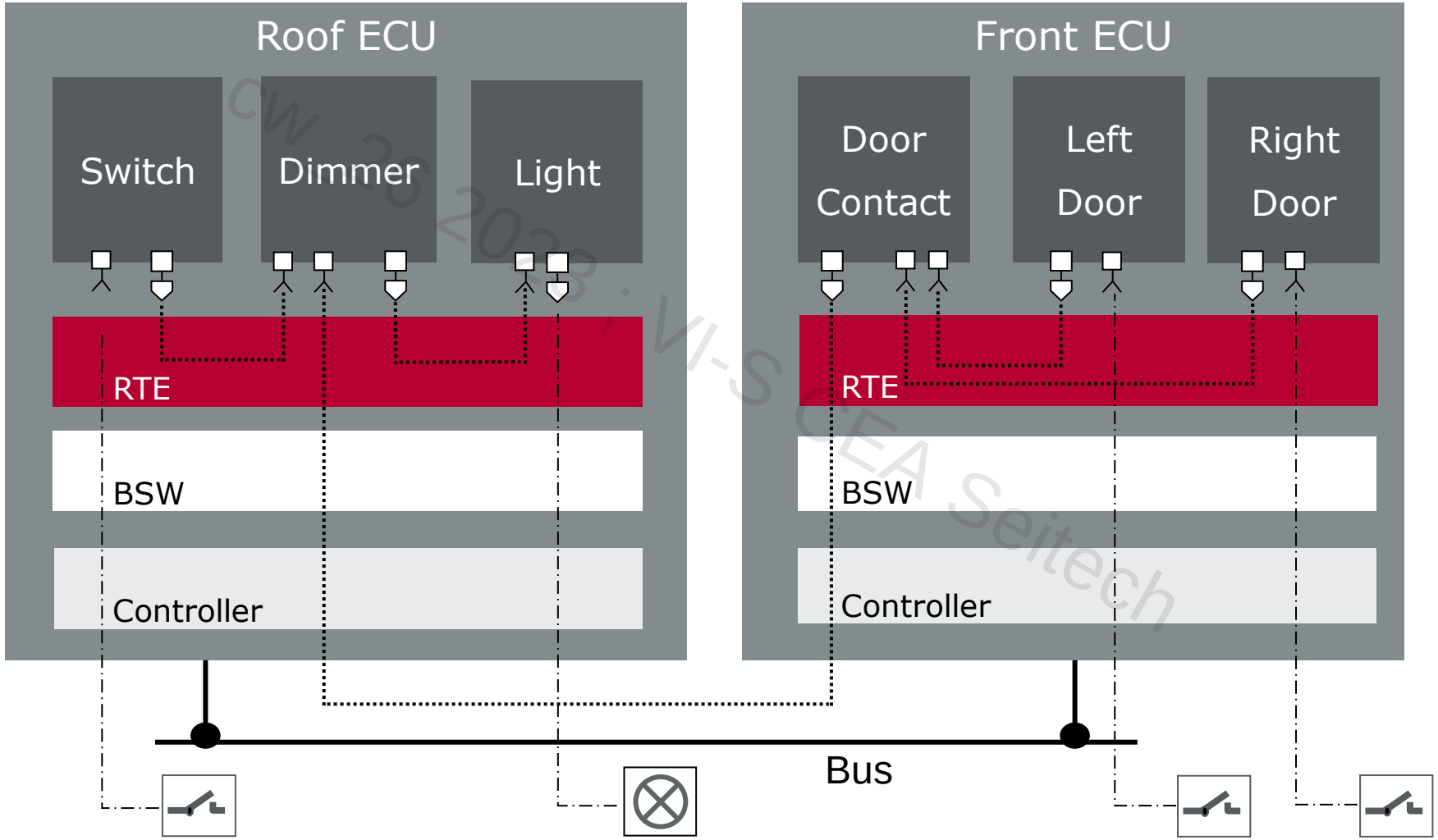
CW_26_2023 · VI-S CEA Seitech

Communication



cw_26 2023 VI-S CEA seitech

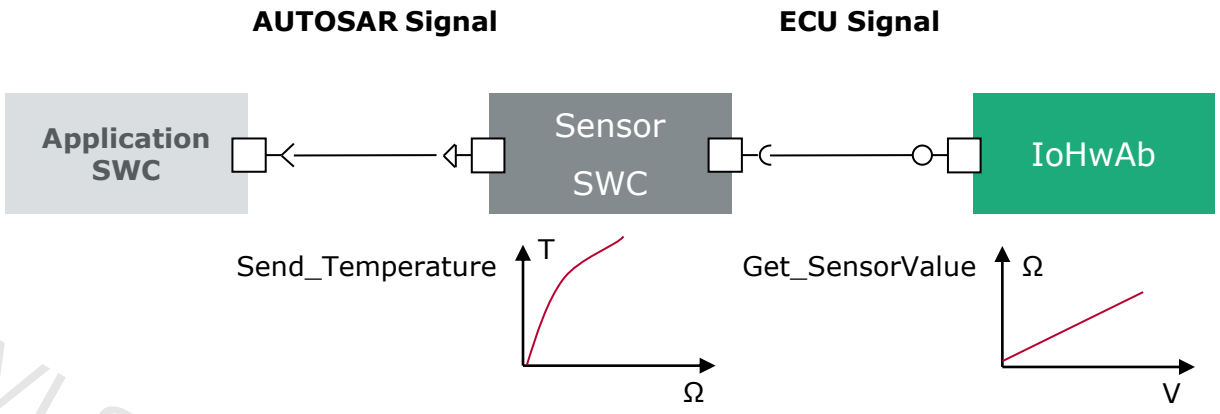
Distribution of Components



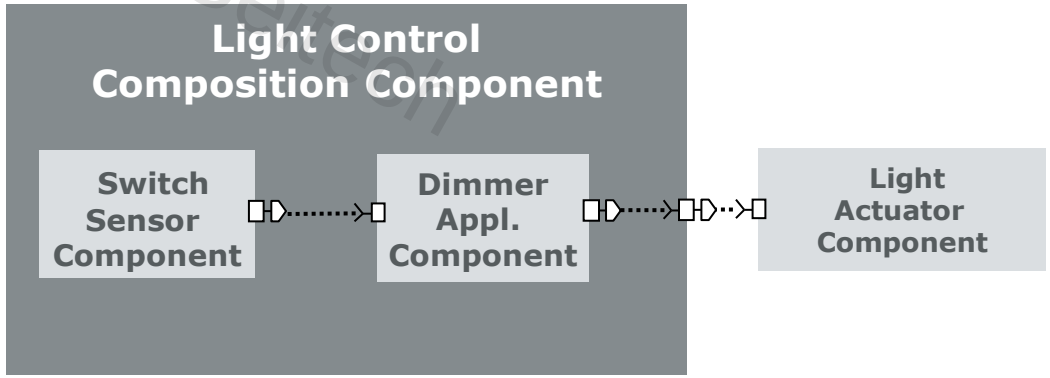
cw 25 2023 ; VI-S GEA Seitech

Types of Software Components

- Atomic component (cannot be further subdivided)
  - Application
    - Implementation of the algorithm
  - Sensor/actuator
    - Preparation of I/O data for the application
    - Component is bound to ECU (constraint for mapping!)



- Composition
  - logical organization or encapsulation of SWCs and/or Composition.

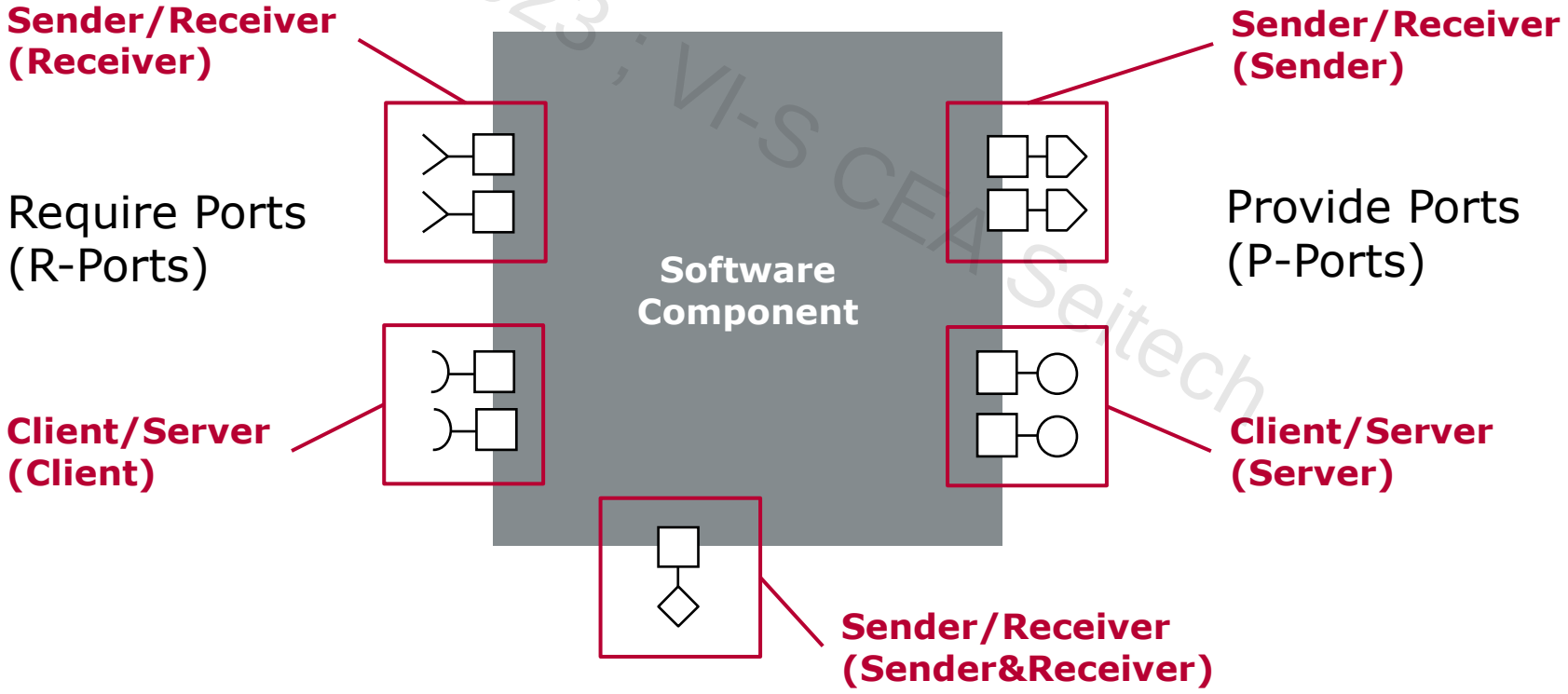


cw 26 2023 ; VI-S CEA Seitzer

Types of Ports

- Software components have Ports
  - Interface to other components for communication
  - Contents: Data elements (S/R) and operations (C/S)

- AUTOSAR 4: Provide Require Ports (PR-Ports):



cw_26_2023 ; VI-S CEA Seitech

AUTOSAR Data Types

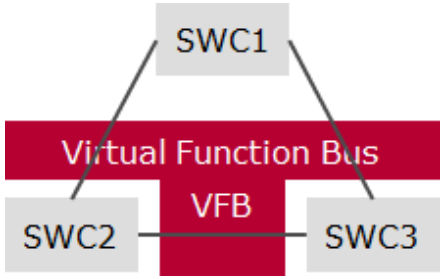
Application Data Level

Implementation Data Level

Base Type Level

Application point of view

- Virtual Functional Bus level
- Typically, OEM with unit
- No endianness, size…



Implementation point of view

- RTE level (ECU specific implementation)
- Programming language C
- Integrator responsibility
- Typically, Tier 1

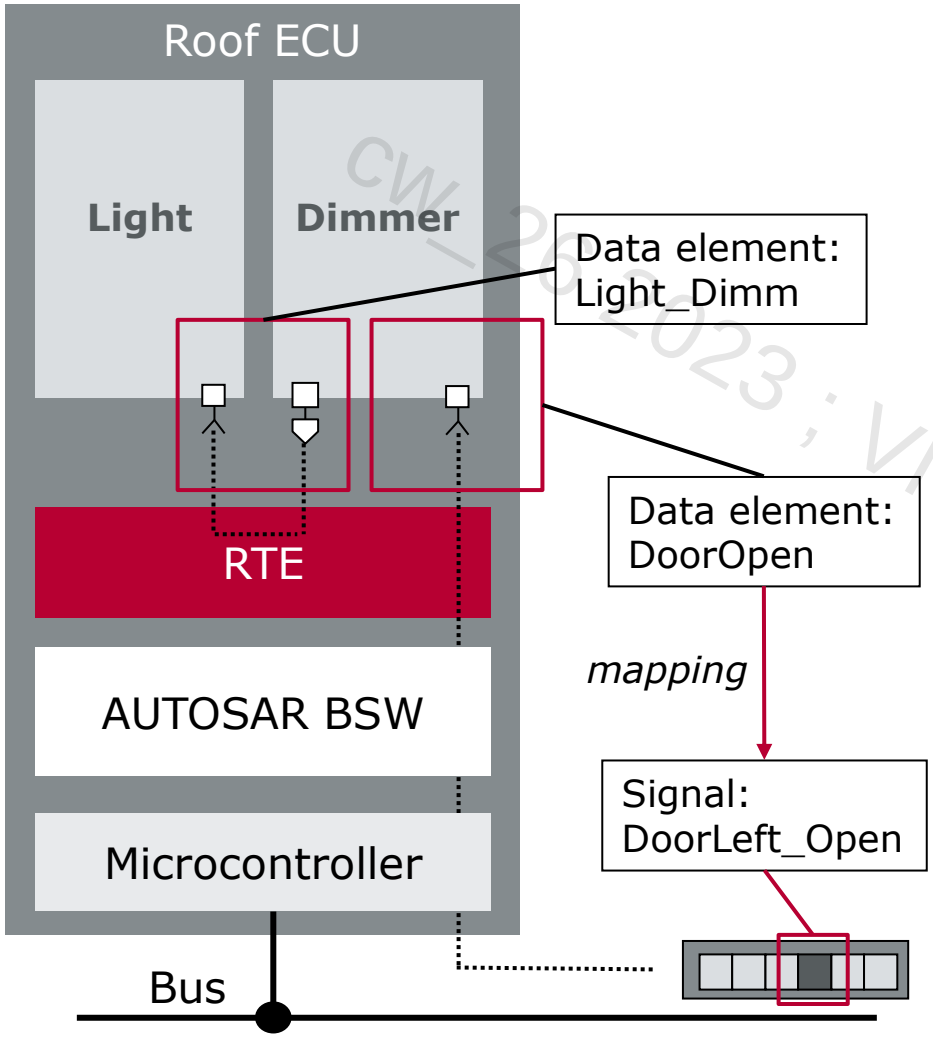
Platform point of view

- CPU level in bits & bytes
- endianness &
- Alignment possible

Basic Software (BSW)

cw_26_2023_VW_CEP_Seitech

Sender-Receiver



- Transport of data
- A port may contain multiple data elements.
- A data element is "mapped" to a signal if the information goes over the bus.
- Simple data types (integer, float) and complex types (array, record).
- Communication: 1:n or n:1

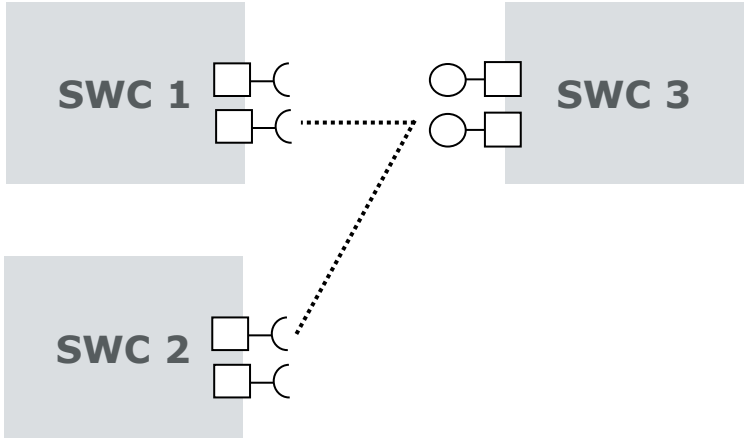
- `Example receiver call: Rte_Read_Door_DoorOpen()`

cw_26_2023 ; VI-S CEA Seitech

Client-Server

- Serving of operations
- Communication: 1:1 or n:1
- Synchronous or asynchronous
- Interface with multiple operations.
- Operations can be called individually.
- Inter and Intra ECU communication possible.

- Example call from Client:
  `Rte_Call_Door_State()`

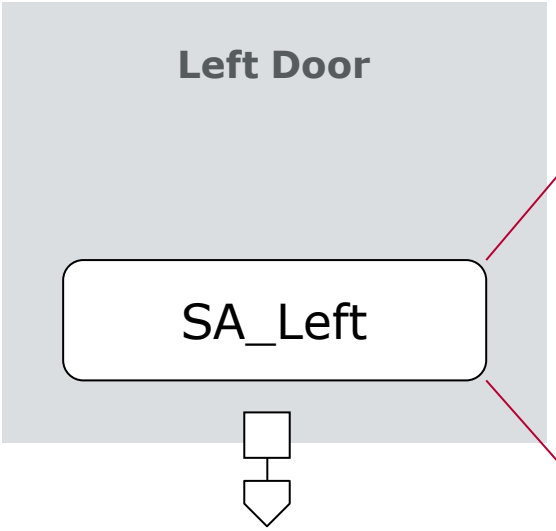


cw 26 2022, VI-S CEA Seitech

Runnables

- Atomic Software components contain
  - Runnable entities (Runnables) as part of their internal behavior
    - Function containing the actual implementation.
    - Function is called by RTE if the corresponding trigger occurs
      - e.g. when data is received a timer expires or by an operation call

V1.20 2023 ; VI-S CFA Seitech



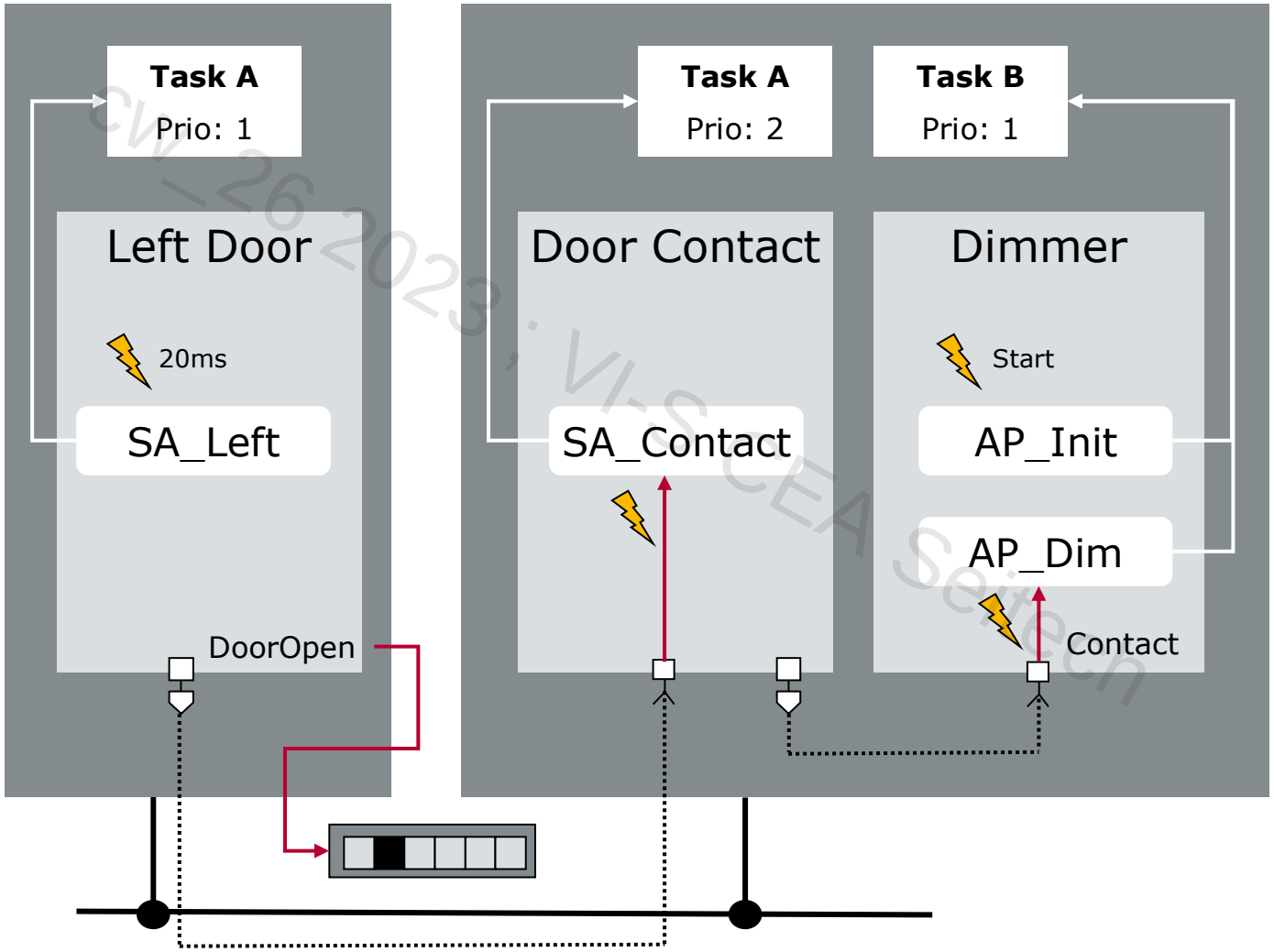

```
// triggered every 200 msec

void SA_Left(void)

{
  /*example implementation start*/
  Std_ReturnType status;
  boolean DoorOpen;
  ...
  status=Rte_Write_<Port>_<Data>(DoorOpen);

  /*example implementation end*/
}
```

Quick Reference to AUTOSAR



cw_26_2023 ; VI-SCEA Seitech

Exercise

Exercise 1:

Outline the functionality of “interior light” utilizing the AUTOSAR modeling objects in a software design. Only use software components that communicate via ports (Sender/Receiver).

- Model a separate software component for each sensor (Door + Switch) and actuator (Light).
- An application software component should interpret the status of doors and pass it to the actuator components.
- Implement a shutoff delay and dimming behavior in another application software component.

Exercise 2:

There are three ECUs (rear, front and roof) which shall realize the functionality of “interior light”:

- Map each SWCs from Exercise 1 to one of the ECUs.
- Which information is exchanged between the ECUs?

cw 26 2023 ; VI-S CEA Seitech

Agenda

Overview and Objectives

AUTOSAR Application

▶ **AUTOSAR RTE**

AUTOSAR BSW

AUTOSAR Methodology

AUTOSAR in Practice

Appendix

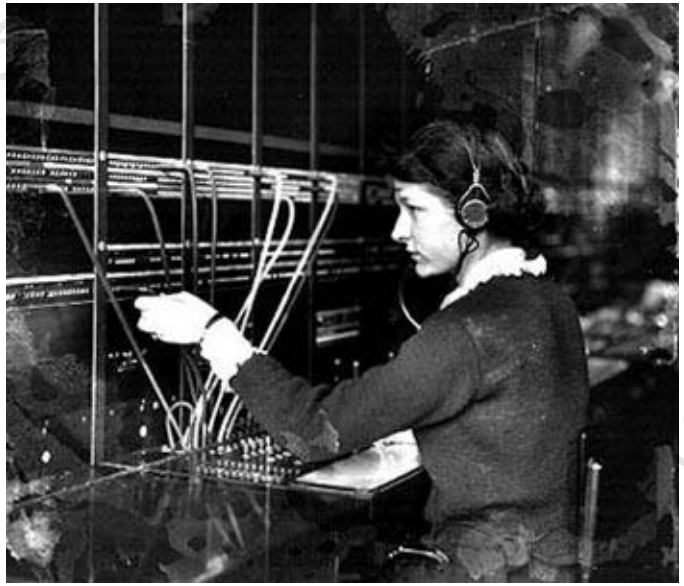
cw_26 2023 ; VI-S CEA Seitech

RTE in the Role of a Switchboard Operator

SWC 2

SWC 1

SWC n



Input

Output

Communication

BSW

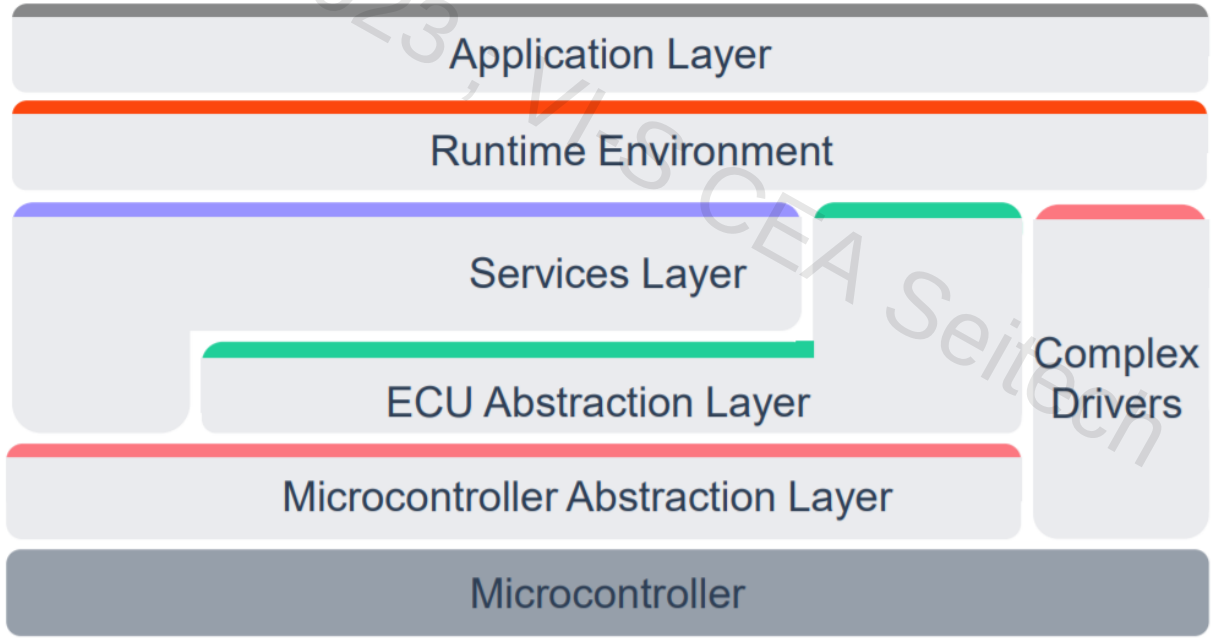
Scheduling

CW_26 Seitech

Architecture

- Task
  - Components independent of ECU mapping.
- Functionality
  - Middleware providing communication services (intra / inter ECU).

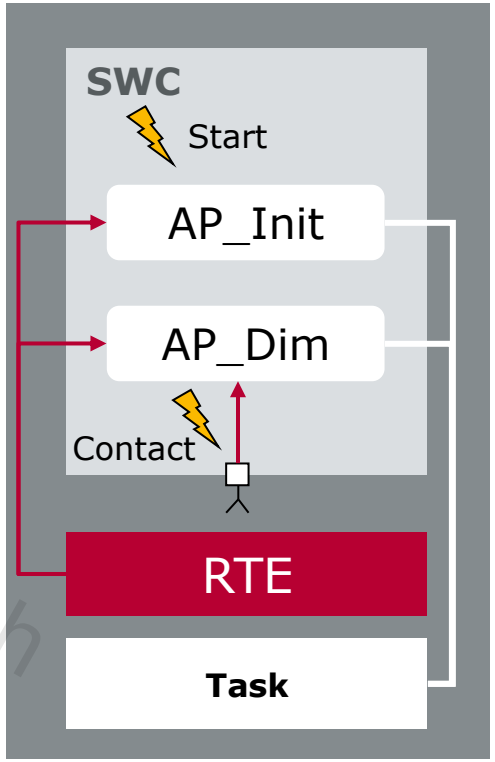
CW_26 2023; VIS CEA Seitech



Source: http://www.autosar.org

RTE as Runtime Environment for Runnables

- RTE is configured (e.g. mapping Runnables to OS tasks).
- Triggering of Runnables via RTE events.
- Includes executable code units for the tasks with calls of the Runnables.
- Configures parts of the OS (tasks, events, alarms).
- Implements communication between components.
- RTE is different for each ECU and is customized to requirements of the SWC.
- RTE abstracts the OS and prevents SWC from directly accessing OS and BSW.
- RTE does not have any runtime component.



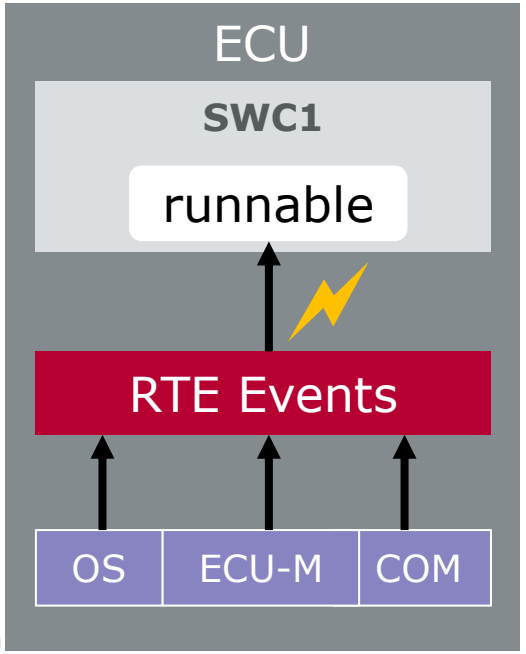
CW_26 2023 ; VI-S CEA Seitech

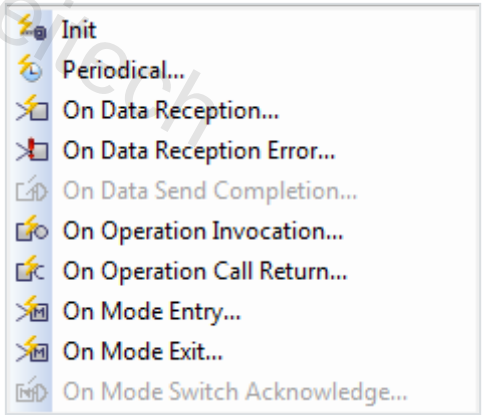
RTE as Runtime Environment for Runnables

- Triggering of Runnables, wake-up from waitpoints
  - Init
  - TimingEvent
  - DataReceivedEvent (S/R)
  - DataReceiveErrorEvent (S/R)
  - DataSendCompletedEvent (S/R)
  - OperationInvokedEvent (C/S)
  - AsynchronousServerCallReturnsEvent (C/S)

  - ModeSwitchEvent
  - ModeSwitchAckEvent

  - ExternalTriggerOccurredEvent
  - InternalTriggerOccurredEvent

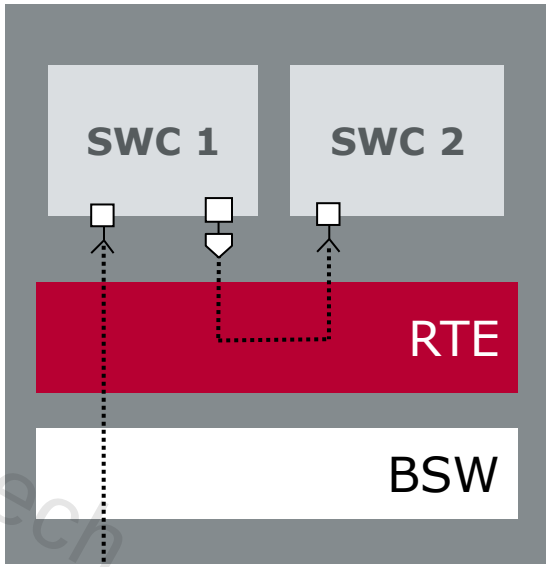




CW_26_2023 : VI-S CEA Saclay

RTE as Communication Interface

- RTE as **communication mechanism** among SWCs and to the BSW.
  - RTE acts as implementation of the VFB
  - Sender-Receiver
  - Client-Server
  - Intra-ECU and Inter-ECU (via COM)
  - RTE implements callbacks of AR-COM



- Other features:
  - Mechanisms for data consistency (inter SWC, intra SWC)
  - Support of primitive and complex data (records)
  - Multiple instantiation of SWC types

cw_26_2023 ; VI-S CEA Seitech

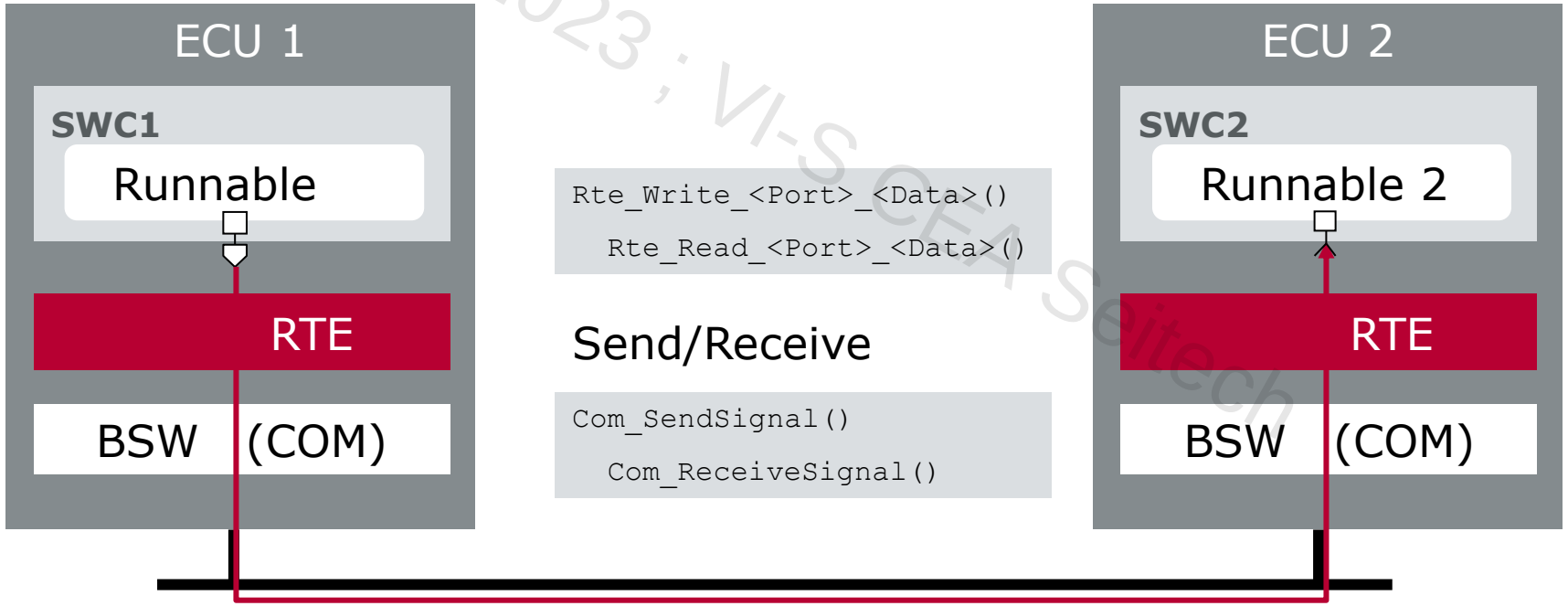
Sender/Receiver Communication: Inter-ECU

Provider (P-Port)

Sender

Require (R-Port)

Receiver

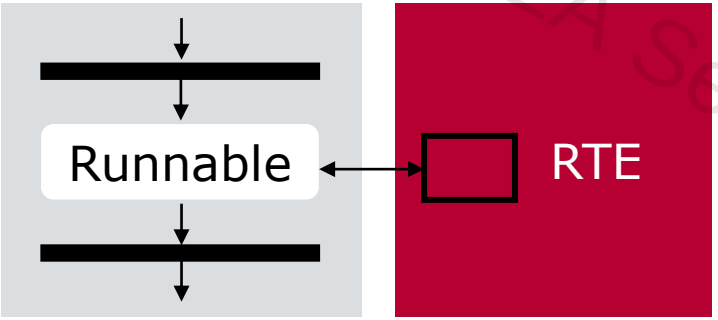


cw_26 2023 ; VI-SCFA Seitech

Sender/Receiver Communication: Direct

- **“last-is-best semantic”** (isQueued=false)
  - RTE uses “direct access” for data buffer
  - 1:n communication
  - Init values as default

```
Std_ReturnType Rte_Read_<p>_<d> (OUT <DataType> *data)
Std_ReturnType Rte_Write_<p>_<d> (IN <DataType> data)
```

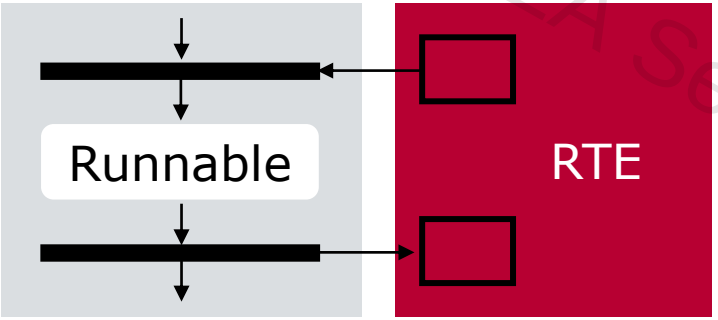


cw_26 2022 , VI-S CEA Seitech

Sender/Receiver Communication: Buffered

- **“last-is-best semantic”** (isQueued=false)
  - RTE generates copy before executing the Runnable
  - After execution of the Runnable the RTE copies the data to a global buffer.
  - Values are not changed during execution

```
<DataType> Rte_IRead_<re>_<p>_<d> (void)
void Rte_IWrite_<re>_<p>_<d> (IN <DataType> data)
```

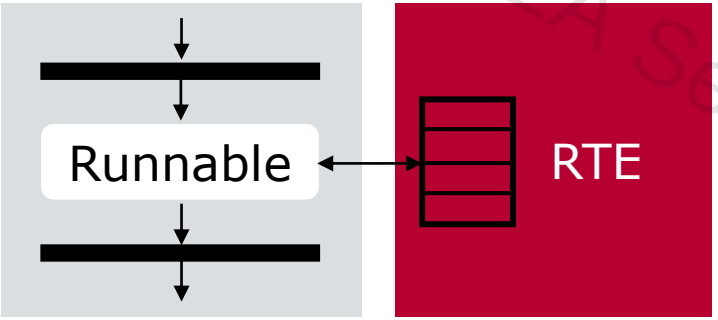


cw_26_2023, VI-S CEA Seitech

Sender/Receiver Communication: Queued

- "event semantic" (isQueued=true)
  - "Polling receive" or "waiting receive"
  - RTE reads from a specific receive queue
  - "Waiting receive" with timeout handling

```
Std_ReturnType Rte_Receive_<p>_<d> (OUT <DataType>*data)
Std_ReturnType Rte_Send_<p>_<d> (IN <DataType> data)
```



cW_26 2023, VI-S CEA Seitech

VECTOR

Sender/Receiver Communication

- **„data element invalidation“**
  - Use case: Sensor delivers invalid values
  - Only for „isQueued=false“ communication
  - realized by „invalid value“
  - Set invalid values on the sender’s side
    - `Rte_Invalidate_<p>_<d>()`
  - Reaction on invalidated data elements on the receiver’s side
    - Evaluate the `Std_ReturnType`. Result would be `RTE_E_INVALID`
      - direct communication: Return value of `Rte_Read_<p>_<d>()`
      - buffered communication: `Rte_IStatus_<re>_<p>_<d>()`
    - Activate Runnable via
      - `DataReceiveErrorEvent`

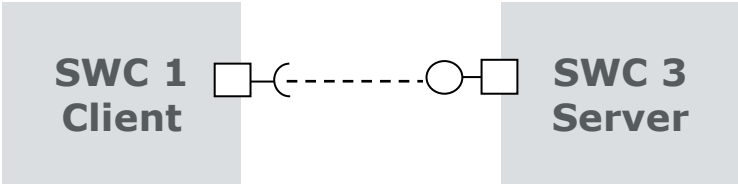
CW_26_2023 VIS CEA Seitech

Client/Server Communication

- Communication
  - n:1
- Server call
  - Client calls Server operation
  - Operation is implemented as Runnable of the Server-SWC
  - Synchronous / asynchronous calls
  - Server Runnables run in the
    - Task context
    - Context of the Client (direct function call)

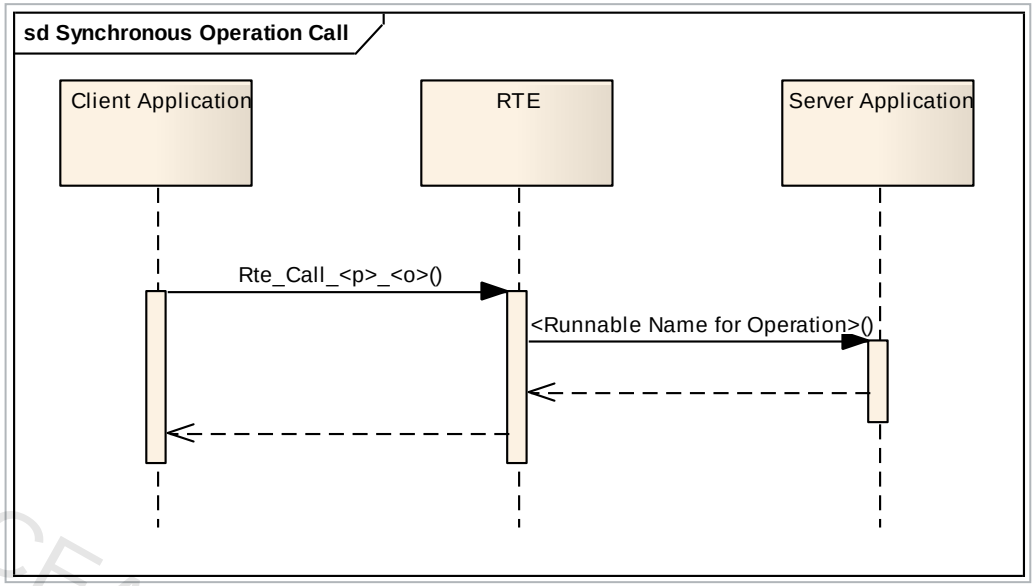
```
Std_ReturnType  Rte_Call_<p>_<o>([IN | IN/OUT | OUT <param_1>],...
        [IN | IN/OUT | OUT <param_n>])
```

cw_26_2023 ; VI-S CEA Seitech



Client/Server Communication

- Synchronous
  - Wait for response of Server (Client is blocked)
  - Results of Server via "OUT" parameter of `Rte_Call_<p>_<o>`



  - Server Runnable

```
Std_ReturnType GetTime(uint32 *hour, uint32 *minute, *uint32 second)
```

  - RTE Client API

```
Std_ReturnType Rte_Call_<Port>_GetTime(uint32 *hour,
    uint32 *minute, uint32 *second)
```

cw_26 2023 ; VI-S CEA Seitech

Client/Server Communication

- Asynchronous
  - Client is not blocked (does not wait for results)
  - Client gets Server results by call of Rte_Result...
    - Polling or waiting
    - Timeout handling
  - RTE Client API

```
Std_ReturnType  Rte_Result_<p>_<o>([IN/OUT | OUT <param_1>],..
                                   [IN/OUT | OUT <param_n>])
```

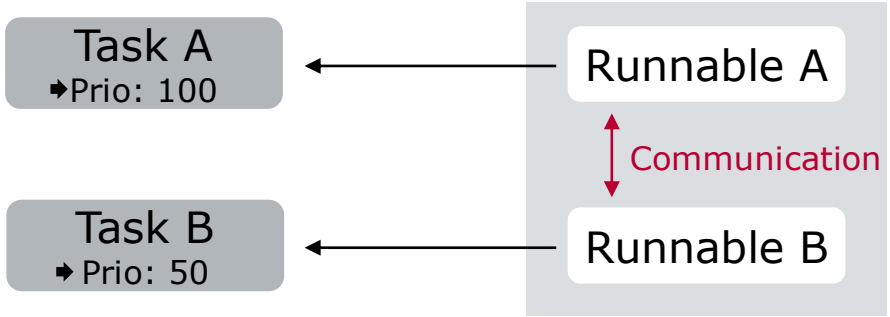
  - Alternative:
    - RTE activates Client Runnables as soon as results are available

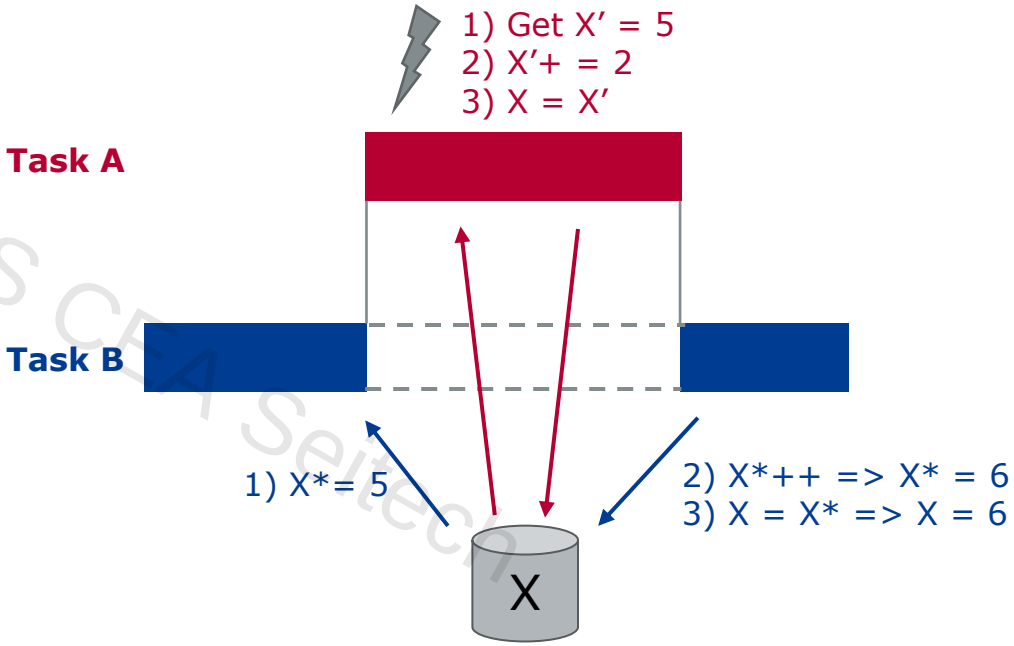
cw_26 2023 ; VI-SCEA Seitech

Intra-SWC Communication

- **Problem**: Communication between Runnables within **one** SWC, which runs potentially on **different** tasks.

cw_26 2023 ; VI-S CFA Seitech





- **Goal**: Data consistency (atomic access on task level).

VECTOR

Intra-SWC Communication Solution

Solution:

Exclusive Areas (EAs)

Enclosed statements are atomic

```
Rte_Enter_<name>();
/* protected statements
*/
Rte_Exit_<name>();
```

Inter-Runnable variables (IRVs)

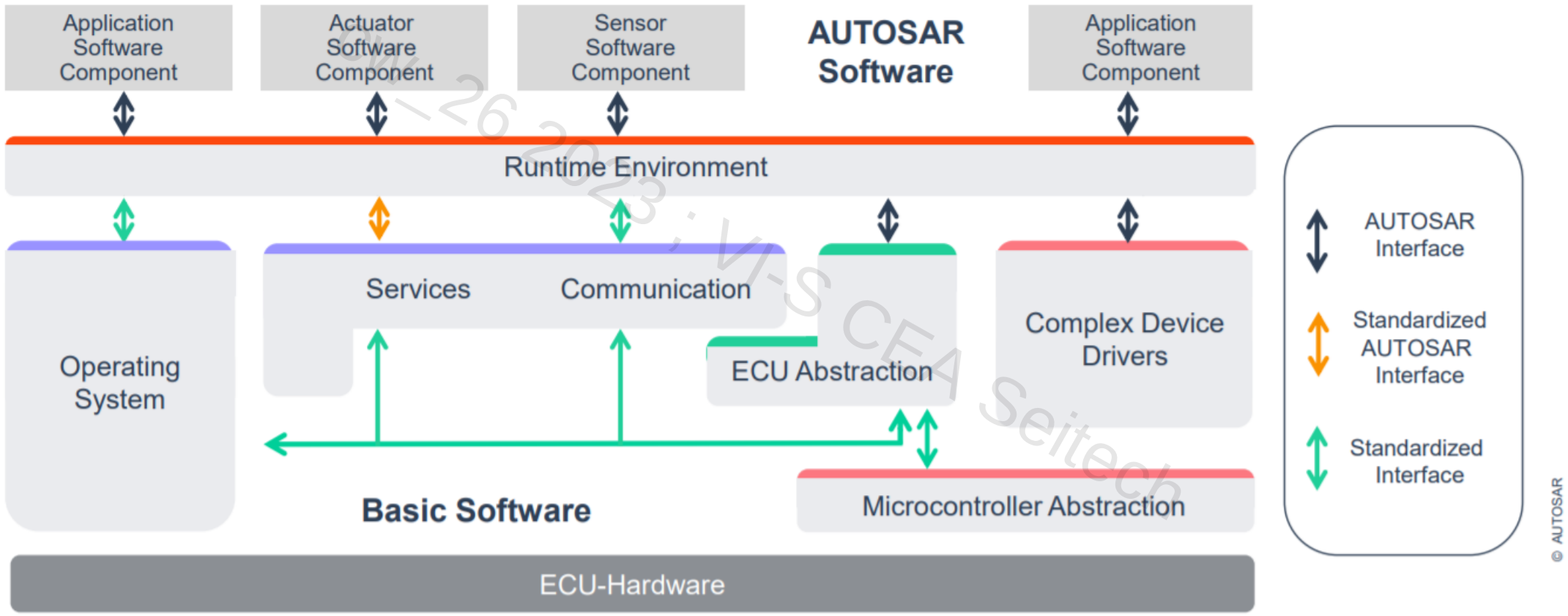
Only access to variable is atomic

`Rte_IrvWrite_<re>_<name>`
or `Rte_IrvRead_<re>_<name>`

Note: There are no problems in intra- and inter-communication of *different* SWCs, since they are handled by the RTE.

cw_26 2023 ; VI-S CEA Seitech

Interfaces

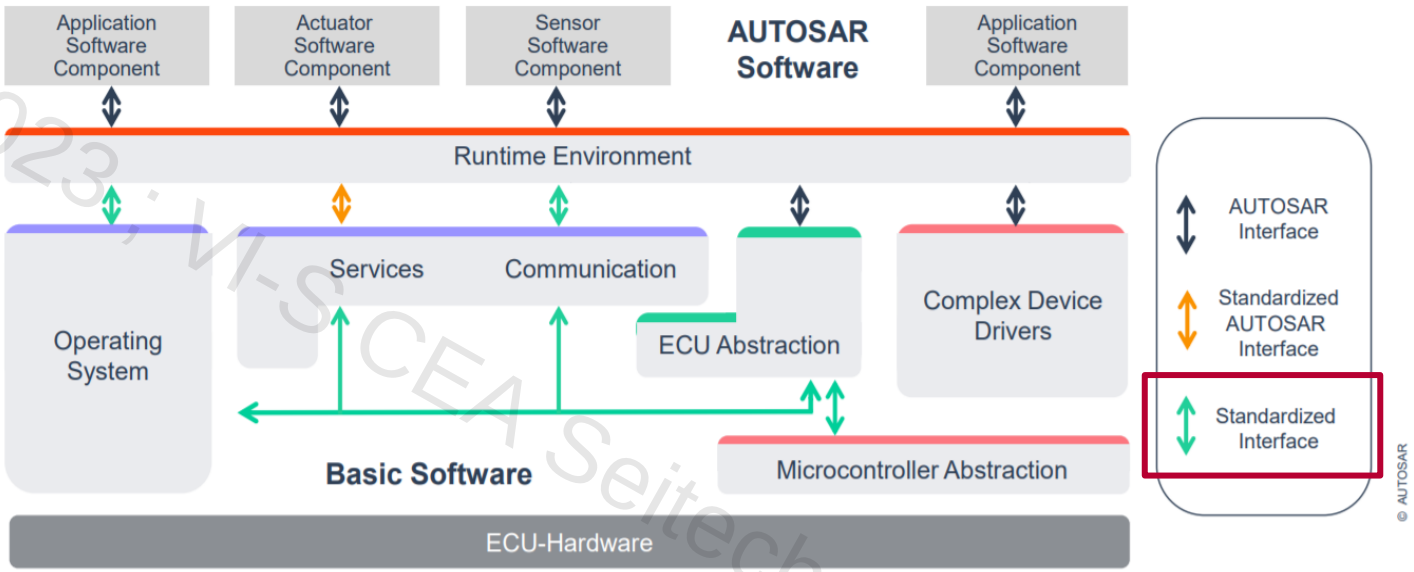


Source: http://www.autosar.org

Oct 26 2023 : VI-S CEA Seitech

Standardized Interface

- Simple C-API
- OS and COM are only available indirectly via the RTE
- Has no ports
- OS: RTE implements the tasks and uses the OS API
  - `Schedule()`, `WaitEvent()`
- COM: RTE implements the callbacks of COM and uses their API
  - `Com_SendSignal()`
- EcuM: `Rte_Start()`, `Rte_Stop()`

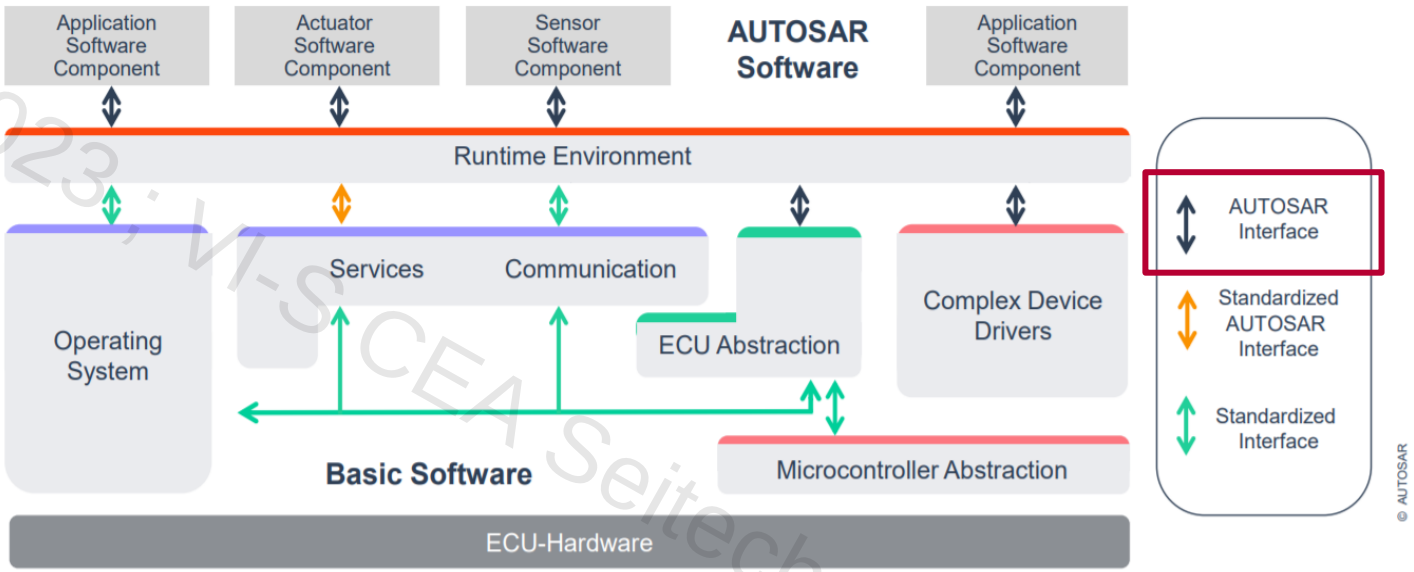


Source: http://www.autosar.org

cw_26 2023; VI-SCEA Seitech

AUTOSAR Interface

- I/O interfaces of hardware are described as in SWC
  - Ports
  - Runnables
- ECU Abstraction = “Firmware”
  - Since access here is exclusively via sensor/actuator SWC
- API function:
  - `S/R -> Rte_Write_<p>_<d>`
    - > **`Rte_Write_DimmLight_DimmValue (value)`**
  - `C/S -> Rte_Call_<p>_<o>`



Source: http://www.autosar.org

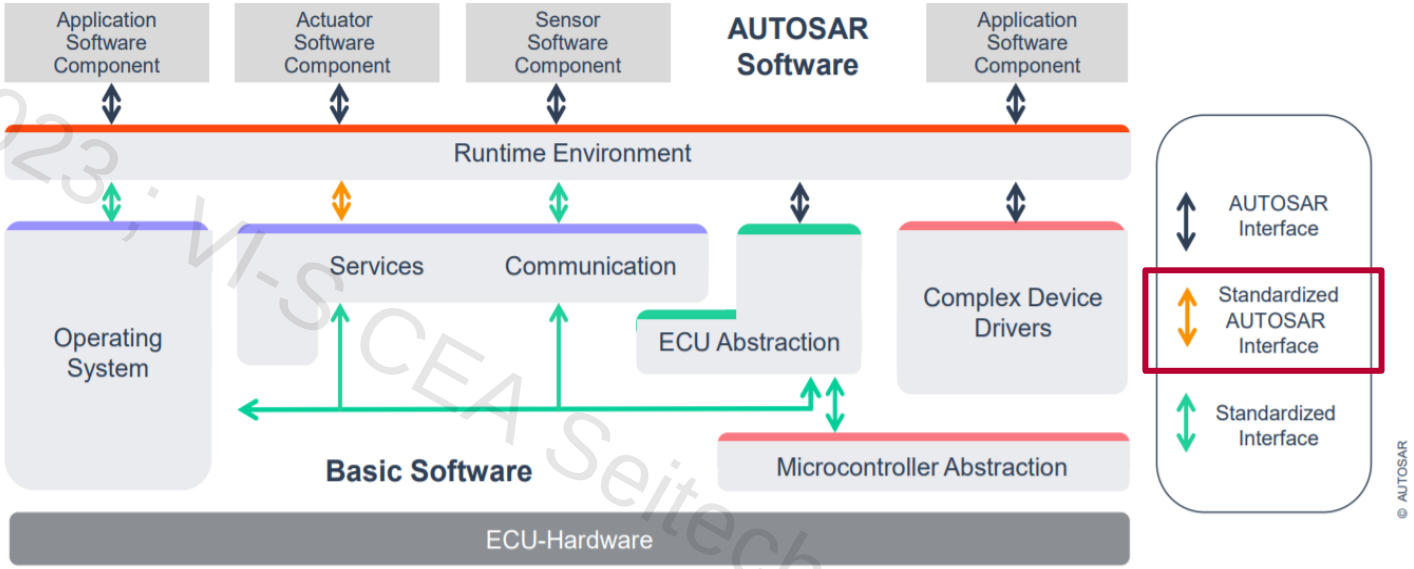
cw_26 2023; VI-SCEA Seitech

Standardized AUTOSAR Interface

- Are standardized and not ECU specific (e.g. ECU Abstraction)

- RTE -> BSW: Here one speaks of **"Service Ports"**

- API function
  - Communication mode for special SWC



Source: http://www.autosar.org

- SWC → RTE

```
Rte_Call_<portPrototype>_<operation>()
```

- <Operation> DEM

```
SetEventStatus (In Dem_EventStatusType
   status)
```

cw_26 2023; VI-S CEA Seitech

Agenda

Overview and Objectives

AUTOSAR Application

AUTOSAR RTE

▶ **AUTOSAR BSW**

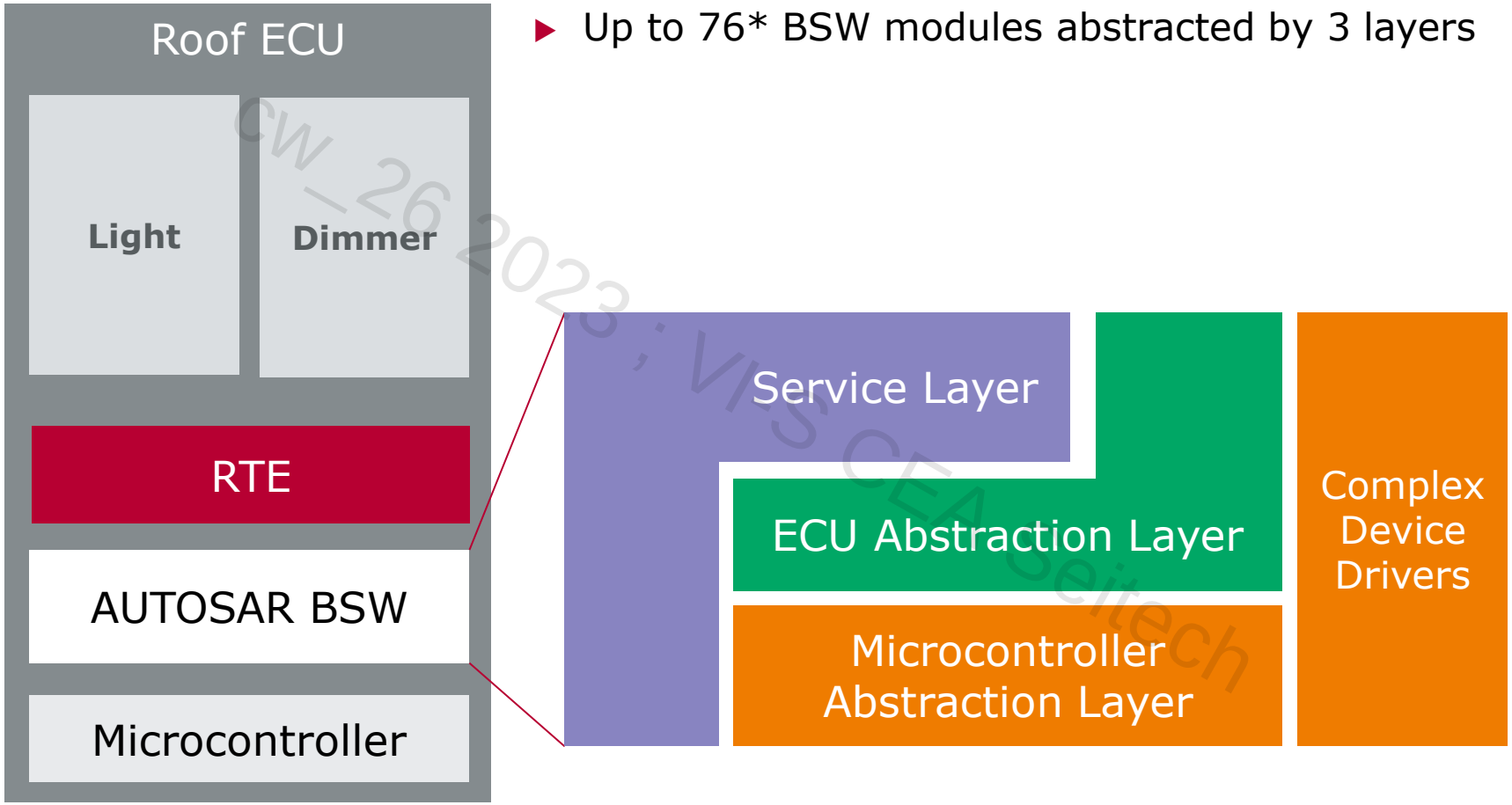
AUTOSAR Methodology

AUTOSAR in Practice

Appendix

cw_26 2023 ; VI-S CEA Seitech

Layered View: Simplified



- Up to 76* BSW modules abstracted by 3 layers

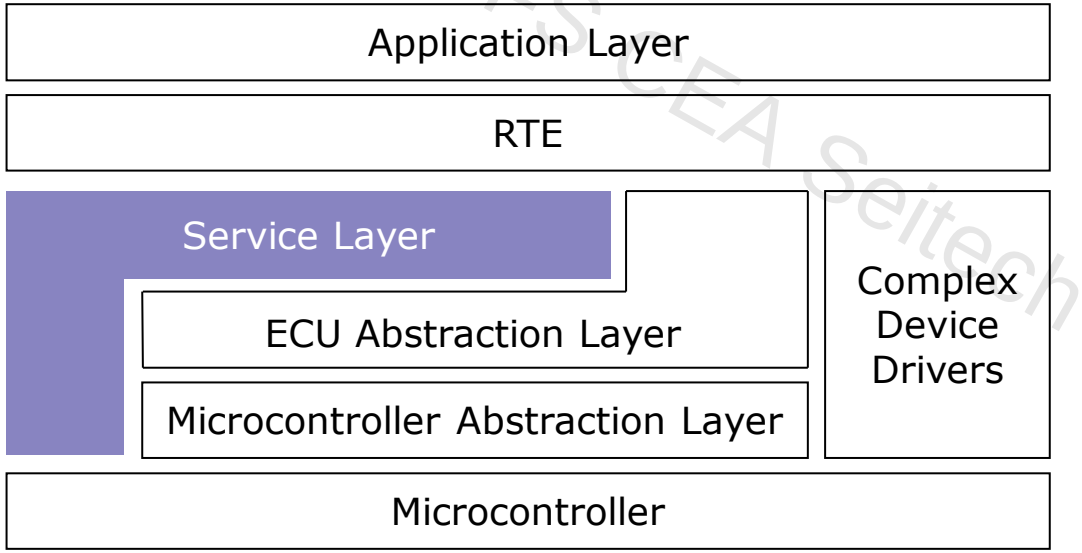
*) Count of the AUTOSAR_SWS specs in AUTOSAR 4.0 R3

cw_26 2023 ; VIS CEA Seitech

Service Layer

- Task
  - Services for application
- Functionality
  - Diagnostics, NVRAM Management, Watchdog Manager, Communication, OS, Schedule Manager, ECU state management, Com Channel Management

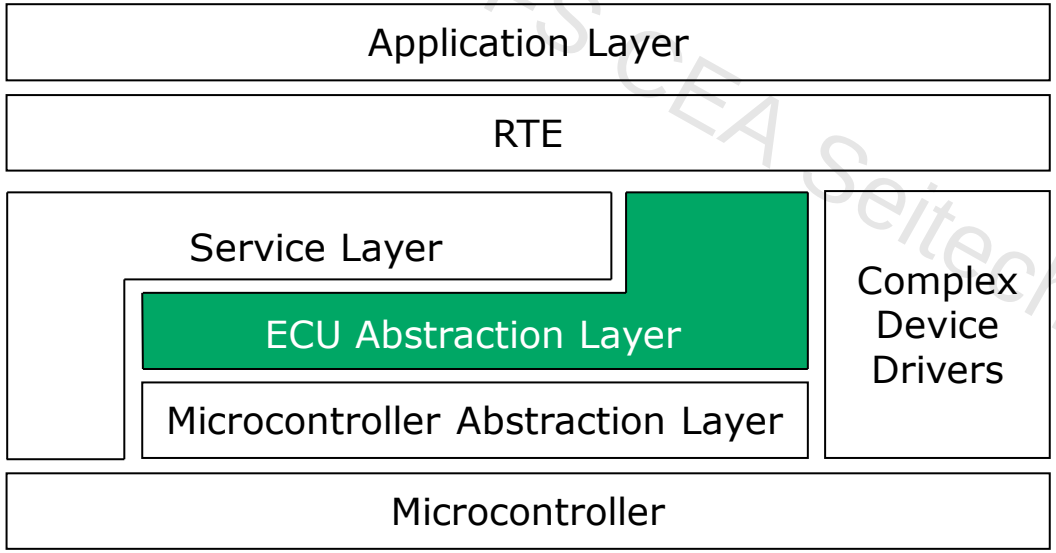
cw_26 2023 ; VI-S CEA Seitech



ECU Abstraction Layer

- Task
  - Make higher levels independent of ECU hardware
- Functionality
  - Driver for external devices
  - Interfaces for internal and external periphery (I/O, Memory, Watchdog(s) and communication)

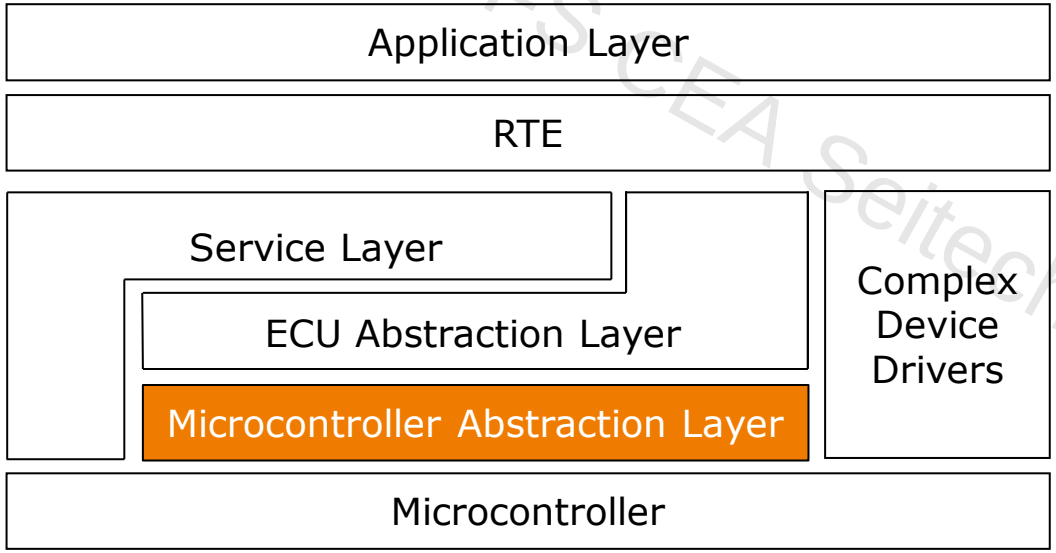
cw_26_2023 ; VL-S CEA seitech



Microcontroller Abstraction Layer

- Task
  - Make higher layers independent of microcontroller.
- Functionality
  - Drivers with direct access to internal periphery of µC.
  - Memory-mapped devices external to µC.

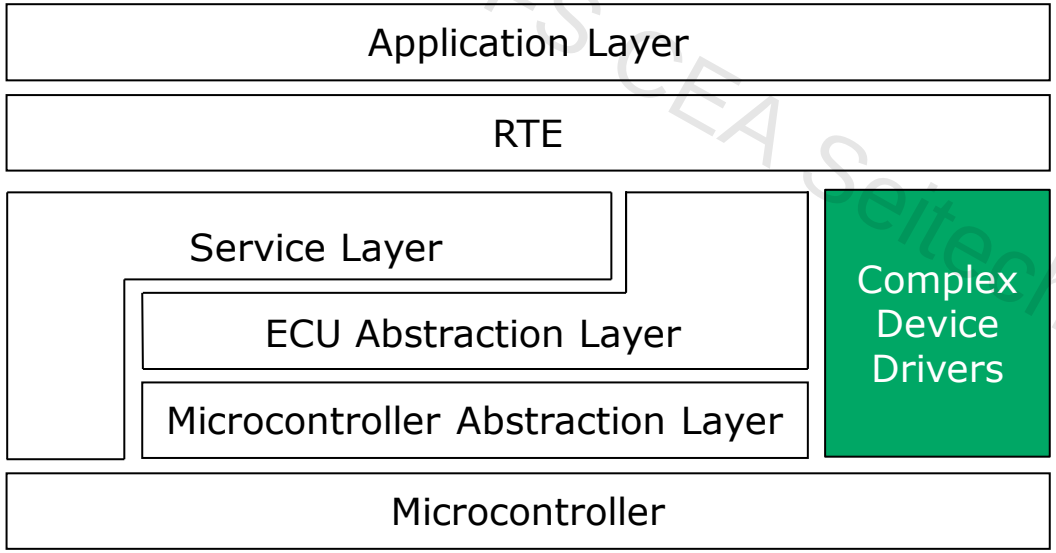
cw_26_2023 ; VL-S CEA Seitech



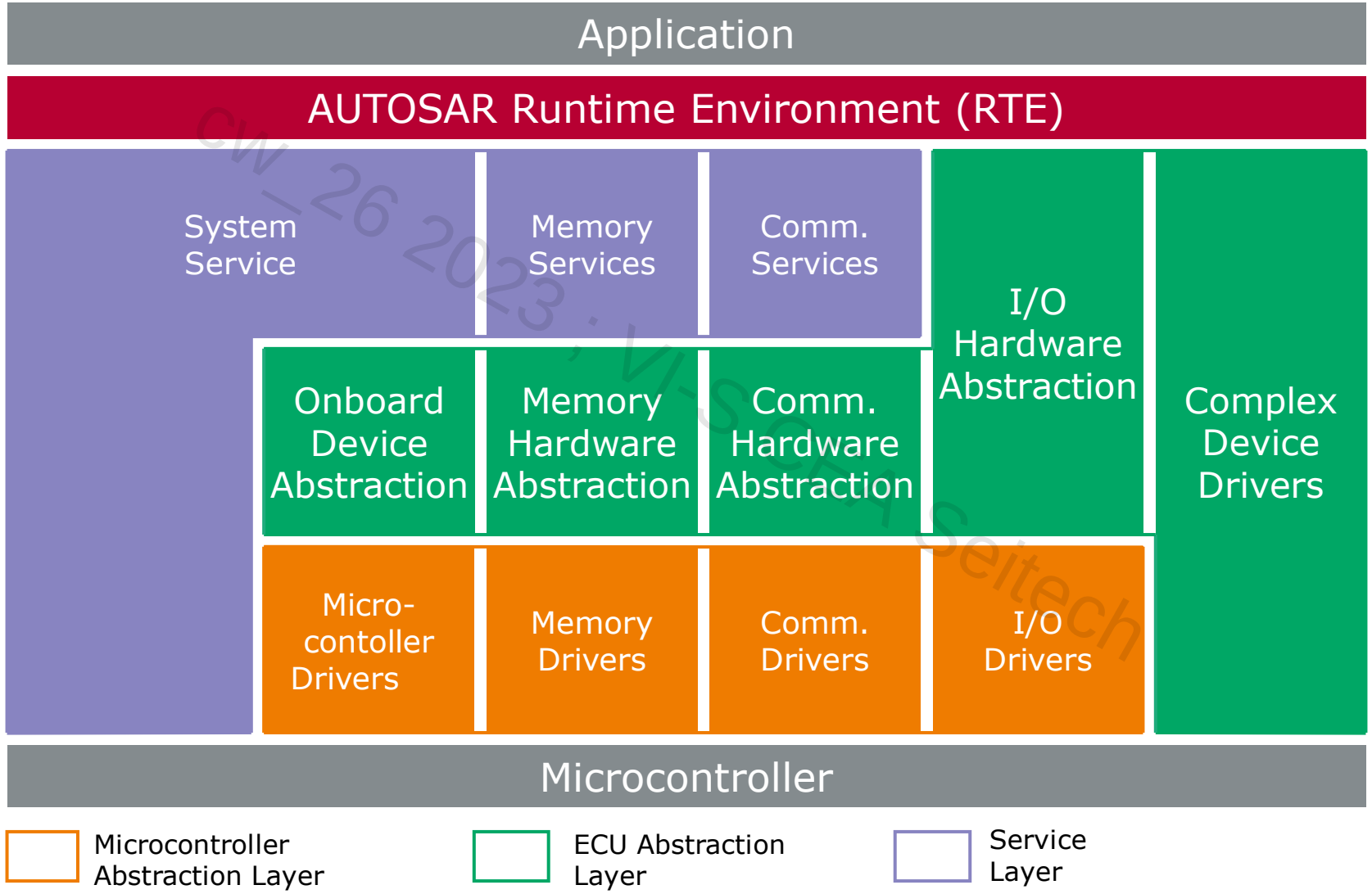
Complex Device Drivers

- Task
  - Offer functionality for complex sensors and actuators.
- Functionality
  - Direct access to resources for critical applications.
  - Examples: Injection control, battery management system.

cw_26_2023 ; VI-S CEA Saitech

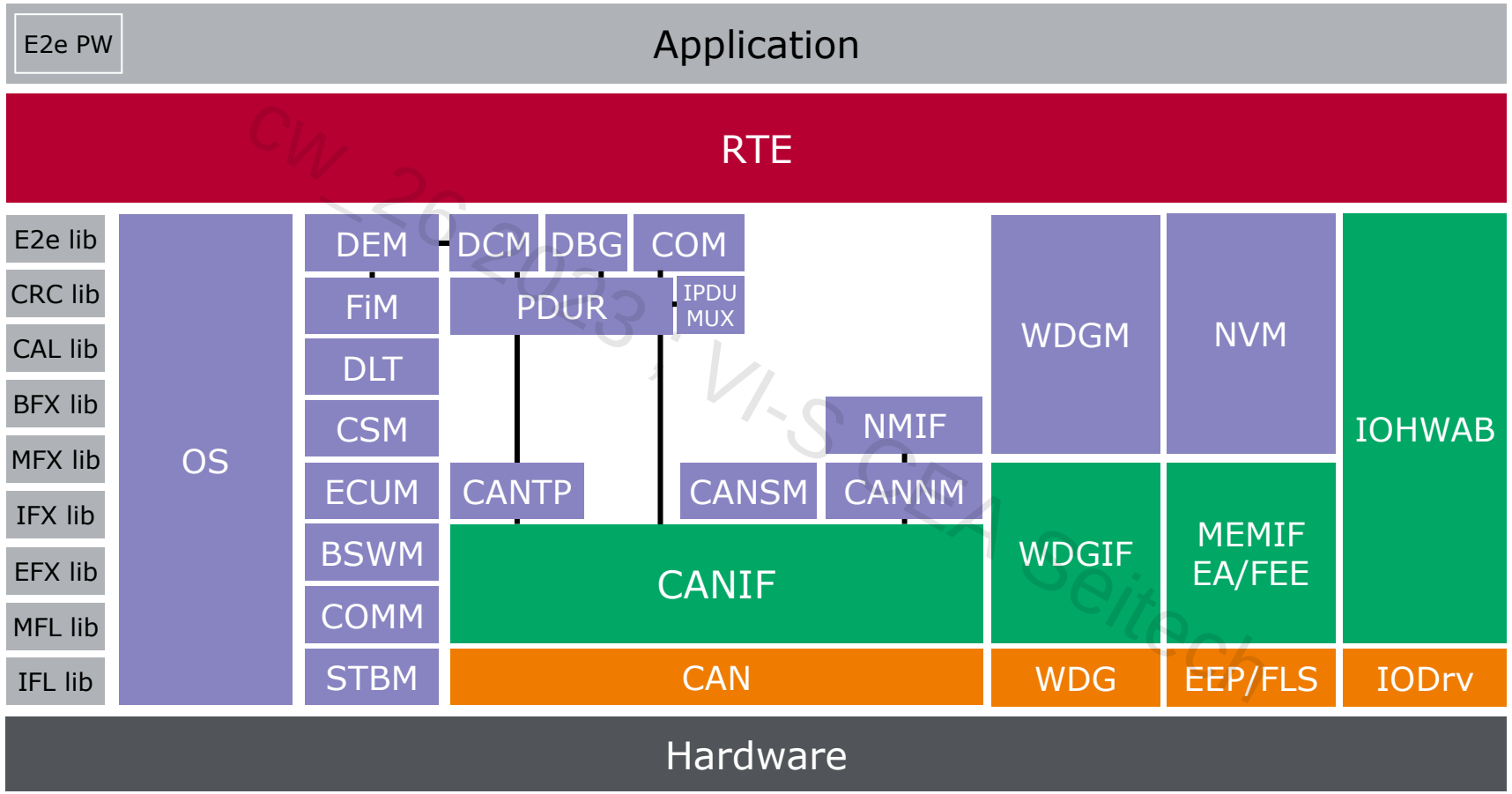


Layered View: Detailed



cw_26_2023 ; VI-SCR ; Seitech

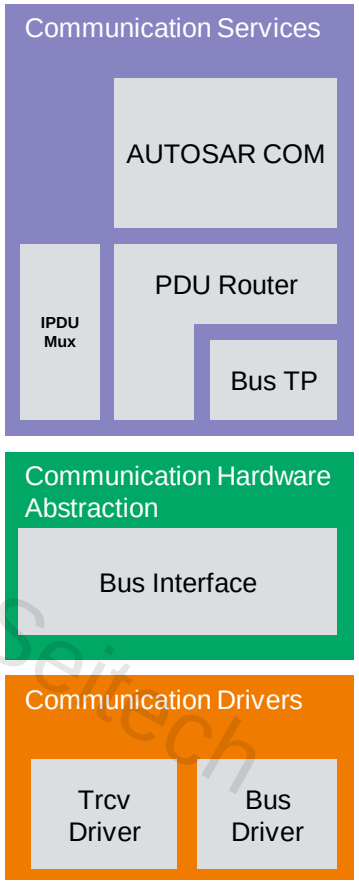
AUTOSAR Software Architecture for CAN ECUs



- High level abstraction
- Many basic software modules

Communication

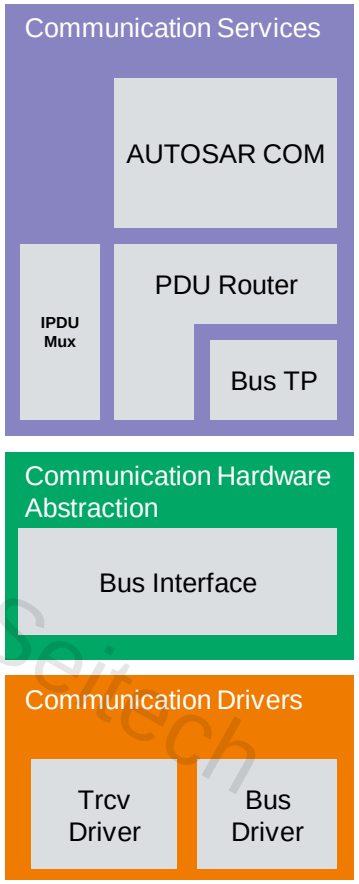
- COM
  - Signal interface
  - Setup/analysis of PDUs
  - Signal gateway
- PDU Router
  - Abstraction of the bus system relative to higher layers
  - PDU gateway
- IPDU Multiplexer (optional)
  - same CAN ID but different signal layout/content
- Transport Protocol
  - Segmented data transmission



CW_26 2023, VI-S CEA Seitech

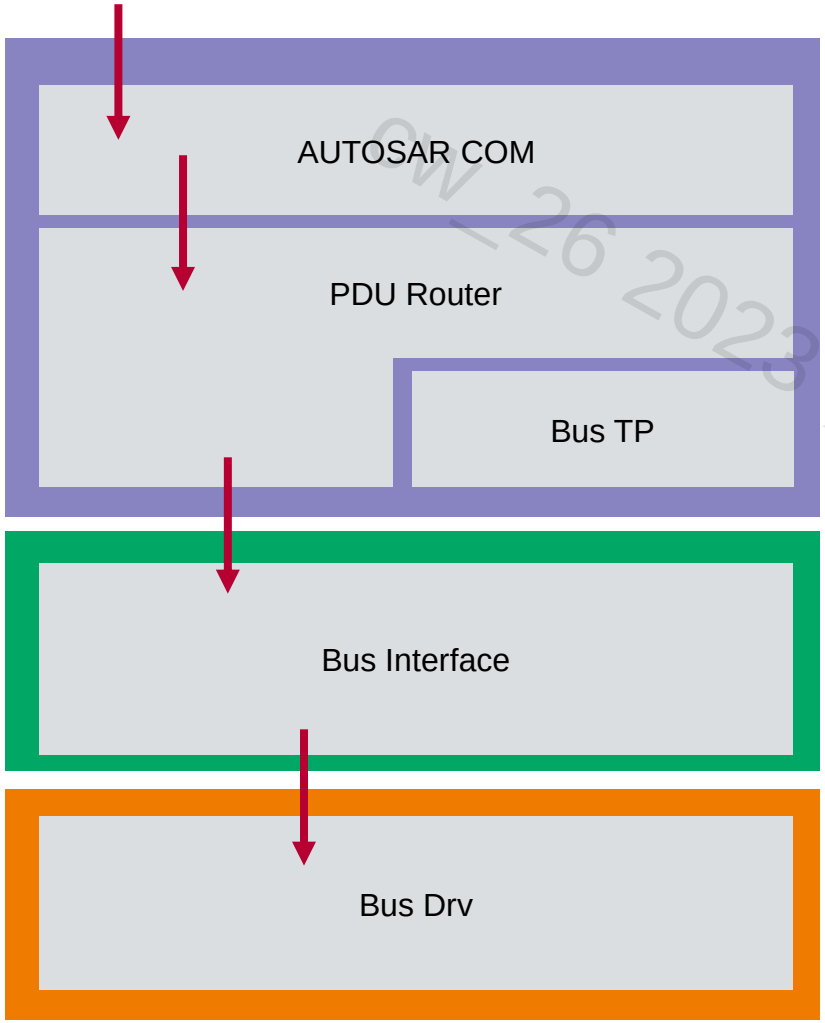
Communication

- Interface
  - HW independent, bus-typical functions
    - Queuing
    - Assembly of data link layer PDUS/frames (with FlexRay)
    - Tx/Rx triggering on time-triggered buses (LIN, FlexRay)
- Driver
  - HW operation
    - Initialization, filling of buffer, ISR implementation
- Transceiver
  - Operation of hardware line driver circuits



cw 36 2023 : VI-S CEA Seitech

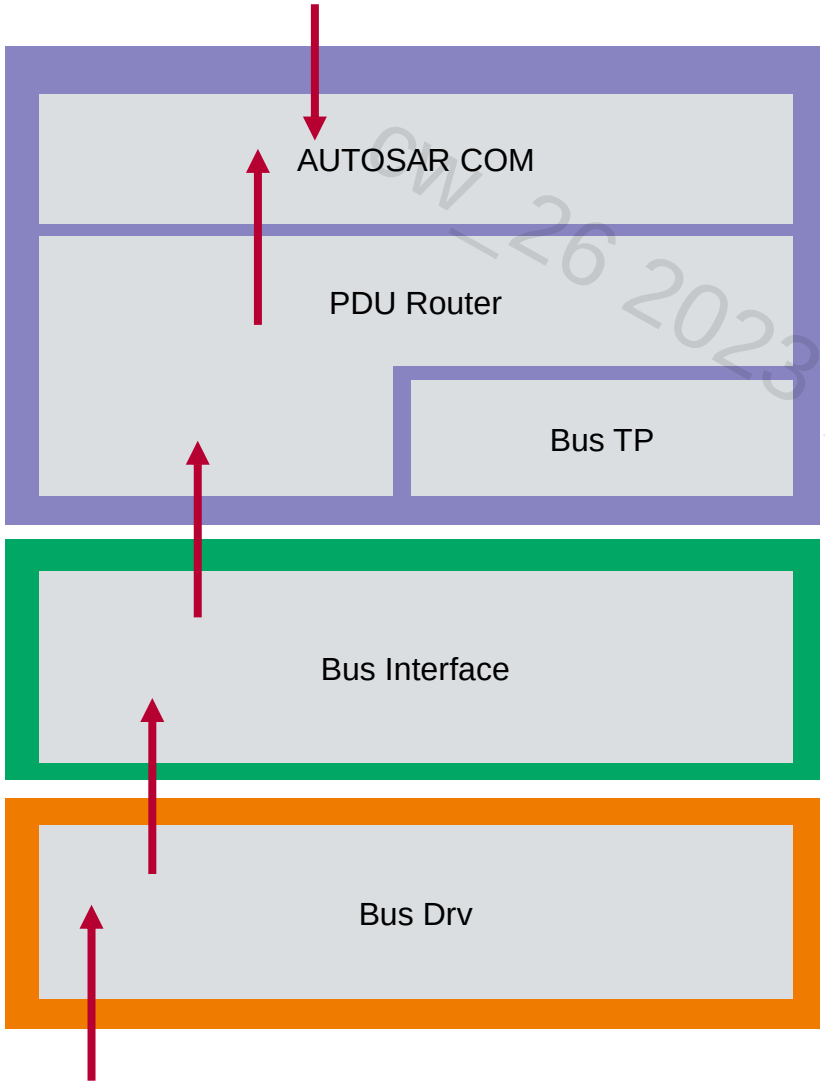
Communication: Sending Controlled by COM



- e.g. CAN (spontaneous + cyclic), FlexRay (dynamic segment)
- Writing a signal
  - Is written to PDU buffer
- PDU is passed to the PDU Router either immediately or periodically
  - Each PDU is identified by an ID
- PDU Router recognizes bus system and passes an associated Interface
- Interface recognizes relevant channel and writes PDU to driver or queue
- Driver sends the message as quickly as possible while observing priorities

cw_26 2023 ; VI-S CEA Saitech

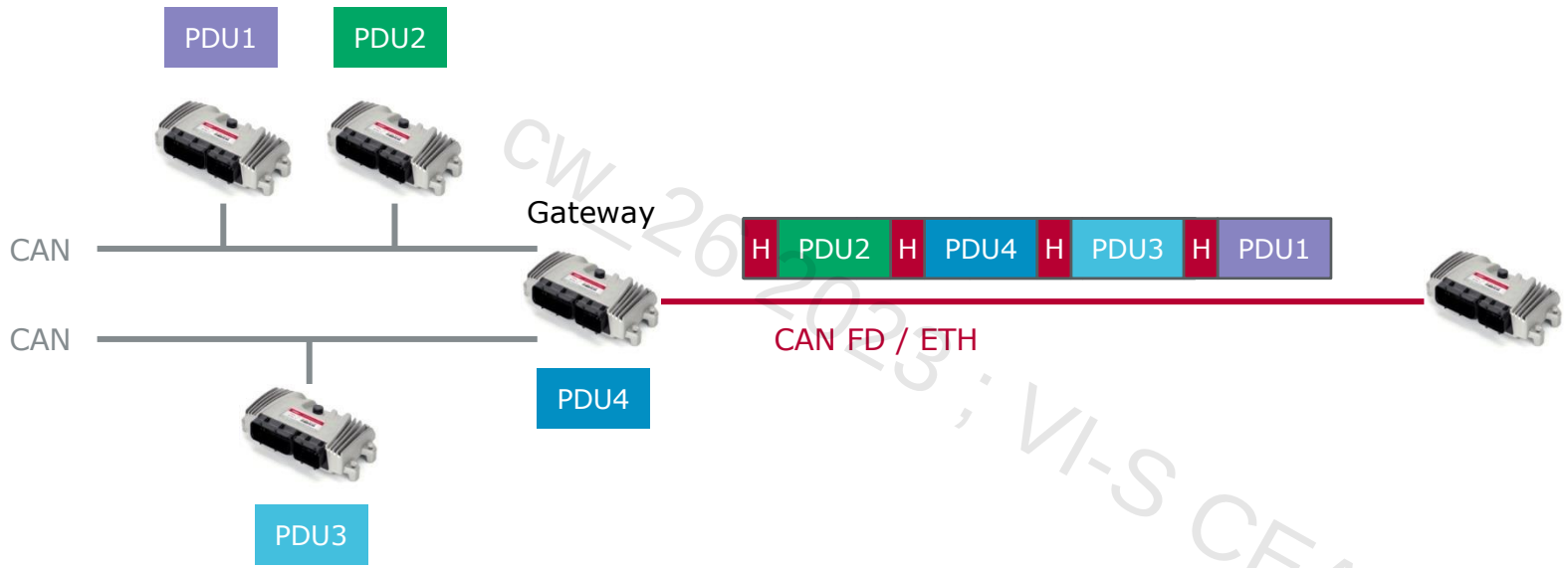
Communication: Receiving



- Interrupt or polling, here interrupt
- Receipt of a frame
- Rx interrupt, processed by driver
- Data are passed to the interface by RxIndication
- Transfer to PDU router
- Transfer to COM
  - Indication to RTE, if SWCs are used with Data Reception Trigger
  - Else stored in buffer
- Signals are read by RTE

ew_26 2023 ; VI-S CEN Gatech

Container PDU Concept (AUTOSAR 4.2.1)



cw_26_2023 ; VI-S CEA Sotech

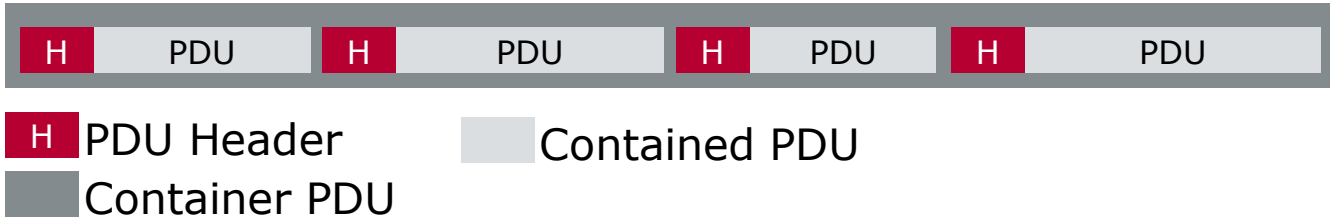
ECU (e.g. gateway) collects PDUs and transmits them in one CAN FD / ETH frame

- The order of PDUs within the frame can dynamically change (e.g. if one PDU needn't be sent)
  - Requires additional header information
  - Different frame triggering maybe necessary

Container PDU Concept (AUTOSAR 4.2.1)

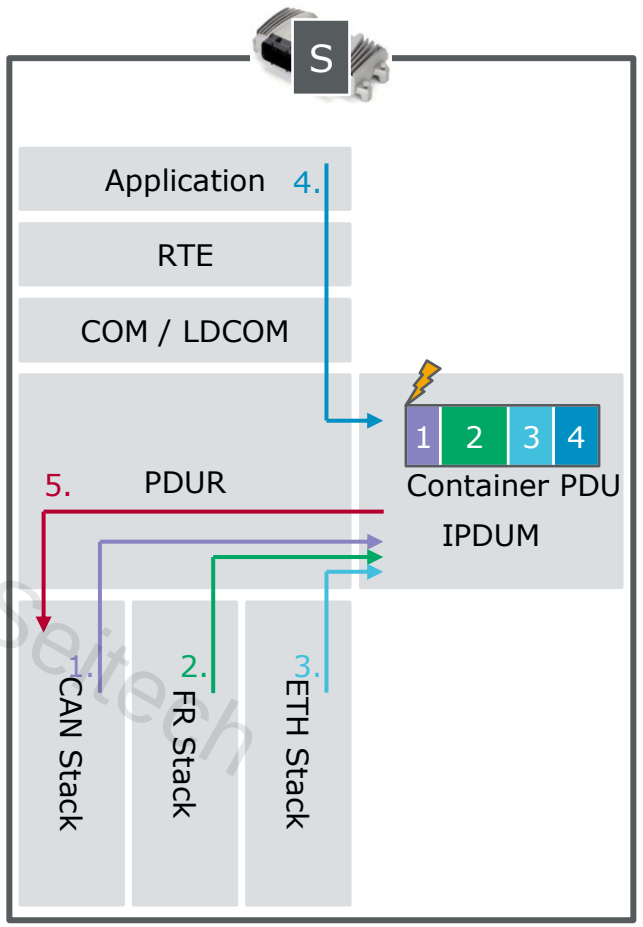
- A Container PDU can aggregate several Contained PDUs
  - Reduces network protocol overhead (at least for Ethernet)
  - Increases flexibility
- Is applied for communication via
  - Ethernet (SOAD) and
  - CAN FD (IPDUM)
- Comparable concept to FlexRay but
  - for FlexRay, the position of the PDU in the frame is static
  - for a Container PDU, the position of the PDU in the container is dynamic
- SOAD and IPDUM interpret a Container via PDU Headers consisting of
  - Id
  - DLC

cw_26 2023 ; VI SCEA Seitech



Container PDU Concept (AUTOSAR 4.2.1)

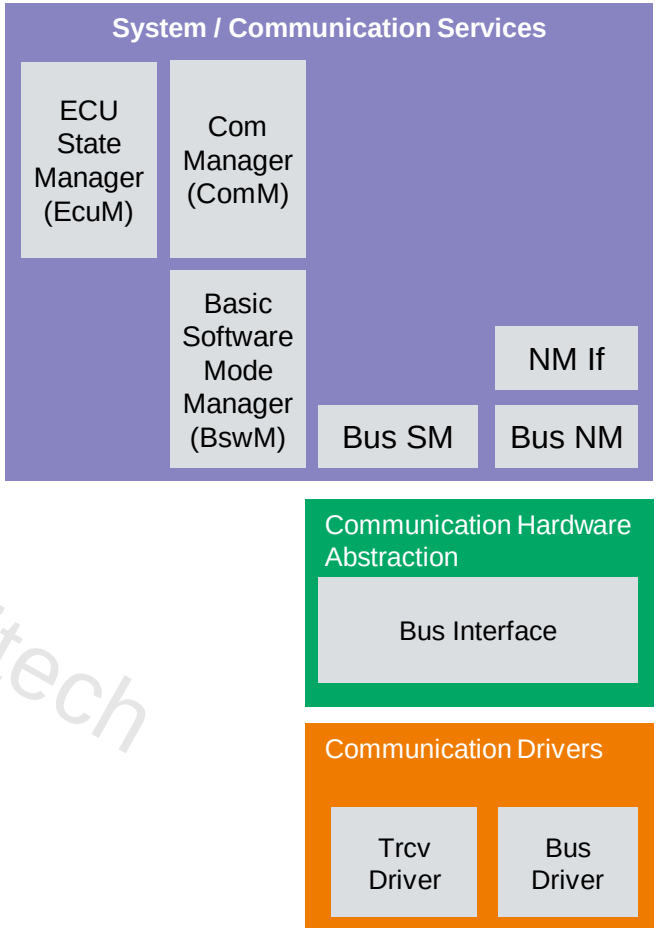
- Realized in the IPDU Multiplexer (IPDUM)
  - Network independent
  - Mapping of
    - "Contained PDU"-to-"Container PDU"
- Container PDU (frame) triggering
  - Container PDU
    - threshold exceeded
    - timeout
  - Contained PDU
    - timeout
    - High-priority
    - First Contained PDU trigger
- Header formats
  - 4 bytes header
    - 3 bytes ID, 1 byte length
  - 8 bytes header
    - 4 bytes ID, 4 bytes length



cw_26_2023 ; VI-S CEA Seitech

Mode Management

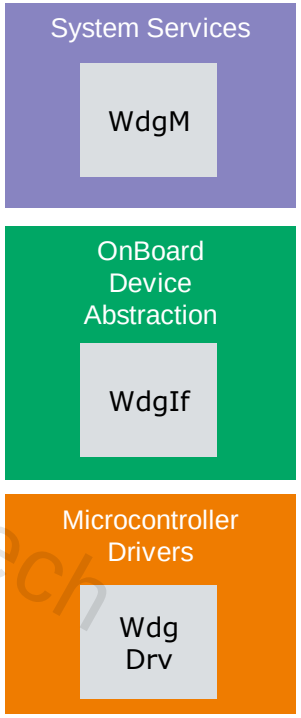
- ECU State Manager (EcuM)
  - ECU state machine
  - Centralizes and validates wake-up events.
  - Coordination of start-up/shutdown
- Communication Manager (ComM)
  - Abstracted bus state machine
  - Startup/shutdown of communication
- Network Management (Nm If, Bus NM)
  - Keep the bus awake, coordinate shutdown
- Bus State Manager (Bus SM)
  - Switch transceiver and controller state
- Basic Software Mode Manager (BswM)
  - Rule based system to control BSW behavior (e.g. switch IPDU Groups in COM)



CW_26 2023 ; VI-S CEA Seitech

Watchdog

- Concept
  - supervise execution of application software
  - trigger watchdog / no not trigger it any more
- WdgM
  - manages alive indications from all supervised entities (Runnables of application)
  - Derives Global Supervision Status
  - triggers WdgIf if Runnables behave like expected
- WdgIf
  - triggers Hardware Watchdog(s)
- WdgDrv
  - Internal or external Watchdog driver

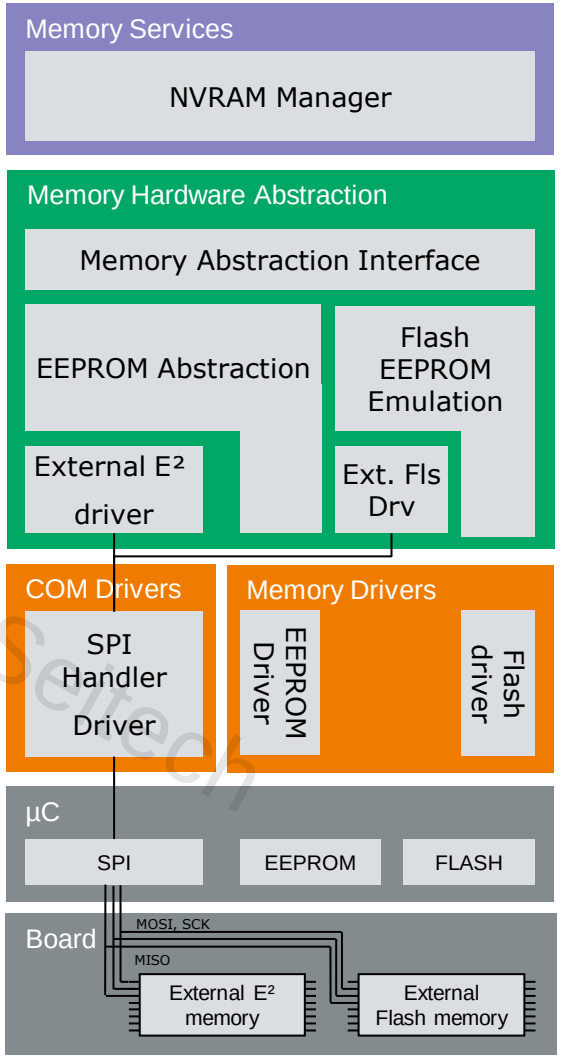


cw_26 2023 ; VI-S CEA Seitech

Memory Services

- Abstracts the locations of internal (on-chip) and external (on-board) memory devices.
  - Same mechanisms for internal and external EEPROM or Flash possible

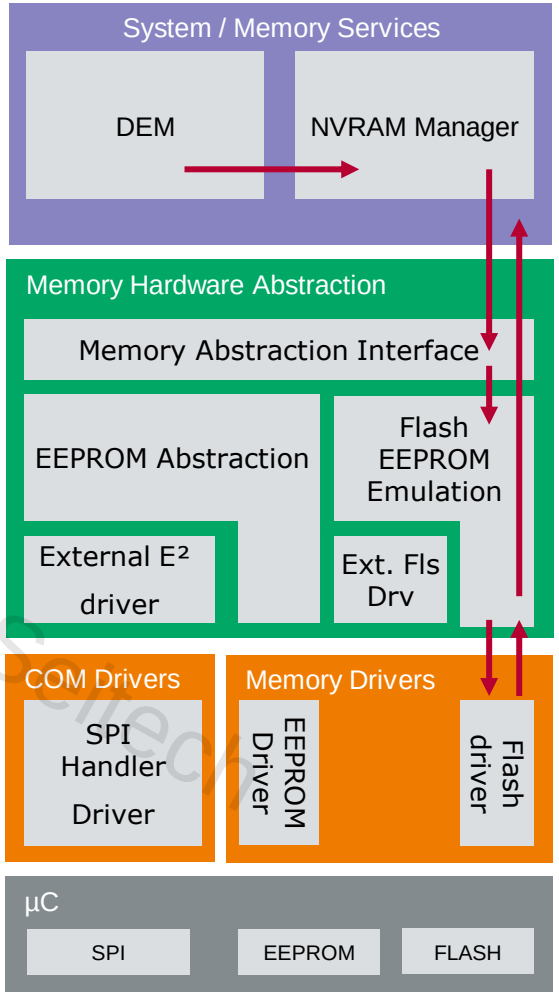
- Architectural subdivision into
  - Service Module: NVRAM manager
  - ECU Abstraction: Memory abstraction interface
  - MCAL Layer: Memory drivers



01_26 2023 : VI-S CEA Setech

Memory Services

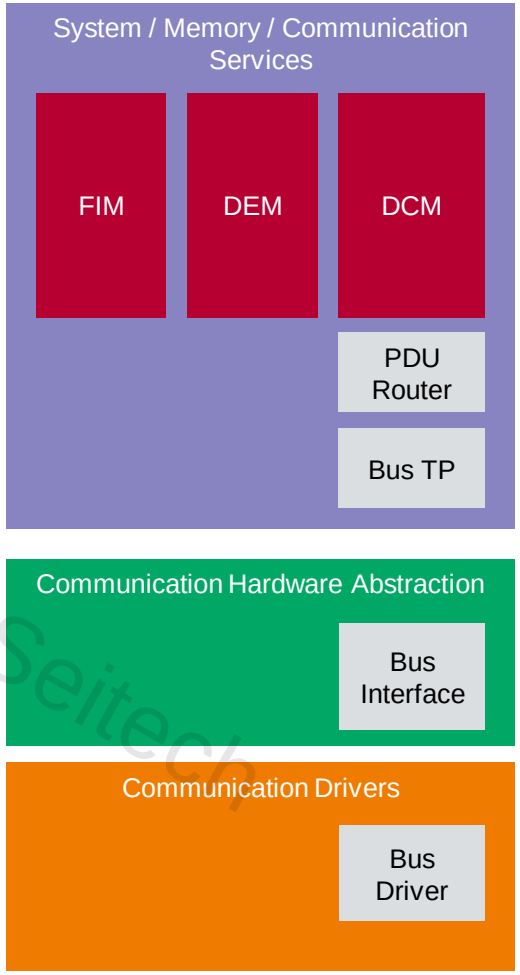
- Writing diagnostic snapshot data to NVRAM
  - Write the data as block with ID.
  - Write to specific address.
    - Memory type (Flash/EEPROM) unknown in NVRAM Manager
  - It is known from the configuration to which memory abstraction the write access should be passed.
  - Access is passed to the driver for internal memory or the driver for external memory.
  - The driver is called periodically to actually write to the memory.
  - Calls the callback of the abstraction when it is finished.
  - Abstraction notifies the NVRAM Manager.



cw_26_2025 ; VI-S CEA Schneider

Diagnostics

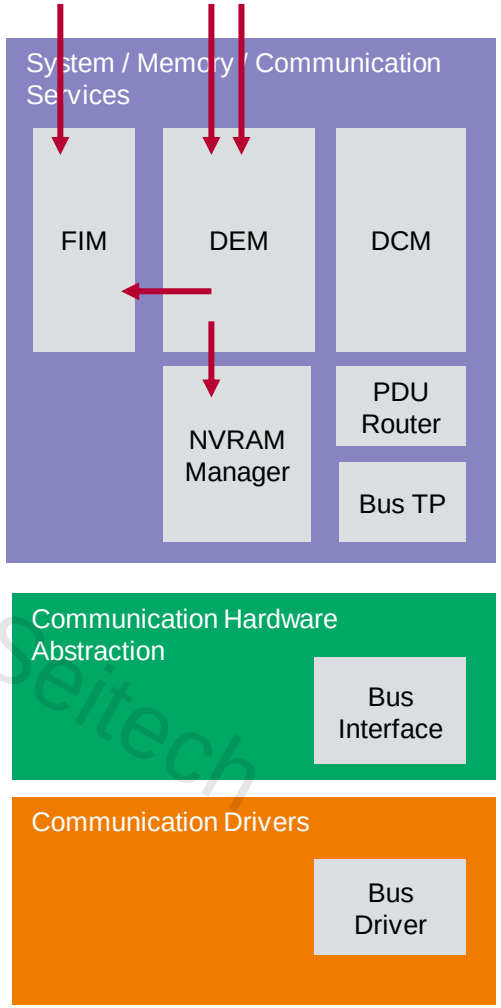
- Diagnostic Communication Manager (DCM)
  - Implements the diagnostic communication protocol.
- Diagnostic Event Manager (DEM)
  - Error memory
    - Saves diagnostic events
- Function Inhibition Manager (FIM)
  - Application functionality is executed conditionally.
    - Depending on error memory contents (diagnostic events)



cw_26 2023 ; VI-S CEA Seitech

Diagnostics

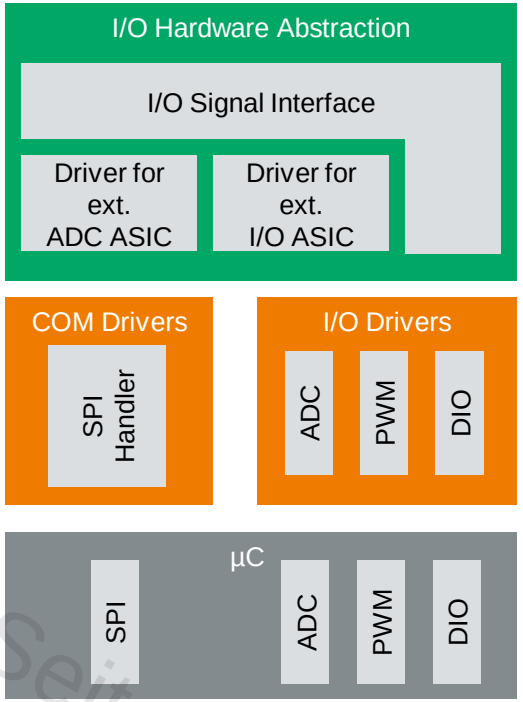
- Error state detection and saving in error memory.
  - Snapshot is saved
  - Event associated with error is set.
  - Associated NVRAM block is written.
- Shutdown of a function due to error memory entries.
  - FIM is notified that certain events are set in DEM.
  - In case of error, a SWC is inhibited by Polling by the SWC.



cw_26_2023 : VHS CEA Seitech

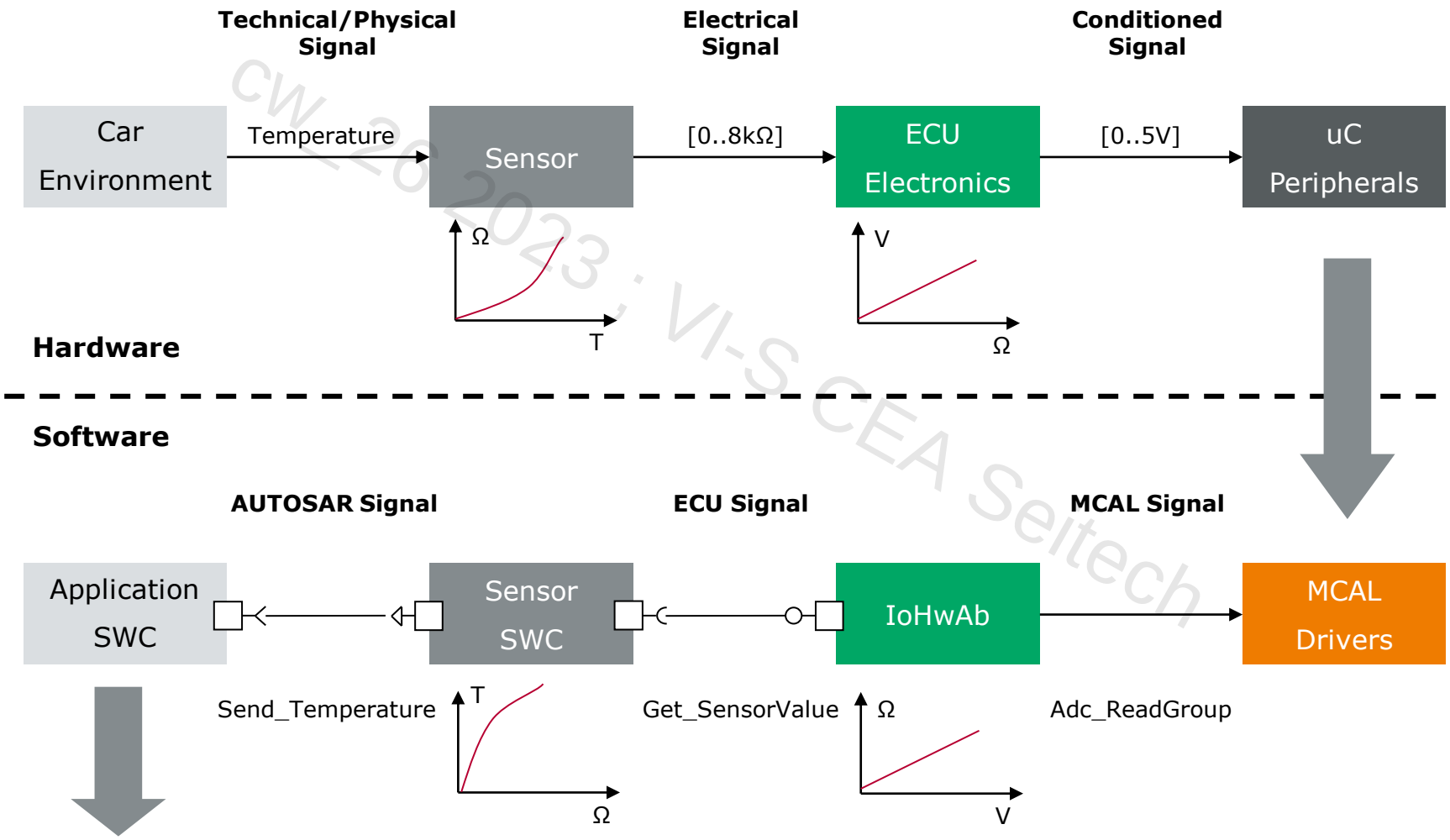
Hardware I/O

- Abstraction of location of I/O peripheral devices (on-chip / on-board) and layout
- No abstraction of sensor/actuator
  - I/O signal interface
    - Conditioning of the read values of inputs.
      - e.g. debouncing of signals
  - Drivers
    - ADC, DIO, PWM



cw_26 2023 ; VI-S CEA Seitech

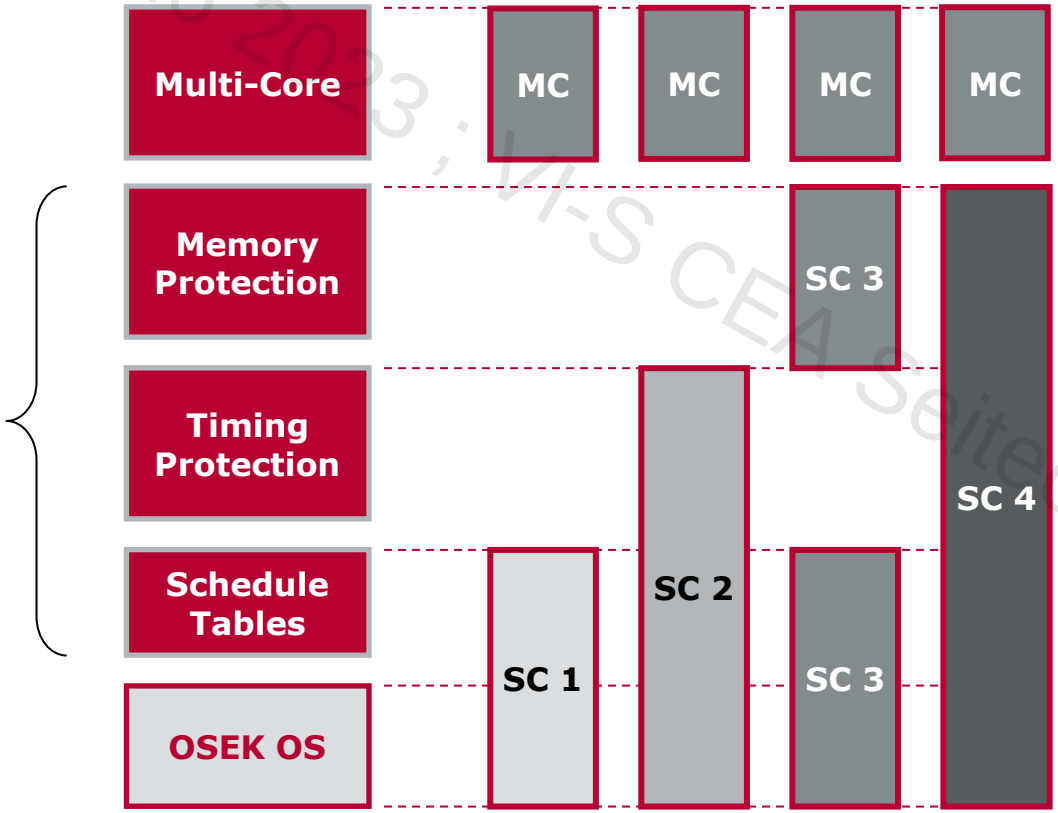
Hardware I/O



cw_26_2023 : VI-S CEA Seitech

New Functionality in Scalability Classes

- AUTOSAR **extends** the OSEK/VDX Operating System standard
- AUTOSAR OS add-ons are segmented into Scalability Classes (SCs)
- Multi-Core support is optional and independent of Scalability Class



CW_26_2023 ; M-S CEA Seitech

Agenda

Overview and Objectives

AUTOSAR Application

AUTOSAR RTE

AUTOSAR BSW

▶ **AUTOSAR Methodology**

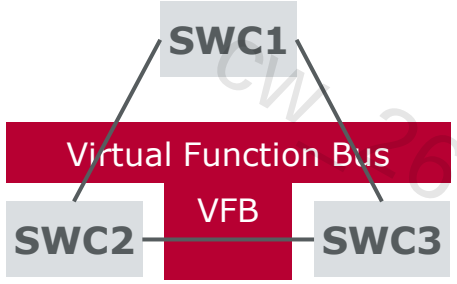
AUTOSAR in Practice

Appendix

cw_26 2023 ; VI-S CEA Seitech

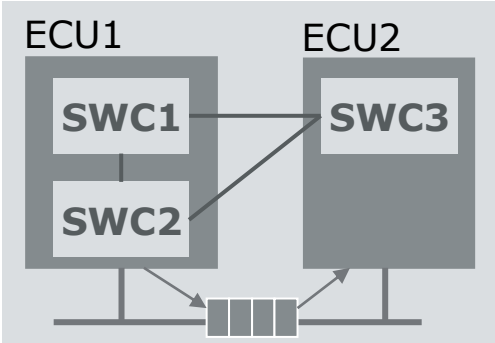
AUTOSAR Workflow

Complete SW functionality of the vehicle is defined as a system of SWCs...



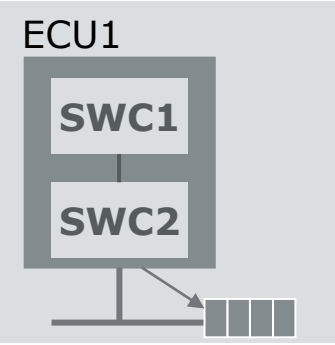
Software Component Description*

..and distributed to ECUs



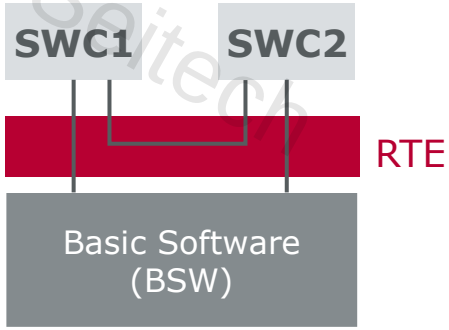
System Description*

An extract is created for each ECU...



Extract of System Description*

The ECU is configured based on the ECU extract.



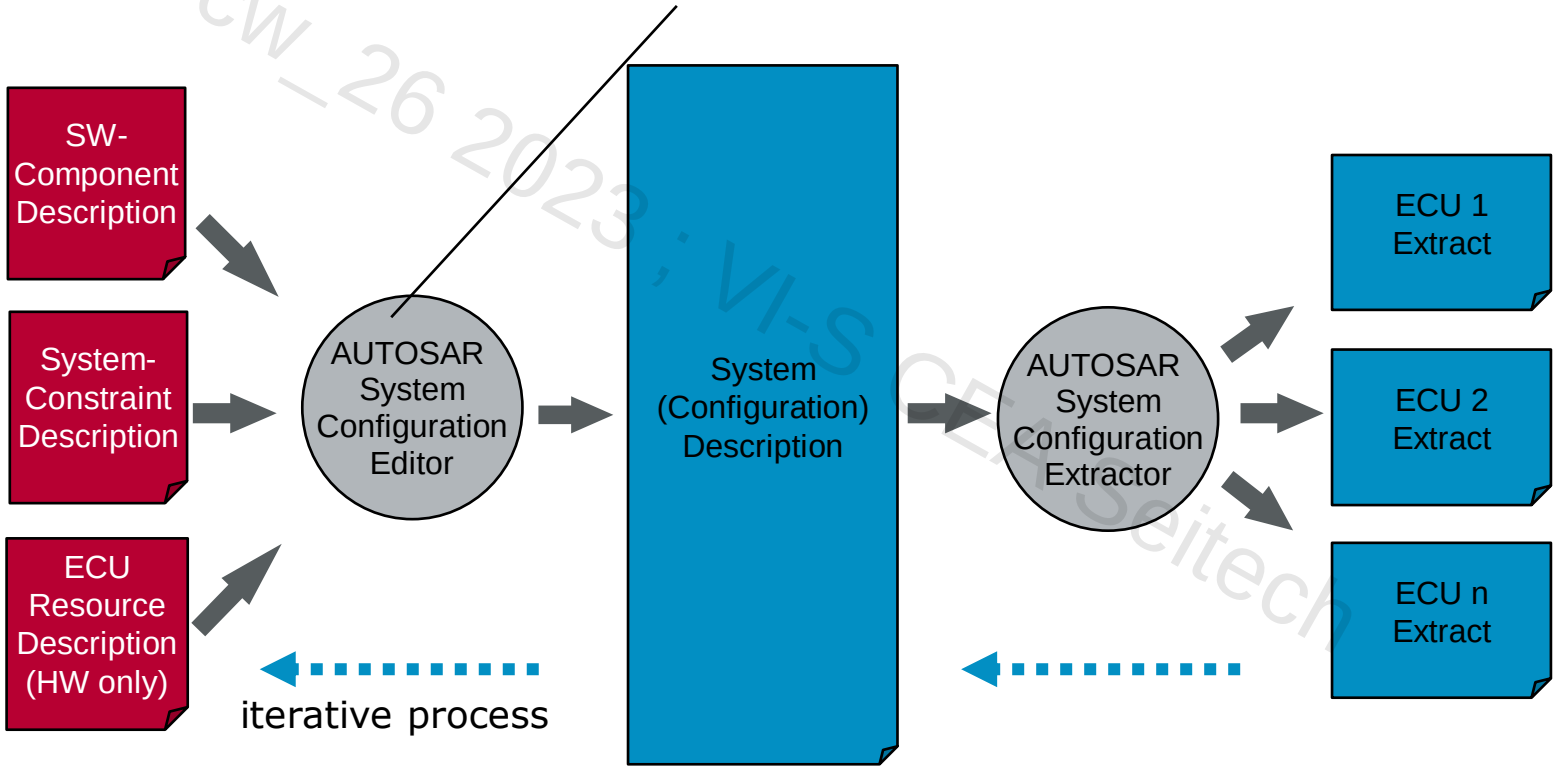
ECU Configuration Description (ECUC)*

cw 26 2023 ; VI-S CEA Seitech

System Description

Decisions:

- mapping of signals to K-matrix
- mapping SWC to ECU

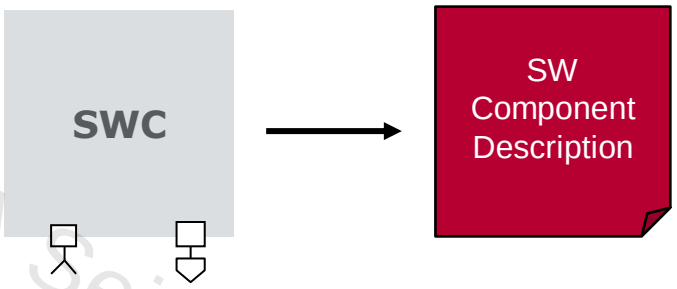


cw_26 2023 ; VI-S CEA Seitech

Software Component Description

- The AUTOSAR “Software Component Description” contains the following information:
  - Data and operations which are part of the component.
  - Resource needs of the component (memory, CPU time, etc.).
  - Information relating to the specific implementation (repetition rate).
  - Component interfaces.

- Structure and format are described by the “Software Component Template”.



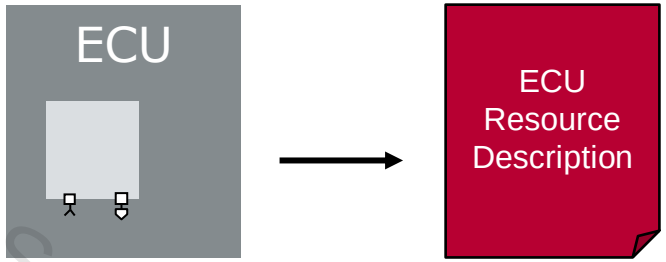
cw_26_2023 ; VI-S CEA Seitech

ECU Description

- The AUTOSAR "ECU Resource Description" contains the following information:

- Description of the hardware being used
  - Sensor, actuator
  - Memory
  - Processor
  - Communications periphery
  - Pin assignments

- The structure and format are described by the "ECU Resource Template".

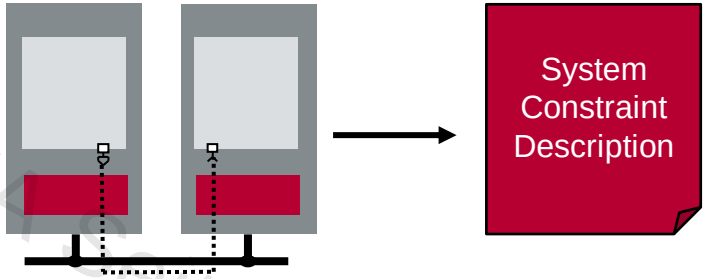


cw_26 2023 ; VI-S CEA Seitech

System Description

- The AUTOSAR “System Constraint Description” contains the following information:
  - Information on network topologies
  - Limitations (“Constraints”)
  - Protocols
  - K-matrix
  - Baud rate and timing parameters

- Structure and format are described by the “System Template”.

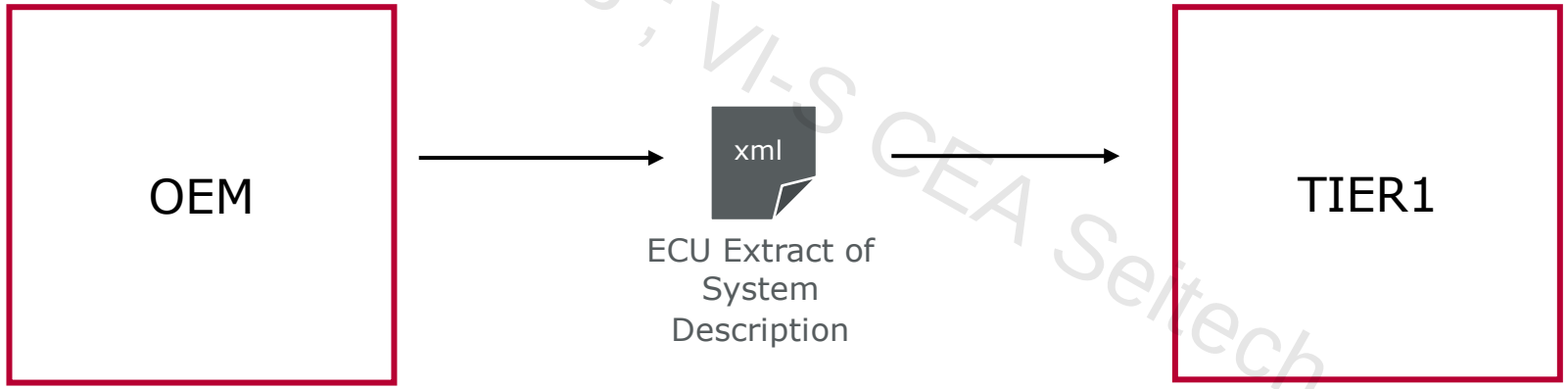


cw_26 2023 ; VI-S CEA scitech

AUTOSAR Workflow with OEM and TIER1

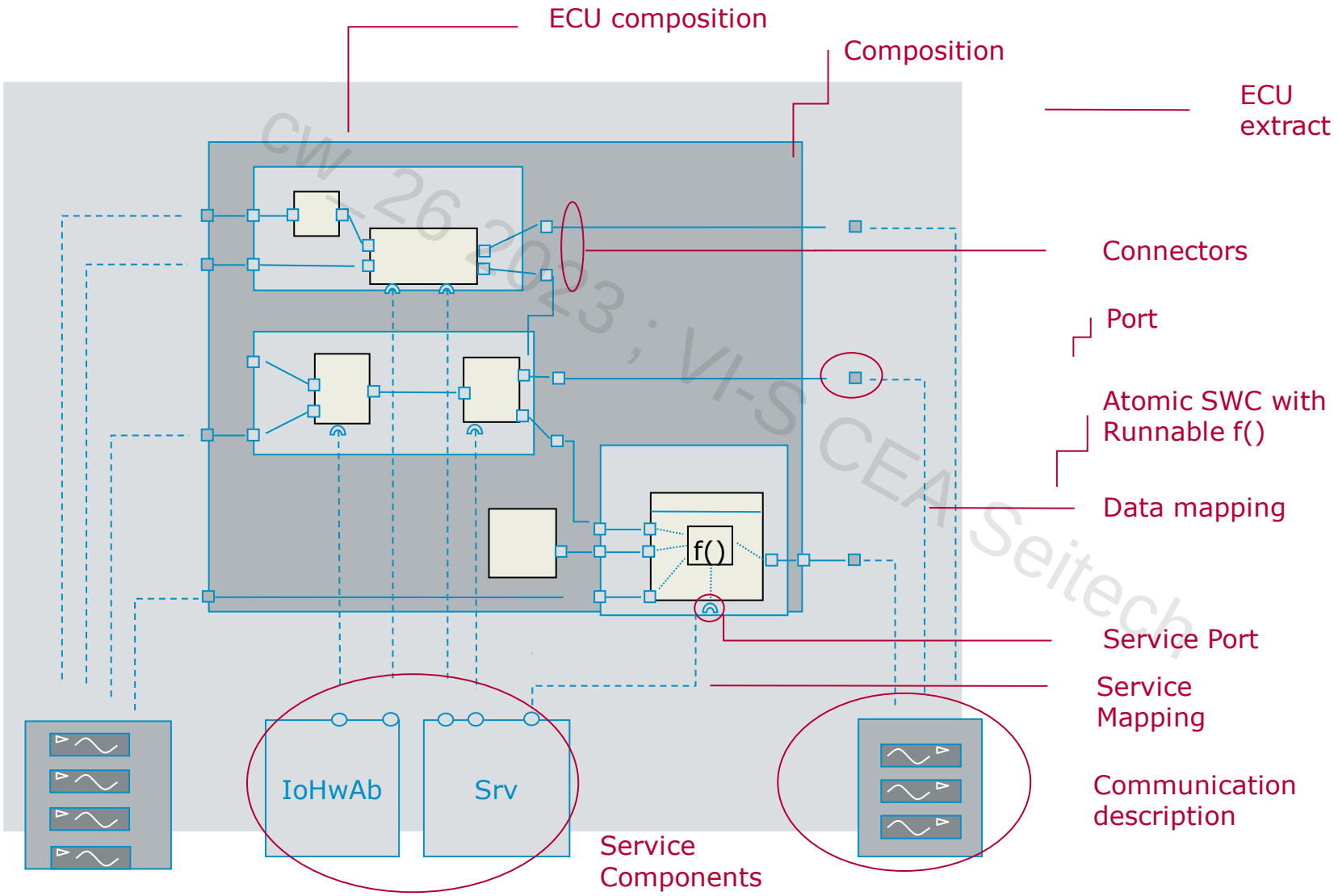
- Interface between OEM* and TIER1*: ECU extract of System Description (ECUEX)
- OEM creates ECUEX based on vehicle system design
- TIER1 configures AUTOSAR ECU based on ECUEX

cw_26 2023 ; VI-S CEA Seltech



*Note: “OEM” and “TIER1” may also be organizational units within one company (“System Responsible” vs. “ECU responsible”)

Content of ECUEX



cw 26 2023 ; VI-S CEA Seitech

Content of ECUEX

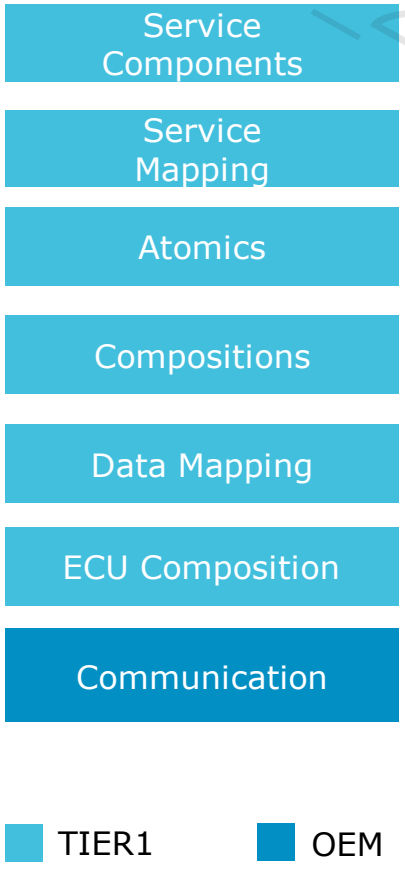
- Communication description of the ECU
  - Tx-/Rx-Frames, IDs, network signals
  - Send types
  - PDUs
- SWCs, Ports (grouped within a central ECU composition)
  - Atomic SWCs, Runnables
  - Compositions
- Port Interfaces, Data Types, Constants
- Connectors between the SWCs
- Data Mapping (connecting SWCs with network signals)
- Service Components (providing interfaces of the actually used BSW services for the SWCs – need to match with the ECU Configuration)
- Service Mapping (connecting SWCs with Service Components)

cw_26_2023 ; VI-S CEA Seitech

AUTOSAR XML format allows also ECUEX files with only a subset of this content!

Use Case 1

- Use Case 1: Network-oriented approach (same process as today, only with new AUTOSAR XML format instead of DBC/FIBEX/LDF).

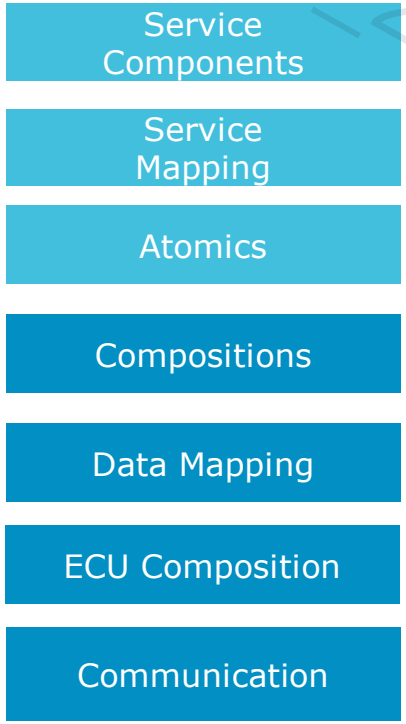


- OEM defines network communication
- TIER1 implements all ECU functions by
  - ECU Composition
  - Data Mapping
  - Compositions
  - Atomics
  - Service Components/Mapping

cw 26 2023 V1 6 CEA Seitech

Use Case 2

- Use Case 2: Functional system design approach, top-level architecture of ECU functional SW specified by OEM.



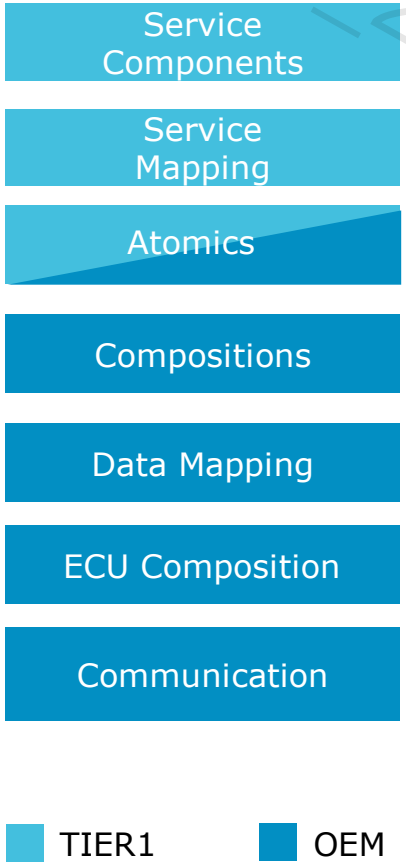
- OEM defines network communication
- OEM defines ECU function interfaces within the ECU by
  - ECU Composition
  - 1 Composition for each ECU function ("empty" composition – only ports, no substructure)
  - Connectors between the Compositions
  - Data Mapping
- TIER1 implements the ECU functions by
  - Atomics
  - Service Components/Mapping

TIER1 OEM

cw 26 2023, VTS CEA Seitech

Use Case 3

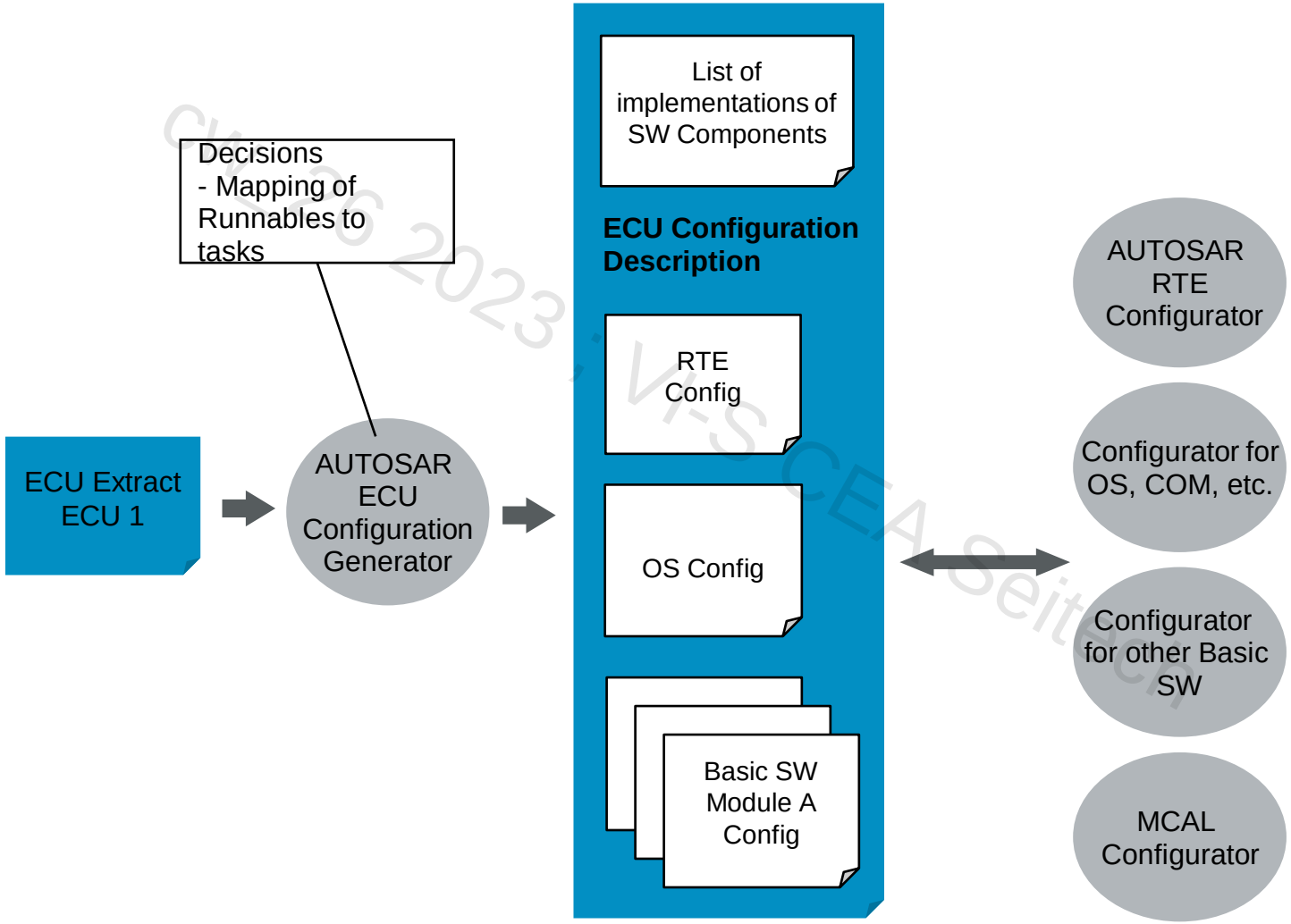
- Use Case 3: Functional system design approach, top-level architecture of ECU functional SW specified by OEM, SW partially implemented by OEM.



- OEM defines network communication
- OEM defines ECU function interfaces within the ECU by
  - ECU Composition
  - 1 Composition for each ECU function (Ports only, no substructure)
  - Connectors between the Compositions
  - Data Mapping
- OEM provides implementation of some of the ECU functions by
  - Atomics
- TIER1 implements the remaining ECU functions by
  - Atomics
  - Service Components/Mapping

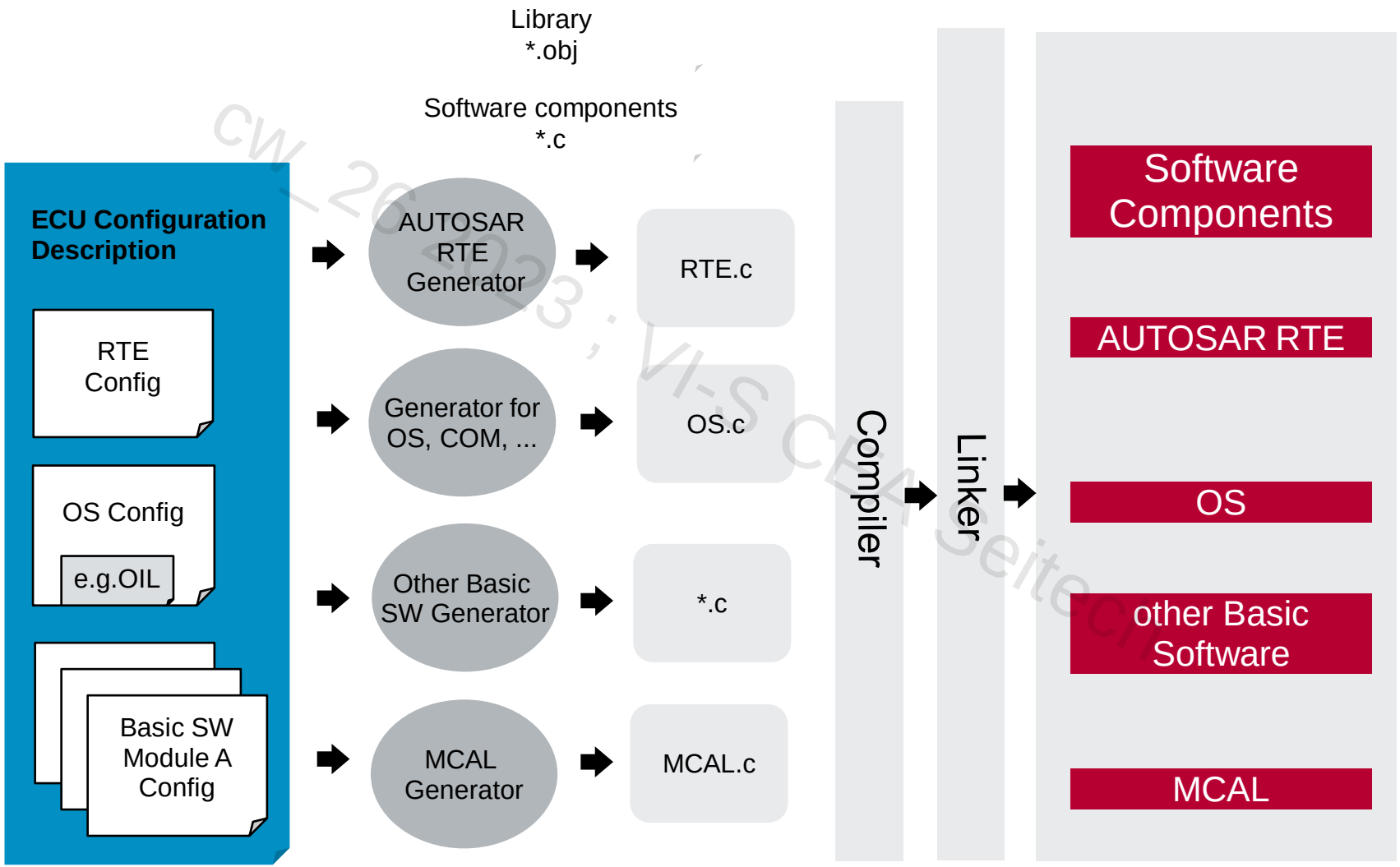
cw 26 2023 ; VI-S CEA Seitech

ECU Configuration



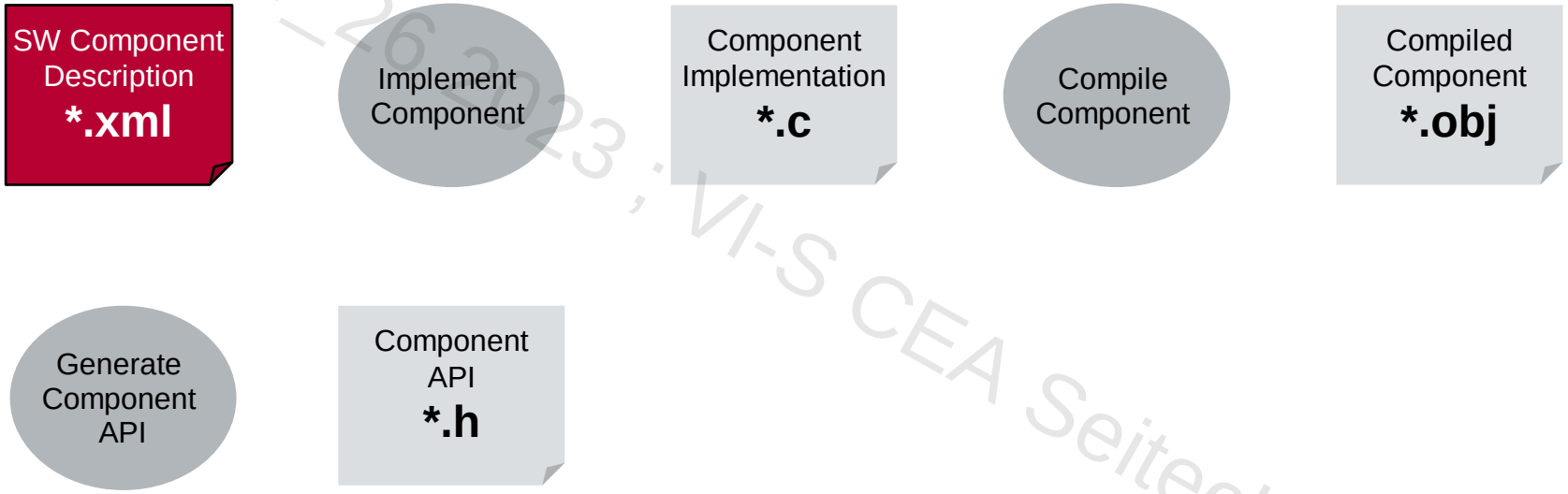
CVC 06 2023 ; VKSCEA Seitech

Executable ECU



cw_26_2023 ; VI-S CEA Seitech

Component Development Process



cw_26_2023 ; VI-S CEA Seitech

Agenda

Overview and Objectives

AUTOSAR Application

AUTOSAR RTE

AUTOSAR BSW

AUTOSAR Methodology

▶ **AUTOSAR in Practice**

Appendix

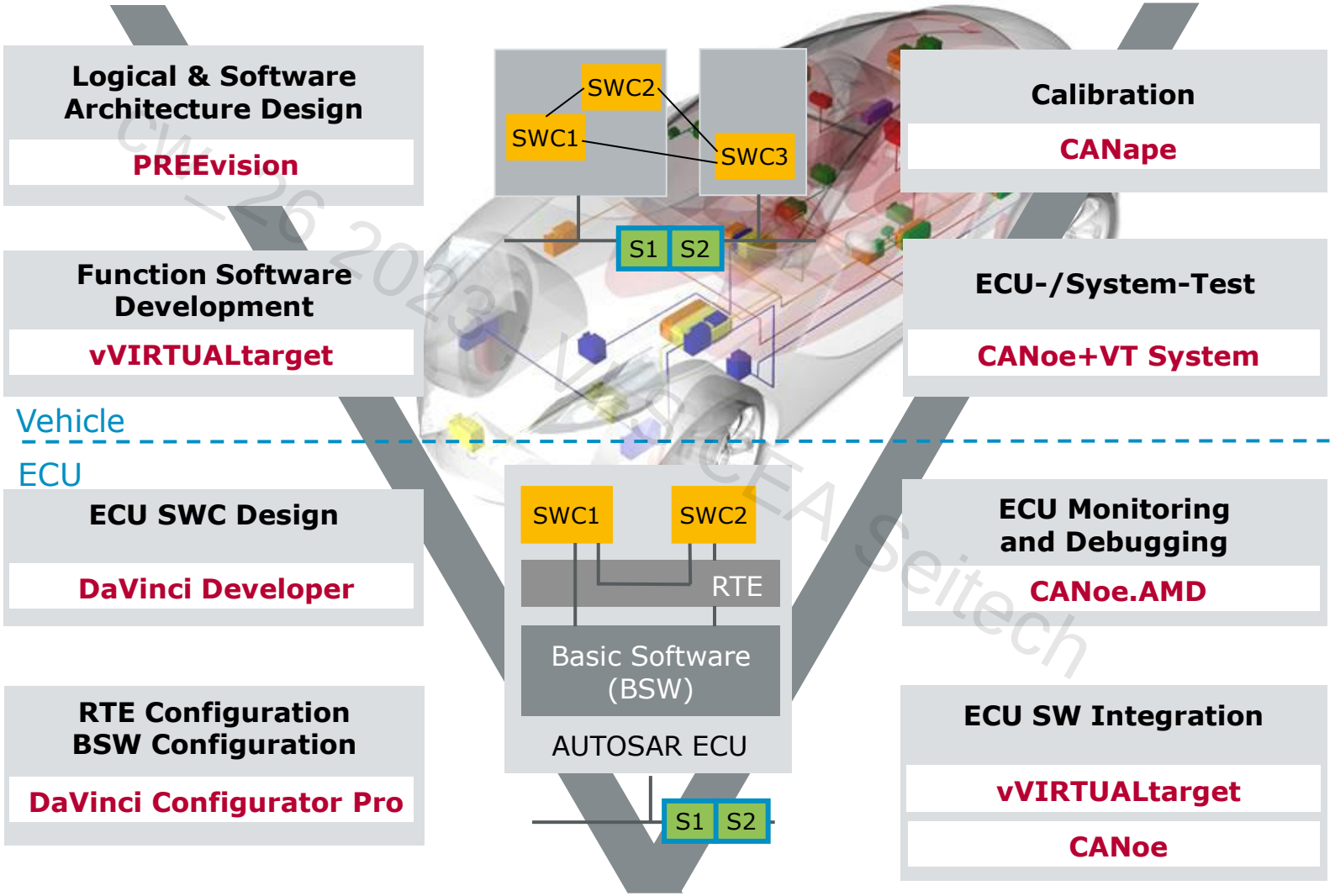
cw_26 2023 ; VI-S CEA Seitech

Exchange Formats

- System Extract / ECU extract of System Description (SYSD/ECUEX)
  - AUTOSAR XML format
  - Contains communication data of one ECU → used instead of DBC, LDF or FIBEX, for configuring the CAN, LIN and FR stack
  - May contain AUTOSAR SWCs
  - This format is created by System Design tools, e.g. PREEvision, or in-house database systems of the OEM
- Basic Software Module Description (BSWMD)
  - AUTOSAR XML format
  - Contains a detailed description of all available parameters of the BSW module. Configuration tools use this information to build up a configuration (ECUC)
- ECU Configuration Description (ECUC)
  - AUTOSAR XML format
  - Contains the module configurations (i.e. the concrete parameter values) of all BSW modules and the RTE
- Software Component Description (SWC description)

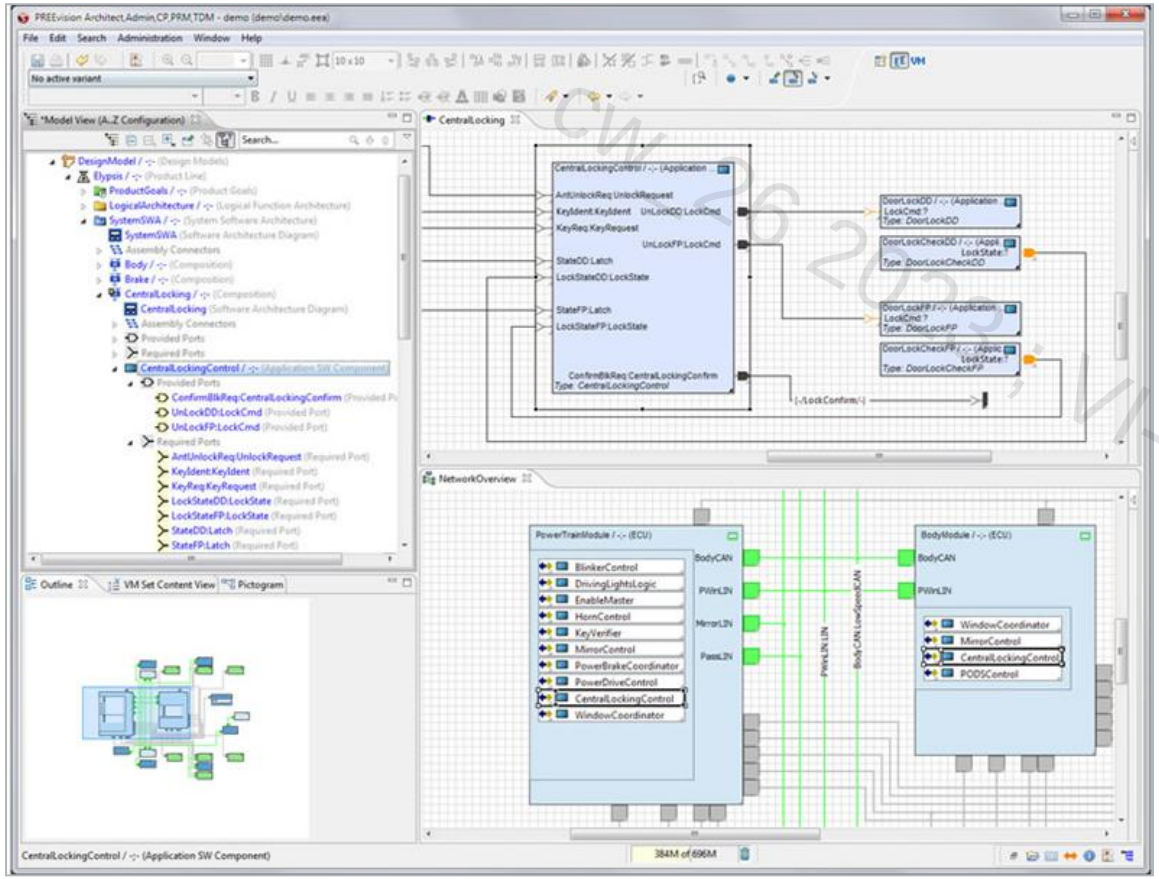
CW 26 2023 : VI-S CEA Seitech

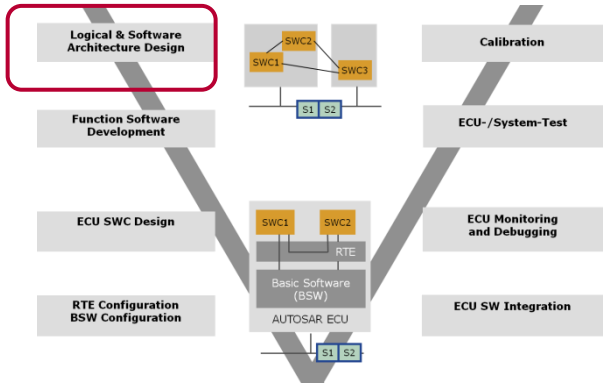
AUTOSAR Tool Chain



cw 26 2023 VDA/ETA Seitech

PREEvision System Design





- System design of vehicles or ECUs
  - Software components
  - Network communication
  - Mapping
- Supports also various architecture levels beyond AUTOSAR, e.g.
  - Requirements
  - Electrical Wiring Harness

CW 26 2023, IKS CEA Seitech

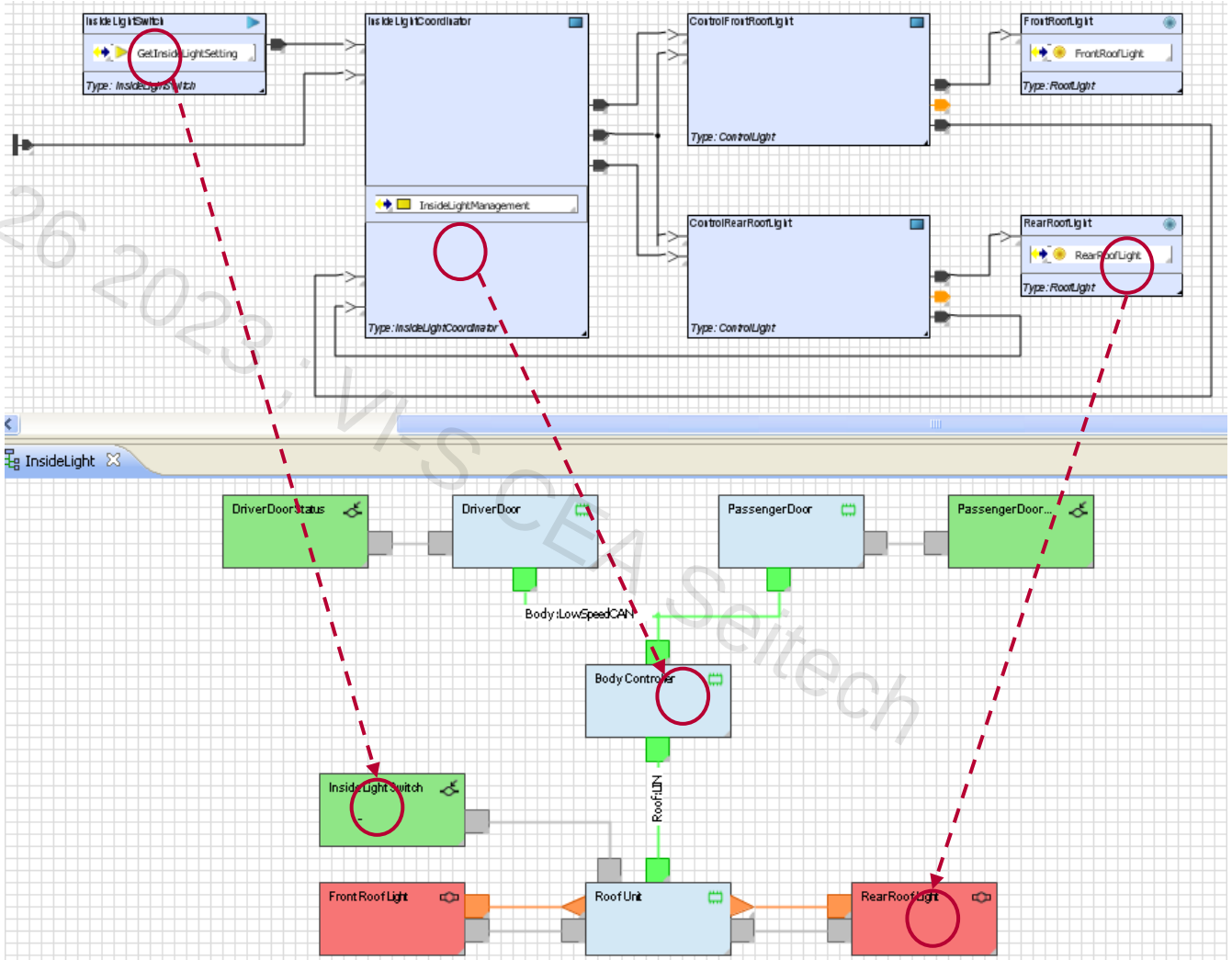
PREEvision System Design

Software Architecture

ROUTE

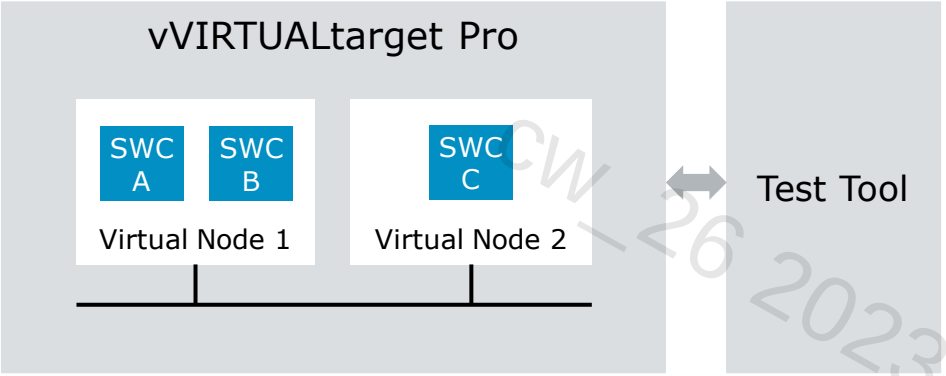
MAP

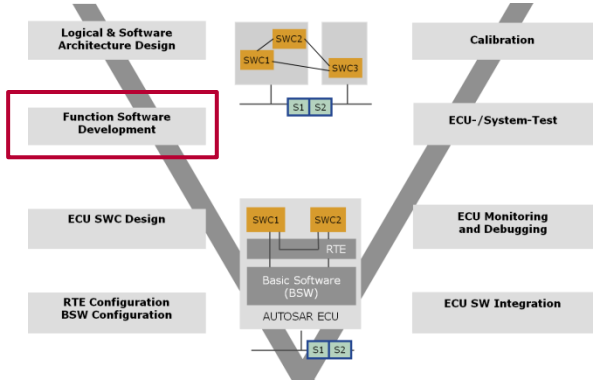
Component Network



cw_26 2020 VRS CEA Seitech

vVIRTUALtarget Pro

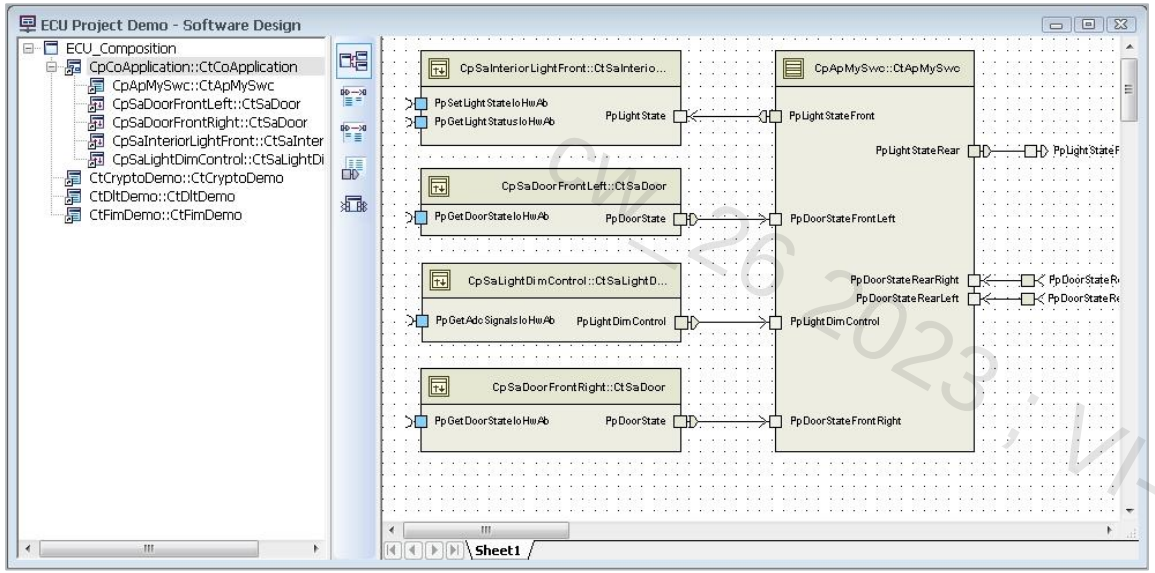


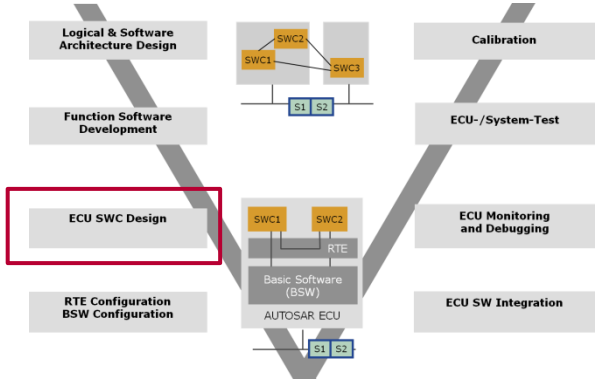


cw_26 2023 ; VI-S CEA Seitech

- Validation of distributed functions
  - Run SWCs in a virtual environment
  - Realistic simulation of AUTOSAR ECU behavior and network communication
- CANoe as test tool
- API to connect other test tools

DaVinci Developer

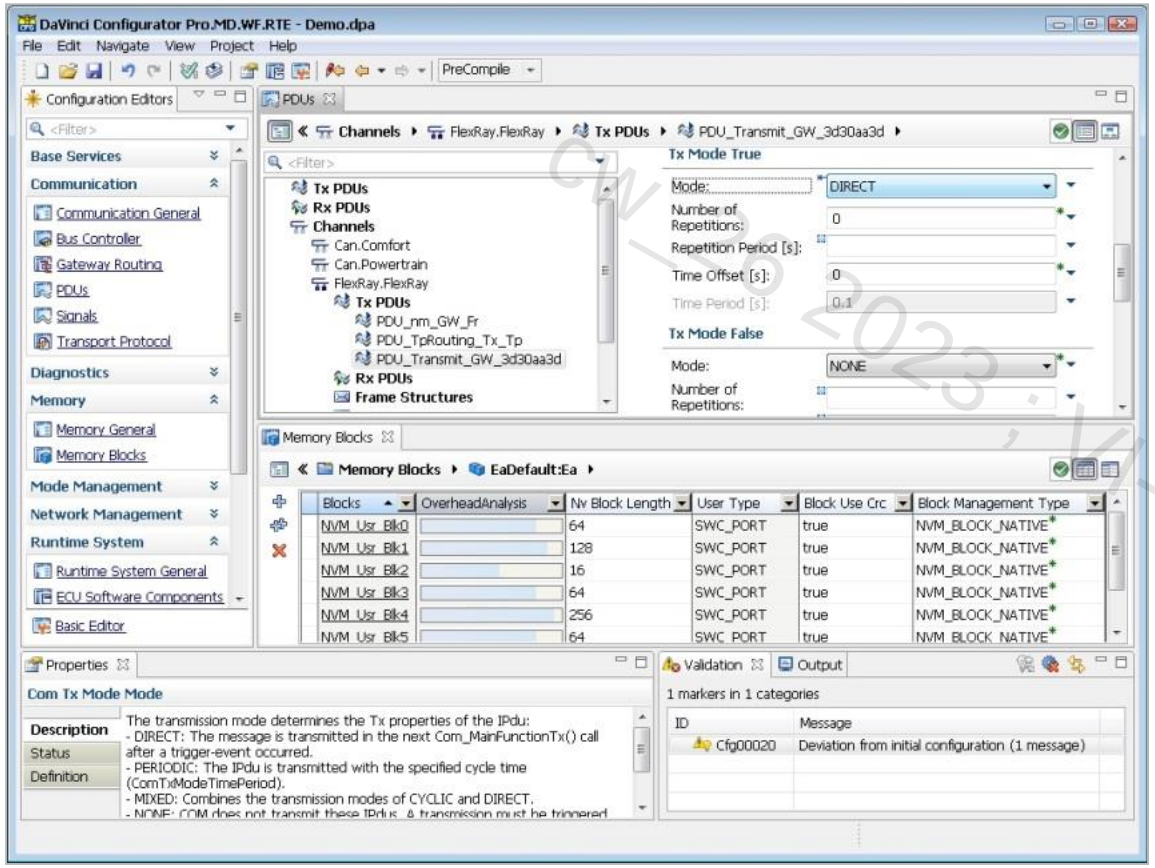


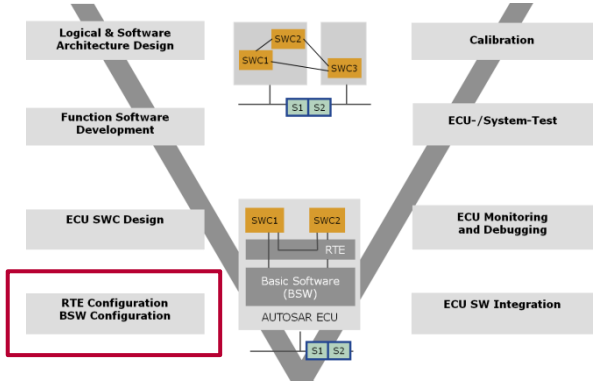


CK 26 2023 . VKS CEA Seitech

- Creation of SWC descriptions with graphical or table-based editors
- Definition of SWC internal behavior (Runnable entities)
- Consistency check of the SWCs
- Data exchange analysis

DaVinci Configurator Pro

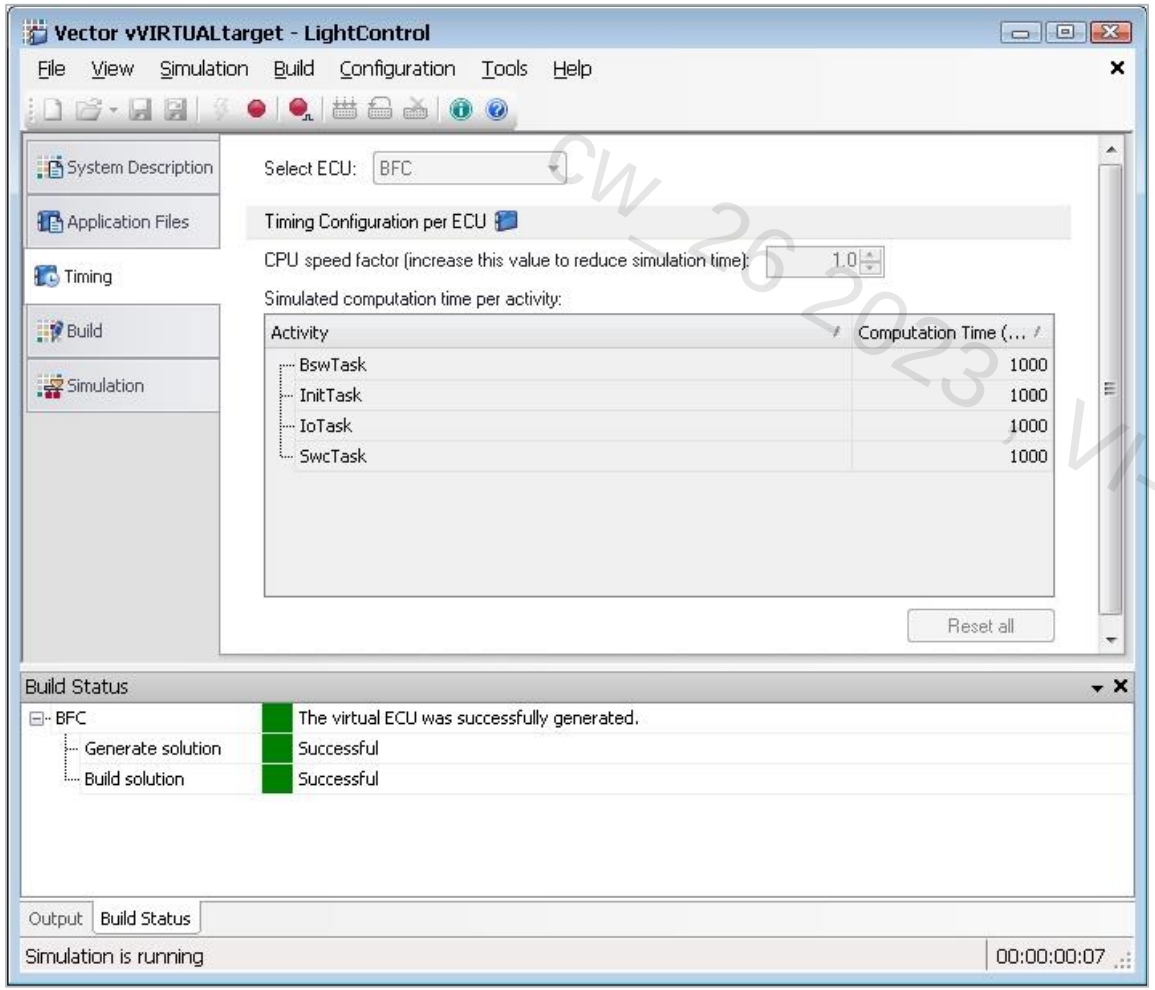


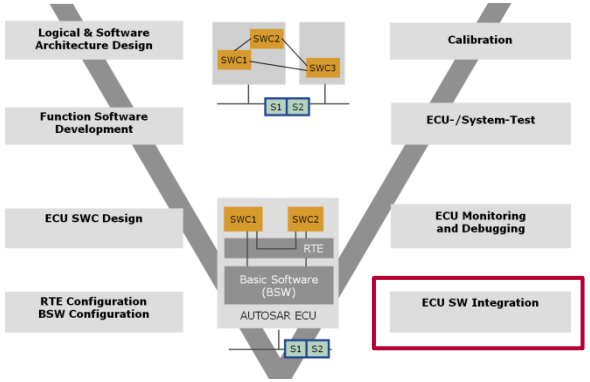


- Creation of ECU configurations
  - MICROSAR BSW and RTE
  - Third party BSW (MCALs)
- Validation function incl. “click-to-solve”
- Use case specific editors and assistant functions
- Smart project update function
- Generation of configurable part of BSW and RTE

CW_10 2023 ; KS CEA Scitech

DaVinci Configurator Pro.VTT + vVIRTUALtarget

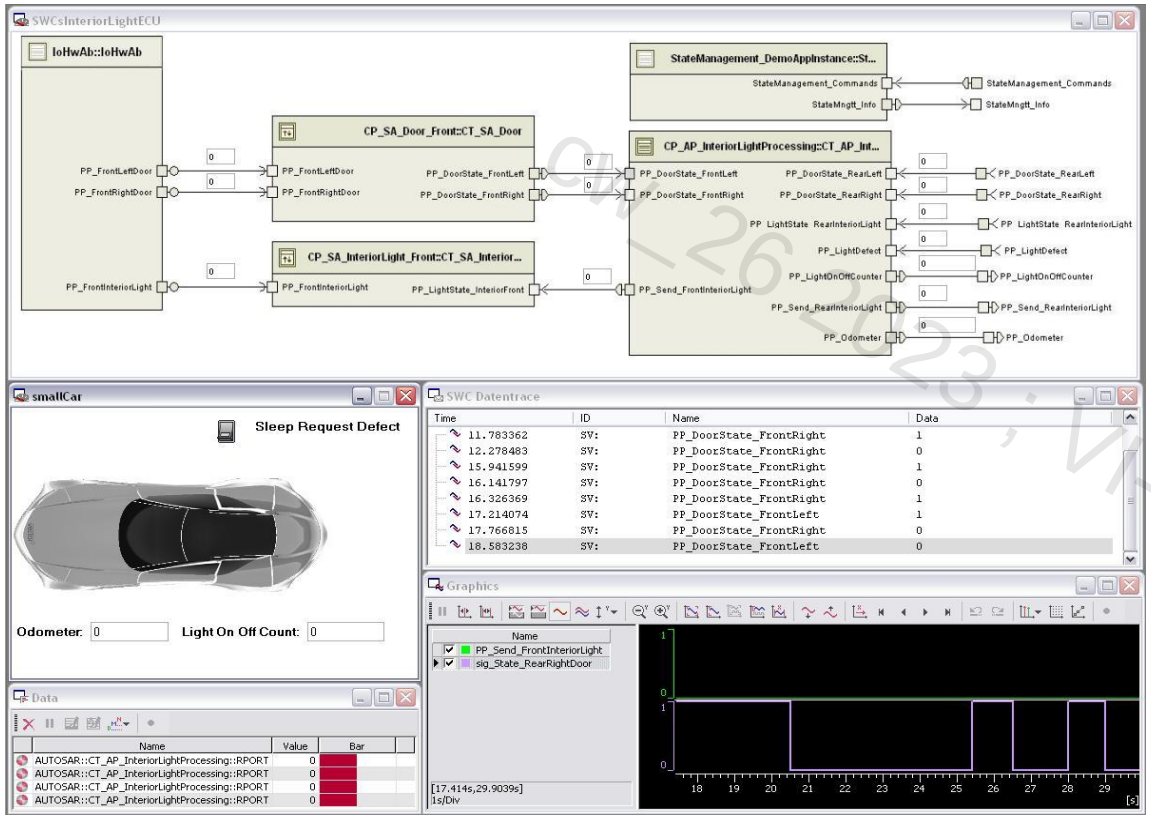




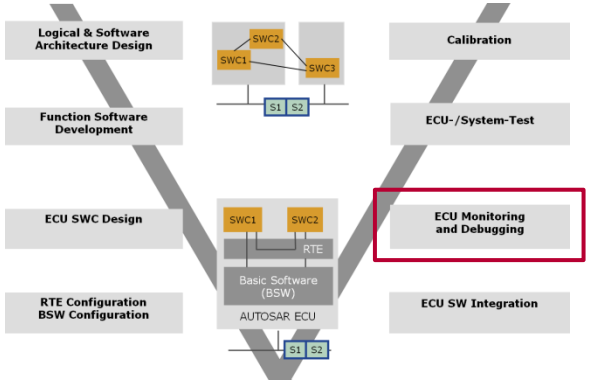
- Integration of ECU SW in a virtual environment
  - Single-source ECU configuration for real ECU and virtual target
  - Execution of ECU code on the computer with vVIRTUALtarget
- CANoe or other tools as test tool
- Debugging of ECU code on the computer

CW_26_2023 WS CEA Sophia

CANoe.AMD



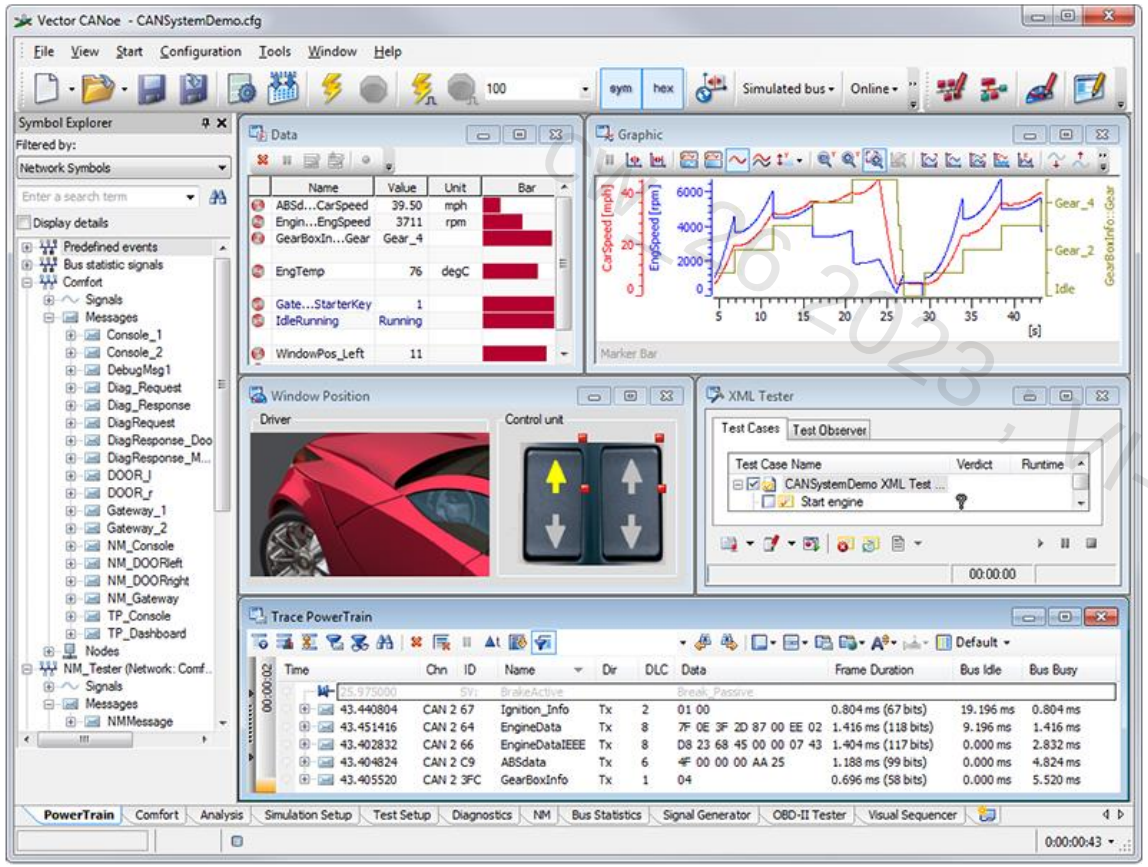
AMD: AUTOSAR Measurement and Debugging

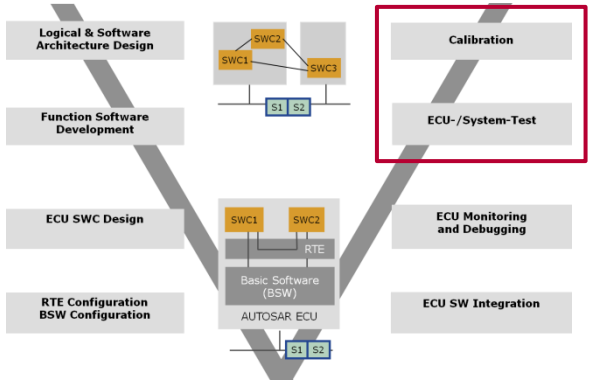


- Real-time access to ECU status
- Runtime measurement of SWC code
- Notification of ECU internal errors and events
- XCP-based data access by CANoe via network channels or calibration interface

CM_26_2023, VFS CEA Seitech

CANoe/CANape





- ECU Test by stimulation and measurement of ECU interfaces
  - Bus Communication
  - I/O
- Calibration of ECU parameters

Copyright 2023, MS CEA Sitech

Agenda

- Overview and Objectives
- AUTOSAR Application
- AUTOSAR RTE
- AUTOSAR BSW
- AUTOSAR Methodology
- AUTOSAR in Practice
- **Appendix**

cw_26 2023 ; VI-S CEA Seitech

Documents for Reference

- Introduction
  - AUTOSAR_TechnicalOverview.pdf
  - AUTOSAR_LayeredSoftwareArchitecture.pdf
- Methodology
  - AUTOSAR_Methodology.pdf
- RTE
  - AUTOSAR_SWS_RTE.pdf
  - AUTOSAR_SoftwareComponentTemplate.pdf
- BSW
  - AUTOSAR_BasicSoftwareModules.pdf

- For additional information go to:
  - http://www.autosar.org or http://www.vector.com

cw_26 2023 ; VI-S CEA Seitech

List of Acronyms/Abbreviations 1

- API — Application Programming Interface
- ASAM MCD — Association for Standardization of Automation- and Measuring Systems Measurement, Calibration and Diagnostics
- AUTOSAR — Automotive Open System Architecture
- BSW — Basic Software
- Bus — Bus stands for CAN, FlexRay, LIN, Ethernet or TTCAN
- CAL — Communication Abstraction Layer
- CAN — Controller Area Network
- CCP — CAN Calibration Protocol
- COM — Communication Layer
- ComM — Communication Manager
- CPU — Central Processing Unit
- ECU — Electronic Control Unit
- EcuM — ECU Manager
- HAL — Hardware Abstraction Layer

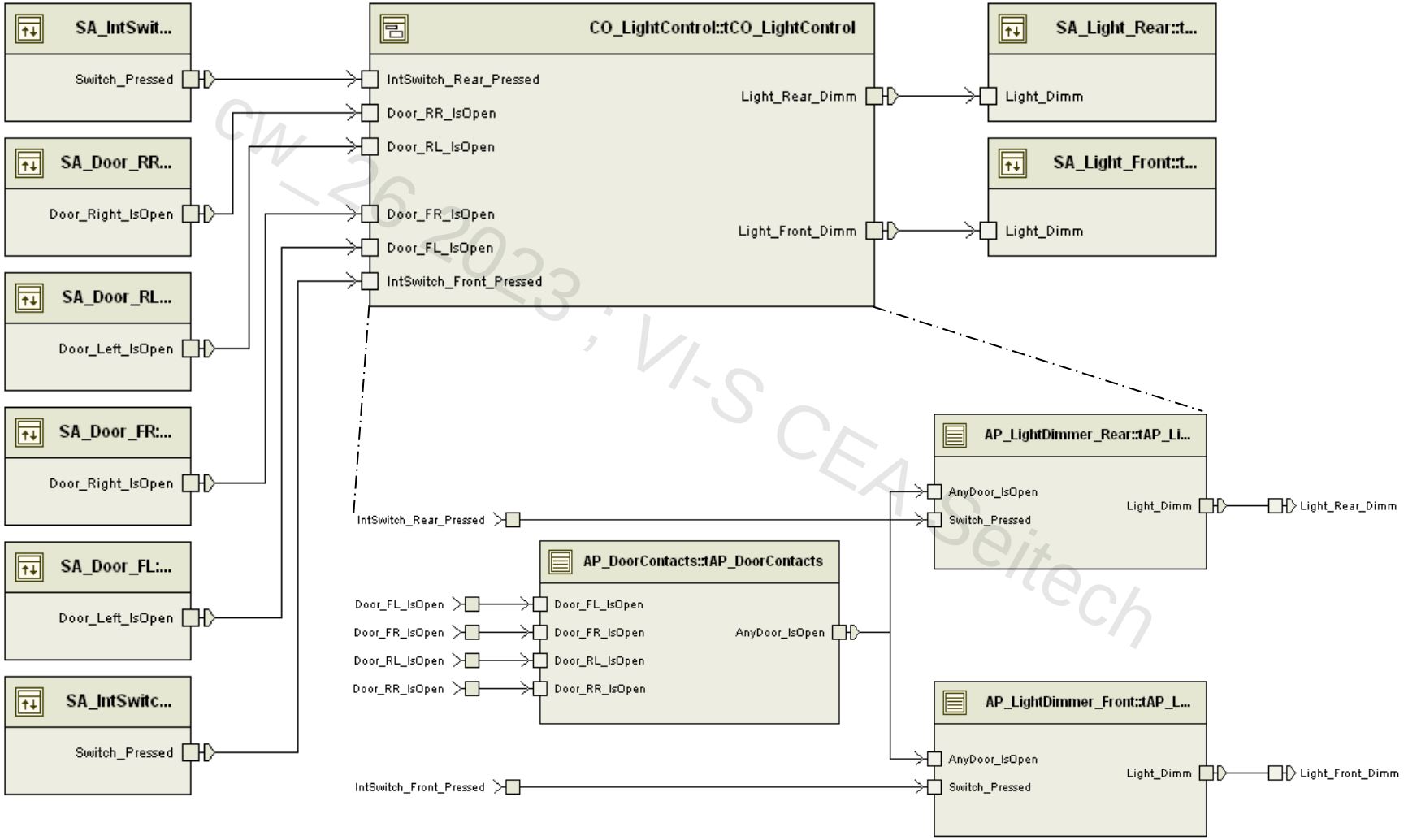
CW_26_2025 : VI-S CEA Seitech

List of Acronyms/Abbreviations 2

- HIS — Hersteller Initiative Software [“OEM Initiative Software”]
- I-PDU — Interaction Layer Protocol Data Unit
- MCAL — MicroController Abstraction Layer
- NM — Network Management
- OEM — Original Equipment Manufacturer
- OS — Operating System
- PDU Router — Protocol Data Unit Router
- RTE — Runtime Environment
- SPAL — Standard Peripheral Abstraction Layer
- SW — Software
- SWC — Software Component
- TP — Transport Protocol
- VFB — Virtual Functional Bus
- XCP — Universal Measurement and Calibration Protocol
- XML — Extensible Markup Language

CW 26 2023 ; VI-S CEA Seitech

Solution to Exercise 1



CW_26_2023 ; VI-S CEA Seitech

For more information about Vector and our products please visit

www.vector.com

cw_26 2023 ; VI-S CEA Seitech

Author:
Vector Academy
Vector Germany

© 2021. Vector Informatik GmbH. All rights reserved. Any distribution or copying is subject to prior written approval by Vector. V1.0.0 | 2023-03-28